



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校

参赛队号

	1.
队员姓名	2.
	3.

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题目：通用神经网络处理器的多核切图与调度优化

摘要：

面向多计算管道和共享外存的神经网络处理器，本文在算子依赖约束下联合选择子图划分、核心映射与执行顺序，以官方仿真总完成时间为主要目标，同时间下优先减少额外数据搬运。模型按跨子图经 DDR、同核驻留与跨核同步、共享只读 FIFO 缓存三种机制逐层扩展。

对问题一，在共同的无环划分与事件释放模型下讨论两种求解方案。结构感知搜索以结构感知初解和受保护的有限评价机会求解；多候选精修组合粗细拓扑窗口等多种划分，以列表调度和官方模拟筛选，并作五核局部改进。百图五核平均加速比分别为 3.665903 和 4.026824，总周期分别为 73170501 和 68905422。多候选精修的候选与历史计算预算更大，故该结果表示交付质量比较，而非等预算消融结论。

对问题二，按跨核搬运完成后 500 周期释放建立事件约束，采用保护基础搜索的结构算法组合，并复用不变的候选生成准备。五核逐图平均加速比为 4.264407，总周期为 65342192。对问题三，加入共享 FIFO 状态和 DDR、L2 独立带宽池，在普通结构候选之外引入缓存事件邻域；五核平均加速比为 4.364503，总周期为 63759268。固定最终方案、仅切换 L2 时，相同核数的平均时间比为 1.030244，与跨方案综合比 1.027332 分开报告。

结构感知搜索及问题二、三均保存 100 图、1 至 5 核的逐配置官方结果；多候选精修亦已完成全部 500 组复核，完整核数曲线与五核逐图对照共同报告。结果表明，减少搬运或增加缓存命中并不保证缩短关键路径，应以完整事件仿真裁决。算法不穷举组合空间；冷启动大图压力核验与历史初解成本单列，尚不能把限时搜索结果解释为所有图端到端均在十分钟内完成，也未证明全局最优。

关键词：有向无环图划分；多处理器调度；离散事件仿真；共享缓存

1 问题重述

1.1 背景与任务

通用神经网络处理器以多个 AI 核执行计算图。每核具有矩阵计算、向量计算与数据搬运等独立执行通道，但核内存储有限，多个核心共享核外 DDR 带宽。原始计算图为带张量依赖的有向无环图；切分粒度决定可同时运行的工作量，也改变输入重复读取、跨子图数据搬运和缓存溢出。题目要求输出每个非 COPY_IN/COPY_OUT 算子的子图编号，以及各核心上的子图顺序，再由给定评估程序确定总执行周期和额外搬运量^[1]。

三个问题的决策形式相近，运行语义逐步改变。问题一的场景 A 规定一个子图就是一个 Task，子图边界上的数据总经 DDR 中转；即使两个 Task 在同一核上连续执行，也不保留私有缓存。问题二的场景 B 把同核各子图纳入一个 Task，允许核内驻留，但跨核数据需等待同步释放。问题三在场景 B 上增加全核共享的只读 FIFO L2，读请求可由 DDR 或 L2 服务。因而，问题二和三不能直接沿用场景 A 的通信成本公式，也不能把问题三的缓存命中率当作独立可选变量。

1.2 所需回答与指标

对每张图和每个核数 $k = 1, \dots, 5$ ，本文构造合法的切图、核心分配与核上顺序。首要评价量为官方仿真得到的完成时间 $T_{i,k}$ (cycles)，同时报告逻辑额外数据搬运量 $D_{i,k}$ (bytes)。设 $T_{i,1}^0$ 是题目规定的整图单核核内启发式结果，则前两问的逐图加速比与平均加速比定义为

$$S_{i,k} = \frac{T_{i,1}^0}{T_{i,k}}, \quad \bar{S}_k = \frac{1}{100} \sum_{i=1}^{100} S_{i,k}. \quad (1)$$

这是一百个比值的算术平均，不能换成总单核周期除以总多核周期。问题三另比较相同图、相同核数下的无 L2 与有 L2 周期，单核缓存收益由真实仿真决定，不人为设为 1。

1.3 研究范围

全部实验使用题目给定的 100 张计算图和固定硬件参数。本文不改变图的算子、张量或官方核内启发式；搜索仅改变题目允许的划分、映射和子图顺序。求解器运行秒数与被评估方案的执行周期是不同量，前者反映求解效率，后者是模型的主要目标。各问的逐例结果列于附录，所有整体结论均以同一百图为统计总体。

2 模型假设与符号说明

2.1 假设及适用依据

假设输入图与官方固定配置正确给出算子、张量、周期、依赖和硬件参数；算子不能被拆成更细的可调度原子，张量大小与算子周期在一次评价内确定。忽略 CPU 主控本身未由官方评价器计入的时间，不把网络传输、温度或动态频率引入模型。每个方案需满足题目规定的覆盖、无环、核内顺序、L1/UB 容量及资源占用约束。若容量压力触发溢出，新增 COPY 由原始核内启发式确定，不能由外层算法任意删除。

使用的 100 张图由原附件逐字节无损提取，没有删除异常图、归一化周期或调整张量大小。导入时对全部图执行官方结构校验，并保存源文件与每份模型输入的 SHA-256；所有问题读取同一处理目录。由此，三问结果的差异来自场景语义及合法方案，而非改写数据。前期挑选小图和反例用于调试与同预算对照；最终汇总只以完整百图为准，不把开发样本当独立盲测集。

2.2 固定硬件量

L1 容量为 524 288 bytes，UB 容量为 131 072 bytes，共享 DDR 总带宽为 60 bytes/cycle。场景 A 的跨核任务等待为 1000 cycles，同核连续 Task 间隔为 100 cycles；场景 B 的跨核 COPY 释放延迟为 500 cycles。问题三的共享只读 L2 容量为 1 048 576 bytes，带宽为 250 bytes/cycle。以上均是主实验的固定配置，灵敏度扫描仅用于解释机制，不替换正式结果。

2.3 统一符号

表 1 主要符号

符号	含义
$G = (V, E), V_c$	原计算 DAG 及其中的非 COPY 算子集合
$\pi : V_c \rightarrow \{1, \dots, q\}$	操作至子图的划分; q 为子图数
$m : \{1, \dots, q\} \rightarrow \{1, \dots, k\}$	子图至核心的映射; σ_c 为核心 c 上的合法顺序
$p = (\pi, m, \sigma)$	一份完整调度方案; $\mathcal{P}_A, \mathcal{P}_B$ 为对应场景的合法方案集
$T_i^A(p), T_i^B(p), T_i^{L2}(p)$	第 i 图在三种语义下的官方完成周期
$D_i(p)$	相对原始图 COPY 总量的逻辑额外搬运字节数
$Q(t), C, b_D, b_L$	时刻 t 的共享 FIFO 队列、L2 容量、DDR 与 L2 带宽
h_e, s_e, a_e, f_e	COPY 事件 e 的命中标志、字节数、发射和完成时刻

3 问题分析与总体路线

3.1 关键权衡

合并强依赖算子可以减少边界 COPY、重复读取和任务释放等待，但过度聚合会压缩多核并行空间，并可能让核内启发式在 L1/UB 容量压力下插入更多换入换出。过度切分则使共享 DDR 竞争、同步及启动间隔增加。因此，通信字节数、任务数或各核算子数都不是总完成时间的充分代理。若 Cube 与 Vector 工作量不均，按算子个数平衡也会让其中一条 Pipe 长期空闲。论文以合法方案的官方 Makespan 为最终准则，将成本估计只用于有限候选的生成和排序。

三问共同采用“结构候选—依赖调度—官方评价—有界改进”的框架。先识别计算依赖和共享输入，构造完整连通分量、主导分量有界拆分等不同粒度的合法初解；再按后续关键工作量和最早可行完成时间安排核心与空隙；最后用少量局部划分、迁移和顺序调整寻找更短的总时间。场景 B 的候选必须重新按同核驻留、跨核 COPY 释放和容量规则评价；共享 L2 的候选还需经过 FIFO 事件仿真。算法保留已知可行方案，在限定评价机会内比较，不进行分组、核心和排列的组合穷举。

3.2 三种运行语义的关系

表 2 概括后两问相对前一问的改变。相同的输出字段不意味着相同的代价函数：尤其在场景 B 中，一个核心内的多个子图仍有先后次序，但其通信可利用驻留，跨核释放以具体数据搬运事件为准。问题三缓存只改变合资格读取的服务路径，不能绕过算子依赖、同步和四条 Pipe 的占用。

表 2 三问运行语义与增量机制

内容	问题一：场景 A	问题二：场景 B	问题三：B 加共享 L2
Task 与驻留	一子图一 Task; 边界不驻留	同核子图同属一 Task; 可驻留	保持 B 的 Task 与私有存储
跨核数据	边界经共享 DDR; 按 A 释放	COPY 完成后另有同步释放	同 B, 不因命中取消同步
读取服务	DDR	DDR	COPY_IN 可命中共享 FIFO L2
主要决策	划分、映射、核上序	同左, 重估驻留和生命周期	同左, 重估缓存事件与双带宽池

3.3 结构画像：为什么需要多种初解

在建立具体求解器之前，可用操作数、弱连通分量数、最大分量规模与最长拓扑深度描述图的结构，但不能仅凭这些静态量断言最终周期。表 3 列出本工程早期用于开发与确认的六张图的结构快照；它们不是由最终性能反推而后选出的“六张有利图”，也不是一百图的全体统计。可切计算操作数与总操作数不同，因为原始 COPY 等节点不由外层分配子图。最大分量占比 χ 按式 (57) 用可切操作数计算。第 48 图近乎单一深长依赖分量，第 18、33 图却有大量小分量；对这两类图使用相同的“每块固定算子数”会在一类图上切碎关键链，在另一类图上错失天然独立块。因此分量完整性、主导分量有界切分和拓扑窗口必须作为不同初解来源并存。

表 3 六张结构诊断图的静态画像；不是全百图的分布统计

图号	总操作	可切计算操作	弱连通分量	最大分量	最大拓扑深度
12	1304	972	50	98	18
48	1900	1728	1	1728	368
50	4781	4555	23	4011	185
18	3285	2727	126	59	27
33	4069	3465	95	64	33
67	8996	8804	71	124	123

具体计算最大分量占比：第 48 图 $\chi = 1728/1728 = 1$ ；第 50 图 $\chi = 4011/4555 \approx 0.8806$ 。完整分量直接分核，至少有一个核心需要承担主导分量的大部分工作，难以充分利用其余核心。相反，第 18 图 $\chi = 59/2727 \approx 0.0216$ ，第 33 图 $\chi = 64/3465 \approx 0.0185$ ，分量整体分核已经提供很多可调的独立工作块；对它们过早细切单个小分量，很可能增加 Task 或跨核边界而无明显并行收益。第 67 图操作多，却有 71 个分量、最大分量仅 124 个可切操作，提示“图很大”不等于“需要把某一个分量切成极小块”。这说明主机运行耗时与 NPU 最优划分尺度是不同问题：第 67 图因为节点多、官方核内展开复杂而求解贵，仍可先尝试以完整分量为单位构造高质量初解。

这些结构量也并非独立决定瓶颈。最大拓扑深度是按操作层最长链的节点数，忽略每个操作的周期、Pipe 类别与张量字节；两条深度相同的链可以有完全不同的计算时长和 DDR 压力。弱连通分量不计外部输入共享，因此不同分量即使没有中间依赖，也可能共同消费同一输入张量，分给不同 Task 后重复读取。最大分量占比则只用算子数量，没有权衡某些算子的高周期或大张量。正因如此，表 3 只用于初解门控和结构解释，不能替代官方评价或作为全局最优的预言变量。

3.4 静态结构与五核实测的交叉核对

为检验结构判断有没有和实际结果完全脱节，表 4 把同六张图在五核下的两种场景实测并列。第 48 图在 A 中只取得约 1.1245 倍，在 B 中提高到约 2.1863 倍；第 50 图从约 1.5062 倍到约 2.6254 倍。它们的主导分量和长链为“完整分量单核”带来明显限制，场景 B 允许同核驻留并重构候选后表现更好。不过这个成对结果同时改变了 Task 规则、同步、正式候选集，不能把差值单独归因于某一个因素。第 18、33 图无主导分量，A 的五核表现已接近五倍；B 中绝对周期反而略高。第 67 图 A/B 五核周期也几乎持平。这些反例说明后续场景的机制更丰富，不意味着每张图都严格比前一问快。

对第 48 图，仅看“最大分量占比为一”还不足以决定把链切在何处；若切点位于共享张量的高扇出处，会产生大量边界 COPY。可沿关键输出的祖先链选择候选切点，估计切开后双计算 Pipe 的并行余量，再查看跨切点张量集合和释放点。对第 18、33 图，完整分量数量已远超过五核，最重要的问题转为把分量按两 Pipe 负载放到核心、以及控制共享输入重复读取；继续细化每个很小的分量大概率只是增加候选准备成本。对第 67 图，五核周期很大却已有超过四倍的相对加速，若以赛题逐图均值为目标，它贡献与任一小图相同；

表 4 结构诊断图的五核实测对照：两场景使用各自自己选方案

图号	A 周期	A 逐图加速比	B 周期	B 逐图加速比
12	13406	4.4076	12519	4.7199
48	197613	1.1245	101641	2.1863
50	99382	1.5062	57017	2.6254
18	203806	4.9281	205917	4.8775
33	353592	5.0176	361176	4.9123
67	13455316	4.7442	13461153	4.7422

若以总周期为目标，它的绝对改善又非常重要。结构分型因此同时服务于哪类初解有价值 and 主机搜索时间投向哪里两项不同决策。

3.5 从三问差异推出统一而不混淆的求解路线

共同变量 $p = (\pi, m, \sigma)$ 允许三个问题共享合法性框架和结构候选，但不允许共用一个不变的通信权重矩阵。场景 A 的同核跨 Task 张量也必须经 DDR，故主要边界是子图；场景 B 的同核多个子图在同一 Task 中可驻留，故主要边界转成核心；场景 C 又使相同 B 方案中的某些读入由 L2 而非 DDR 服务，并引入全局 FIFO 历史。若按“通信成本从 A 到 C 逐渐减小”简单描述，会漏掉 L2 竞争与非单调反例。严谨的递进关系是三个评价算子

$$\mathfrak{E}_A \circ \mathfrak{R} \circ \mathfrak{G}_A, \quad \mathfrak{E}_B \circ \mathfrak{R} \circ \mathfrak{G}_B, \quad \mathfrak{E}_C \circ \mathfrak{R} \circ \mathfrak{G}_B, \quad (2)$$

其中 $\mathfrak{E}_C$ 比 $\mathfrak{E}_B$ 多共享 L2 队列和第二服务池，而 $\mathfrak{G}_A$ 与 $\mathfrak{G}_B$ 采用不同 Task 与 COPY 规则。这个表达把可复用的图结构候选与必须重评的物理成本分开，解释了为何问题二以问题一方案作种子、却全部重放正式评价；问题三以问题二方案作保底、再做固定方案消融与少量缓存事件候选。

算法最终回答的是每个配置的具体方案，而非仅给出理论表达式。对一张图，结构初解多样化是为了避免落入单一分块尺度；关键工作量和空隙插入是为了让已存在的计算资源不被低优先级任务占住；有限官方评价是为了让 DDR 竞争、核内溢出与 FIFO 路由这些非线性因素进入最终接受判断。若后续证明某个解析代理完全复刻官方仿真，它在计算成本上就应按正式评价记账；在本论文中代理始终被限定为有误差的排序工具。统一框架的优势不是宣称一个权重公式适用于三问，而是在每问按相应事件语义重新评估的同时保持候选结构、合法性和证据口径的一致。

3.6 共同的图表示与决策空间

原始输入是一张有向无环的算子-张量图。令 $\mathcal{O}$ 为操作节点， $\mathcal{X}$ 为张量节点；张量 x 的生产操作为 $p(x)$ ，消费操作集合为 $C(x)$ ，大小为 b_x 字节。原始的 COPY_IN 与 COPY_OUT 是既有数据路径，不属于外层切图决策；令 $\mathcal{O}_c$ 为允许分配子图的其余操作。为讨论依赖，在操作层构造投影弧集合

$$E_c = \{(u, v) : \exists x \in \mathcal{X}, p(x) = u, v \in C(x), u, v \in \mathcal{O}_c\}. \quad (3)$$

投影只便于分析，不丢弃张量的身份、字节数及一对多消费关系。若一个输入张量同时供给三个算子，三条投影边仍共享同一个逻辑张量；通信是否发生三次必须由具体场景的 Task 和缓存规则决定。将 $(p(x), C(x))$ 看成有向超边，正是为了避免把共享输入的读入次数误等同于普通割边条数。

对于核数 k ，方案表示为 $p = (\pi, m, \sigma)$ 。其中 $\pi : \mathcal{O}_c \rightarrow \{1, \dots, q\}$ 为非空子图划分， $m : \{1, \dots, q\} \rightarrow \{1, \dots, k\}$ 为核心映射， σ_c 是核心 c 上各子图的排列。若 $\pi(u) \neq \pi(v)$ 且 $(u, v) \in E_c$ ，则商图有弧 $\pi(u) \rightarrow \pi(v)$ 。核心上的顺序还诱导相邻子图串行弧。令

$$H(p) = (\{1, \dots, q\}, E_\pi \cup E_\sigma), \quad E_\pi = \{(\pi(u), \pi(v)) : (u, v) \in E_c, \pi(u) \neq \pi(v)\}. \quad (4)$$

必要合法条件是 $H(p)$ 无环。证明并不只依赖原图无环：即使 E_π 无环，错误的核上顺序也

可能产生 $A \rightarrow B$ 的数据依赖和 $B \rightarrow A$ 的串行顺序，形成互等。对每个候选重新拓扑检查 $H(p)$ ，才能保证至少存在满足先后关系的顺序扩展。图中的任意子图必须非空，且每个可切算子仅出现一次：

$$V_r = \pi^{-1}(r) \neq \emptyset, \quad \bigcup_{r=1}^q V_r = \mathcal{O}_c, \quad V_r \cap V_s = \emptyset \quad (r \neq s). \quad (5)$$

本文不把“每块固定包含若干算子”当作可行性条件；固定块长仅是生成一种候选的启发式参数。任务数、映射和排列均随图结构变化。

这种联合决策的规模解释了为什么必须限制搜索。忽略依赖后， $n = |\mathcal{O}_c|$ 个带标签算子划成 q 个非空无标签子图已有第二类 Stirling 数 $S(n, q)$ 种，满足

$$S(n+1, q) = S(n, q-1) + qS(n, q). \quad (6)$$

前一项让新算子独立成组，后一项把它加入已有 q 组之一。再为 q 个子图选核心，最多有 k^q 个映射；固定每核的子图集合后，还存在各核排列。依赖和内存约束会删去部分组合，但并不会使“枚举全部划分、核心、次序”在百图规模上可行。因此结构候选只覆盖有依据的局部，官方评价机会在同一预算下分配，而不是对指数空间作均匀遍历。

3.7 从固定工作量到事件完成时间

若把每个子图预先赋予一个固定时长，再做普通多机排程，模型会漏掉两层内生变化。其一，外层划分改变核内张量的生存区间，官方核内启发式可能在容量压力下插入换出与换入；于是同一子图放在不同上下文中，展开后的 COPY 与 Pipe 序都可能不同。其二，多个核心的搬运共同占用 DDR，实际传输时长取决于同时在途操作的集合。令 e 为展开后的执行事件， s_e, f_e 为起止时刻， $d_e = f_e - s_e$ 为其实现时长。对固定时长的计算操作，有 $d_e = c_e$ ；对搬运事件， d_e 由带宽过程产生。发射必须等待它的所有前驱、同一 Pipe 的上一事件和外部释放：

$$s_e \geq \max \left\{ \max_{a \in \text{pred}(e)} f_a, f_{\text{prevPipe}(e)}, r_e^{\text{ext}} \right\}. \quad (7)$$

上式是依赖约束，尚不是一个可直接把 d_e 相加的显式日程；传输与计算可在不同 Pipe 上重叠。将所有实际事件和资源先后关系构成扩展事件图 $\mathcal{H}(p)$ 后，在固定一次仿真已经给出的搬运时长条件下，完成时间等于源到汇的最长事件路径：

$$T(p) = \max_{\gamma \in \Gamma(\mathcal{H}(p))} \left(\sum_{e \in \gamma} d_e + \sum_{(a,e) \in \gamma} \ell_{ae} \right). \quad (8)$$

这里 ℓ_{ae} 只放路径上真正额外发生的释放等待。不同任务的 DDR 忙碌、计算和同步可能同时发生，不能把全局“DDR 忙碌总时长”“Pipe 空闲总时长”“任务释放等待总时长”直接相加当作可节省周期。诊断必须追问某段等待是否落在最终输出的控制路径上。式 (8) 是事后时间线的解释式；因为 d_e 随竞争状态变化，它不是外层优化的线性目标。

共享 DDR 可以按连续时间的服务工作量描述。令 $A_D^+(t)$ 为时刻 t 尚有正剩余工作量的 DDR 搬运集合，事件 e 发射时的独占参考工作量为 $w_e(s_e) = \lceil b_e/B_D \rceil$ ，其中 $B_D = 60$ bytes/cycle。在活跃集合不变的开区间内，公平共享的连续部分满足

$$\frac{dw_e(t)}{dt} = -\frac{1}{|A_D^+(t)|}, \quad e \in A_D^+(t). \quad (9)$$

到达或完成事件发生时先结算过去时间段的剩余工作，再重算下一完成事件；最后按官方整数周期与同刻事件次序执行。式 (9) 解释了为什么只对某个 Task 加上 $b_e/60$ 惩罚不准确：新传输会拖慢同时在途的其他传输，而与其无依赖的 Cube/Vector 计算可能继续。一个候选改变启动时刻后，竞争集合也随之改变，局部估价只能为官方评价筛选候选。

3.8 可证明下界与诊断的边界

令 W_M 、 W_V 为原图中矩阵和向量固定周期操作的总工作量， L_{op} 为只计这些操作本身周期的最长依赖链。无论切图怎样安排，在 k 核、每核各一条同类 **Pipe** 的条件下，都有

$$T_{i,k} \geq L_0(k) := \max \left\{ \frac{W_M}{k}, \frac{W_V}{k}, L_{\text{op}} \right\}. \quad (10)$$

工作量项由抽屉原理推出：若所有核心的矩阵 **Pipe** 占用都小于 W_M/k ，总占用便小于必须执行的 W_M ，矛盾；向量 **Pipe** 同理。最长链项由逐条算子依赖递推，链上计算不能并行。原图固定周期操作不因划分消失，所以式 (10) 对三个问题均适用。它忽略了存储、启动和同步，因而可能很松，但与候选方案无关，可以安全作为统一理论参照。

对已固定的方案 p ，设实际事件展开中 DDR 总搬运字节为 $B_D(p)$ 。带宽容量再给出一个更强但方案依赖的事后界：

$$T(p) \geq \frac{B_D(p)}{60}, \quad L(p) = \max \left\{ L_0(k), \frac{B_D(p)}{60} \right\} \leq T(p). \quad (11)$$

因为 DDR 在一周期内合计最多搬运 60 字节，全部字节即便连续传输也要至少上述时间。 $B_D(p)$ 包含原始 COPY、因划分新增的 COPY 和核内溢出；它随 p 改变。因此 $T(p) - L(p)$ 只能描述这个候选相对自身宽松界的松弛，不能据此宣称它距所有方案的最优值还有多少。即使差距很小，也可能有另一方案同时显著减少 B_D 和 T 。对于问题三，还须把 DDR 与 L2 分成两个独立服务池，不能把两条带宽简单相加。

3.9 两种全量指标的严格区别

对图 i ，题目指定整图核内启发式给出固定单核参考时间 $T_{i,1}^0$ ，所交多核方案的时间为 $T_{i,k}$ 。赛题规定的“平均加速比”是

$$\bar{S}_k = \frac{1}{N} \sum_{i=1}^N \frac{T_{i,1}^0}{T_{i,k}}, \quad N = 100. \quad (12)$$

若关心这一批图合计要占用多少周期，也可报告总周期下降率

$$\rho_k = 1 - \frac{\sum_i T_{i,k}}{\sum_i T_{i,1}^0}. \quad (13)$$

二者不是同一统计量。令 $S_{i,k} = T_{i,1}^0/T_{i,k}$ ，则总周期比可改写为

$$\frac{\sum_i T_{i,1}^0}{\sum_i T_{i,k}} = \frac{\sum_i T_{i,k} S_{i,k}}{\sum_i T_{i,k}} = \sum_i \omega_{i,k} S_{i,k}, \quad \omega_{i,k} = \frac{T_{i,k}}{\sum_j T_{j,k}}. \quad (14)$$

所以总周期比是以多核耗时为权重的加速比，而式 (12) 是每图等权。两个数值可以同时陈述，但不能互换作题目要求的折线图纵轴。若某些大图总周期改善很多、而大量小图改善一般，总周期下降可以明显，平均加速比却不一定同幅增长。后续结果表同时列出平均加速比、百图总周期和额外搬运，正是为了避免单一指标遮蔽这层差异。图 1 用相同的依赖关系说明三种场景的服务路径。P、Q、R 表示子图，P 与 Q 同核、R 在另一核心；示意图只说明规则，不表示实测时长。同核驻留仍受依赖与容量约束；共享只读 L2 的命中不取消原有跨核同步。

3.10 从操作依赖到真实可执行任务的两级展开

外层变量 $p = (\pi, m, \sigma)$ 只有子图标识、核归属及同核顺序，尚不是可直接执行的周期时间线。它必须经过两个确定性映射：首先是场景规则 $\mathfrak{G}_s$ ，依据场景 $s \in \{A, B, C\}$ 把跨子图张量改写成必要的 COPY、同步和 Task 释放；随后由核内启发式 $\mathfrak{R}$ 为各 Task 建立 Pipe 顺序与容量安全的换入换出；最后事件仿真器 $\mathfrak{E}_s$ 给出性能。可把评估写成复合函数

$$(T_s(p), D_s(p), \mathcal{L}_s(p)) = \mathfrak{E}_s(\mathfrak{R}(\mathfrak{G}_s(G, p))), \quad (15)$$

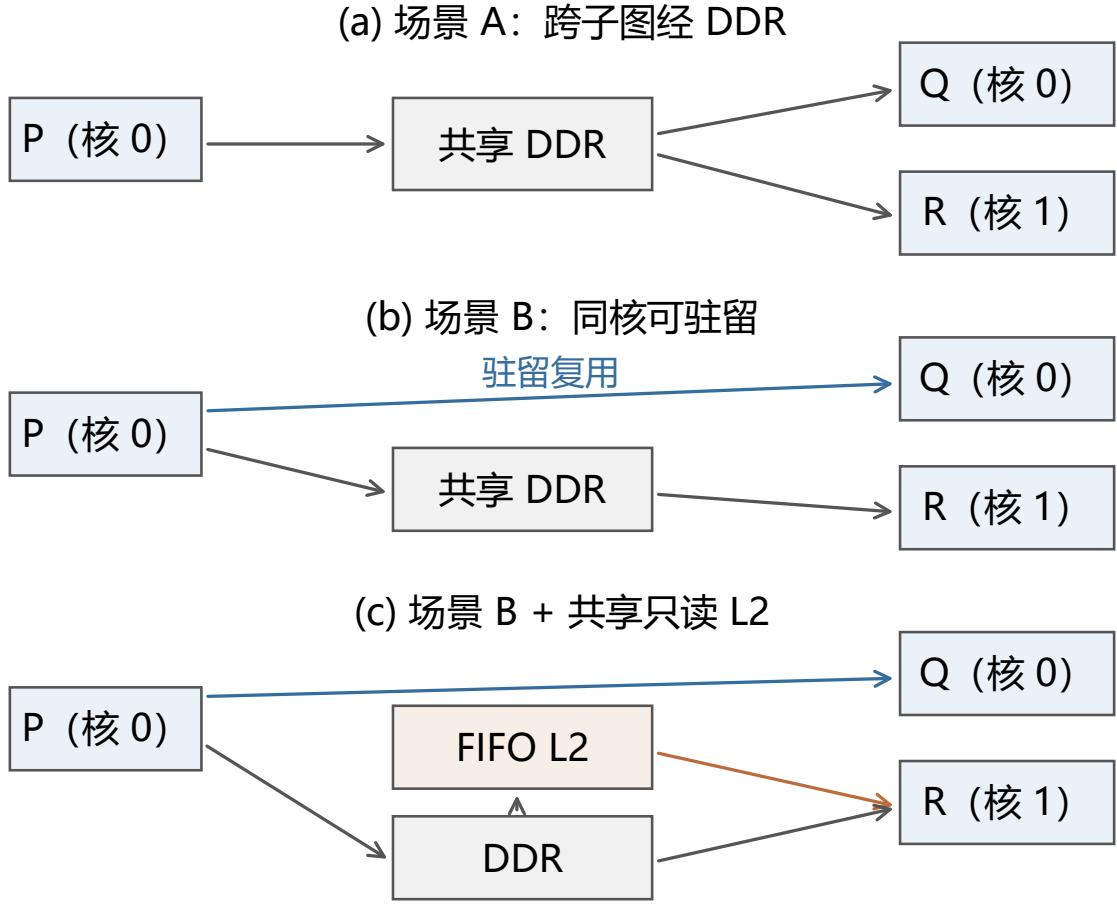


图 1 三种场景的子图数据路径对照

其中 $\mathcal{L}_s(p)$ 为时间线与资源账本。提出一个新划分时, $\mathfrak{G}_s$ 的输出可能有不同数量的 COPY, $\mathfrak{K}$ 的换出决定亦可能变化; 因此 $T_s(p)$ 不是一个关于子图边界的线性函数。场景 C 在相同图展开后又增加状态相关的 FIFO 路由, 故事件映射也随历史轨迹变化。这一复合结构是模型与算法分工的根本: 解析式负责刻画合法性、建立下界和排序候选; 完整事件评估负责判断候选的真实目标值。

对固定 Task r , 定义其操作事件集合 E_r , 每个事件 e 有一个所属 Pipe、前驱集合及可能的张量写入或释放动作。如果把全部事件的持续时间直接相加, 得到的是串行上界式直觉, 并非真实时长。例如 Cube 上的事件 a 与 Vector 上的事件 b 若无相互依赖, 时间区间可以相交, Task 长度至少为 $\max\{d_a, d_b\}$, 不必达到 $d_a + d_b$; 若 b 依赖一个正在 DDR 上读取的张量, 它只能等待对应读入结束, 而另一个无关的 Cube 仍可继续。因而当 DDR 拥塞增加一段 δ 周期时, Task 的完成时间增量满足的只有 $0 \leq \Delta T \leq \delta$ 这一类在固定事件图和单处扰动下的条件性关系; 若延迟改变事件顺序, 连这些条件也须重新检查。不能把每条搬运的增量一概加到整个 Task 长度。

3.11 共享带宽的状态方程与整数周期边界

为说明仿真为何必须事件驱动, 设 DDR 搬运 e 的剩余独占工作量为 $w_e(t) \geq 0$, 发射事件发生于 a_e , 初值 $w_e(a_e) = \lceil b_e/B_D \rceil$ 。在无到达和无完成的区间 $[t, t + \Delta)$, 若活跃集合大小为 n_D , 则

$$w_e(t + \Delta) = \max\{0, w_e(t) - \Delta/n_D\}, \quad e \in A_D(t). \quad (16)$$

所能安全推进的下一个完成间隔为 $\Delta_D = n_D \min_{e \in A_D(t)} w_e(t)$; 外部新任务、核内 Pipe 事件可能更早发射, 因此实际推进量应取 $\min\{\Delta_D, \Delta_{\text{next arrival}}, \Delta_{\text{next compute}}\}$ 。事件发生后重新计

算活跃集合及其服务份额。官方整数周期语义包含取整和同刻事件处理顺序，式 (16) 是解释该机制的连续服务形式；没有复刻同刻顺序的自写代理，不能声称和官方完全等价。该表述也说明“每条边按 $b_e/60$ 加一项”有两类误差：其一，独占时间忽略已有并发搬运在新流加入后变慢；其二，持续时间不能直接转换成最终输出的增量，因为一部分拥塞藏在其它 Pipe 的计算后面。

若对方案 p 的 DDR 事件集合记为 $E_D(p)$ ，其总独占工作量定义为

$$W_D(p) = \sum_{e \in E_D(p)} \left\lceil \frac{b_e}{B_D} \right\rceil. \quad (17)$$

连续公平服务在所有在途传输之间重新分配这份工作。若每条传输的整数周期取整后必须服务至少 $\lceil b_e/B_D \rceil$ 个“单流等效周期”，便有 $T(p) \geq W_D(p)$ ；但是当官方把取整置于不同层级时，以字节总量给出的 $T(p) \geq B_D(p)/B_D$ 才是更稳妥的容量界。本文在理论式中采用后者，并把独占工作量只当作局部代理输入。这样区分可避免把对官方舍入细节的假设写成不加条件的严格下界。

3.12 关键路径归因与重叠时间的去重

给定已经评估的时间线，取最后完成的输出事件 z ，沿使其开始时刻达到最大值的约束边逆行追溯。若 z 的前一事件完成于 t_1 、其跨核数据直到 t_2 才释放、其 Pipe 直到 t_3 才空闲，则其发射为 $\max\{t_1, t_2, t_3\}$ ，真正使其等待的只有达到最大值的约束；另外两个可能同时存在，却不是该次发射的边际原因。多个约束同时达到最大值时存在并列关键边，不能强制把同一等待周期完全分给某一个原因。为避免重叠时间多计，可为每段 $[u, v)$ 定义同时满足的状态指标：DDR 忙碌、相关任务未释放、目标 Pipe 空闲。三类指标会重叠，求和可能大于 $v - u$ 。只有在逐事件逆向追溯后，把关键链上唯一或并列的激活约束计入候选诊断，才能解释改善机会。

这一方法也限定了“减少跨核边”和“减少额外搬运”两类结构指标的用途。若一条边处于非关键分支，即使删去其 1000 周期释放规则，也可能不改变最终完成时间；若某个共享输入读入的流量完全与长时计算重叠，减少其 DDR 字节也可能使 D 降低而 T 不动。反之，减少关键输出前的一个很小搬运，可能使后续多个 Task 提前释放，最终节省远高于该条搬运的独占时间。局部候选选择因此考虑边是否与关键输出的祖先集合相交、是否与 DDR 拥塞窗口相交、改变后是否可能触发另一条路径成为关键路径，而不使用单一的全局边权阈值决定接受。

设当前已实现的关键路径长度为 T_1 ，第二长路径为 T_2 ，且候选只缩短第一条路径 δ ，不改变其它路径和资源竞争。则新完成时间为 $T' = \max\{T_1 - \delta, T_2\}$ ，边际改善为

$$T_1 - T' = \min\{\delta, T_1 - T_2\}. \quad (18)$$

这是固定路径集合下的条件性推导，说明单个瓶颈的收益会在第二路径接管时饱和。真实方案改变会重构路径，式 (18) 仅用作排序和解释，不作为官方评价结果。它也解释为何反复围绕一条旧关键链做开环搜索，后期容易出现大量完全持平的候选；继续增大区域未必有效，可能只增加评价机会成本。

3.13 候选排序的可证范围与公平预算

一个候选池 $\mathcal{C} = \{p_1, \dots, p_M\}$ 中，代理给出 $\hat{T}_j$ ，官方评价给出 T_j 。若代理只能评价所有 M 个、官方最多评价 $B < M$ 个，选择器的实际损失取决于前 B 个覆盖了多少真优候选，而不是代理均方误差本身。令 $j^* = \arg \min_j T_j$ ，代理按升序排列索引为 $\hat{j}_1, \dots, \hat{j}_M$ ，可定义候选池内选择遗憾

$$R_B = \min_{1 \leq b \leq B} T_{\hat{j}_b} - T_{j^*} \geq 0. \quad (19)$$

当 $R_B = 0$ 时，前 B 覆盖了该池的官方最优；但这不表示全局最优已经找到。若同一指纹的方案重复占用多个名额，名义预算 B 与真正不同的 B_{unique} 不同，必须按唯一完整方案

数登记。冻结同一核内准备与成本状态后，才能把多个候选的代理名次和官方名次相配对，分析误差来自 Task 释放、Pipe 路径还是共享 DDR；若在候选间随意重算准备状态，相关系数和遗憾的解释基础也消失。

公平对照使用相同的图、核数、代码版本、随机种子、官方评价次数和候选保底。若新增一个机制挤掉原来有效的候选，它的表现收益包含预算替代效应；若只给新机制额外评价机会，则结果又混入预算扩张效应。定义 $\mathcal{B}$ 为固定官方评价机会，基础搜索分配 B_0 ，结构候选分配 B_1 ，局部修复分配 B_2 ，需满足 $B_0 + B_1 + B_2 \leq \mathcal{B}$ 。在某一邻域连续无效时可以把尚未使用的 B_1 或 B_2 转给另一个，但必须保留基础搜索下限，且在日志记录实际采用的次数；否则同样的名义预算可能对应不同的有效探索。本文有限预算优化的目标是高质量地使用这 $\mathcal{B}$ 次官方调用，而不是把公式下界当成可直接提交的方案。

3.14 可行性、最优性与求解效率的三重区分

可行性由唯一覆盖、商图无环、场景同步规则和物理存储约束决定；性能由 $T_s(p)$ 和 $D_s(p)$ 决定；计算效率则由生成并评价 p 的主机耗时决定。这三者在论文中分别检验。一个有效地压缩了 CPU 搜索时间的候选缓存，不一定改变 NPU 执行周期；一个周期很小的代理预测方案，若未经过正式评价与内存检查，不能称为高质量可行结果；一个正式评价优于对照的方案，也不能据此推断全局最优。

对每个固定候选池 $\mathcal{C}$ ，官方评价中的最小值 $\min_{p \in \mathcal{C}} T_s(p)$ 只提供该池上的上界，即全局最优 T_s^* 满足 $T_s^* \leq \min_{p \in \mathcal{C}} T_s(p)$ 。通用计算下界提供相反方向的 $L_0(k) \leq T_s^*$ 。于是合法的、但一般较宽的夹逼关系为

$$L_0(k) \leq T_s^* \leq \min_{p \in \mathcal{C}} T_s(p). \quad (20)$$

这个区间明确表明已找到方案的质量边界，也避免把某个方案自身的搬运下界误用为全局下界。当 $L_0(k)$ 很低时，不能从宽区间推断还剩多少可实现提升；当某一大图距 $L_0(k)$ 很远时，也不能断言差距都可消除，因为同步、内存与带宽尚未进入 L_0 。因此本文以同口径官方结果、负例和结构诊断来评价启发式，而不虚构最优性证明。

4 问题一：场景 A 的两种切图与调度方案

两种求解方案均面向场景 A：跨子图张量经共享 DDR 中转，私有缓存不跨 Task 保留。结构感知搜索以结构感知初解和保护有限评价机会控制计算成本；多候选精修扩大结构候选覆盖，再用官方模拟筛选并局部改进。两者共享目标函数与可行域，差别发生在候选生成、任务放置和搜索资源分配，而非硬件假设。以下先建立共同模型，再分别讨论算法，最后在相同单核基准下比较五核结果。

4.1 场景 A 的共同组合优化模型

令 V_c 为所有非 COPY 算子。映射 π 把它们恰好分配给 q 个非空子图， m 把每个子图分配给一个核心， σ_c 给出核心 c 上各子图的顺序。把跨子图的数据前驱构成商图，再加入每核顺序边，所得扩展商图必须无环；否则即使原图无环，核心顺序仍可能与跨核等待构成死锁。合法方案集合记为 $\mathcal{P}_A(k)$ 。对于每份方案，官方内层启发式先产生实际 COPY 和核内 Pipe 序，再进行共享 DDR 事件仿真。我们以此可计算但不便线性化的函数定义目标：

$$\min_{p \in \mathcal{P}_A(k)} \text{lex}(T_i^A(p), D_i(p)), \quad T_i^A(p) = \max_{u \in \mathcal{T}(p)} f_u. \quad (21)$$

这里的字典序表示优先缩短最后一个 Task 的完成时间；仅在周期相同时比较额外搬运。它是算法的选优规则，不把较少搬运误说成必然较短的周期。

Task v 的开始须满足所有数据前驱释放和同核前一 Task 的间隔。以 F_u 表示前驱完成时刻， $P_c(v)$ 表示核心 c 上的前一 Task，可写为

$$R_v \geq \max_{u \in \text{pred}(v)} (F_u + 1000 \mathbf{1}\{m_u \neq m_v\}), \quad R_v \geq F_{P_{m_v}(v)} + 100. \quad (22)$$

同核依赖不额外施加跨核的 1000 周期，但跨 Task 张量仍须由 DDR 中转。公式 (22) 仅描述释放约束，不能单独算出时长；Cube、Vector、搬运 Pipe 的先后关系、L1/UB 溢出以及同时在途的 DDR 传输会改变 F_u 。实际评价必须采用题目给出的事件过程。

为了识别一份输入被多个子图重复读取的情形，我们把共享张量与多个消费算子的关系视为有向超边，保留原图方向和商图无环性。多层次无环划分研究说明，此类表示比仅统计两两割边更适合描述共享输入^[2]。我们借鉴这一表示和粗化思想，但没有照搬其平衡目标或宣称实现了文献的完整算法；本题的最终目标仍是式 (21)。

4.2 场景 A 的物理语义与可行性

场景 A 中子图与 Task 一一对应，即 $\mathcal{T}(p) = \{V_1, \dots, V_q\}$ 。两个 Task 即使被映射到同一个核心，前者的私有 L1/UB 状态也不能跨 Task 保留；跨子图张量必须先写至 DDR，后继 Task 再读回。因此子图边界上的数据路径由 π 决定，核心映射 m 并不能把同核边界通信改成免费的片上读取。这一点与场景 B 是根本区别。核心 c 上按 σ_c 串行执行的 Task 之间另有固定的 100 周期切换间隔；跨核数据前驱给目标 Task 施加 1000 周期等待。若有多个前驱，目标 Task 等到最后一个约束满足即可，而非把所有前驱延迟相加。

将 Task r 的实际开始、完成记为 S_r, F_r ，其数据前驱集合为 $\text{Pred}_\pi(r)$ 。若 $a \prec_c r$ 表示核心 $c = m(r)$ 上排在 r 前面的最后一个 Task，释放规则可写为

$$\begin{aligned} S_r &\geq \max_{a \in \text{Pred}_\pi(r)} \{F_a + \delta_{ar}\}, & \delta_{ar} &= 1000 \mathbf{1}\{m(a) \neq m(r)\}, \\ S_r &\geq F_{\text{prev}_{\sigma_c}(r)} + 100 & & \text{若该前一 Task 存在.} \end{aligned} \quad (23)$$

式 (23) 只描述外层 Task 的可发射时刻。某 Task 的 $F_r - S_r$ 并非常数：展开出的四条 Pipe 上的操作依赖、共享 DDR 竞争以及 L1/UB 压力都会影响它。优化器因此必须先给出合法 (π, m, σ) ，再调用官方核内与多核评价过程得到真实 F_r 。若仅用一个预先标定的“子图长度”代入普通列表调度，便会在两个候选改变 DDR 发射重叠时给出相同预测，却可能得到不同的正式完成时间。

四条 Pipe 分别执行矩阵运算、向量运算和两类搬运。用 $P(e)$ 表示事件 e 所属 Pipe，核内候选排程给每条 Pipe 一个确定顺序。对相同核心及相同 Pipe 的前后事件有 $f_{\text{prevPipe}(e)} \leq s_e$ ；跨 Pipe 只由真实算子依赖与内存复用依赖约束，不人为强制串行。对任一私有存储池 $r \in \{\text{L1}, \text{UB}\}$ ，展开后的物理驻留集合 $\mathcal{R}_{c,r}(t)$ 必须满足

$$\sum_{x \in \mathcal{R}_{c,r}(t)} b_x \leq C_r, \quad C_{\text{L1}} = 524288, \quad C_{\text{UB}} = 131072 \quad (\forall c, t). \quad (24)$$

如果未经换出换入就会越界，官方核内启发式会引入附加搬运来维持容量合法。因而式 (24) 约束的是实际展开的执行，不能把“候选静态张量大小总和未超限”误认为充分条件，也不能把溢出惩罚写成只由子图算子数量决定的固定函数。

4.3 额外搬运的计数口径

令 $B_{\text{all}}(p)$ 为官方展开方案在 DDR 上的总逻辑 COPY 字节数， B_{orig} 为未切图输入中已经要求的基础 COPY 字节数。题目所报的额外搬运定义为

$$D(p) = B_{\text{all}}(p) - B_{\text{orig}}. \quad (25)$$

这个定义覆盖了子图边界传递、原始输入的重复读取，以及核内启发式因容量不足插入的换入换出。正文始终用式 (25) 与官方输出核对，而不用简单割边权和代替它。设输入张量 x 被 $r_x(\pi)$ 个不同 Task 消费，则在独立 Task 都需要从 DDR 读入这一语义下，相对一次读取的重复输入字节可用

$$D_{\text{input}}^{\text{proxy}}(\pi) = \sum_{x \in \mathcal{X}_{\text{input}}} (r_x(\pi) - 1)_+ b_x \quad (26)$$

作静态代理。它反映多消费超边的割开程度：同一张量的三个消费者分处三个 Task 时，不宜只看三条两两边的权重；若三个消费者聚在一个 Task，代理重复读入为零。但式 (26) 只涉及外部输入的一个组成项，不能再把同一张量的边界读入重复加进另一个“割边项”以免双计。中间张量的写出次数、各目标 Task 的读取及真实溢出，由官方展开后的事件记录裁决。

减少 $D(p)$ 并不等价于减少 $T(p)$ 。反例是合并两个可并行子图：该动作可能删除边界 COPY，却使原来可分别运行的矩阵链和向量链被一个 Task 的内部顺序或容量压力拖慢。反过来，少量额外搬运若释放关键任务的并行空间，也可能缩短 Makespan。因此式 (21) 采用完成时间优先的字典序；并在结果章节同时报告 $D(p)$ 的变化，而不预设它必与主目标同向。

4.4 方案条件下的下界及其推导

共同计算下界 $L_0(k)$ 已由式 (10) 给出。场景 A 还允许对固定方案 p 构造更贴近其 Task 结构的事后下界。令 $W_{M,r} = \sum_{e \in V_r \cap O_M} c_e$, $W_{V,r} = \sum_{e \in V_r \cap O_V} c_e$ ；同一 Task 内这两类 Pipe 可以重叠，却各自不能在本 Pipe 上并发，因此

$$F_r - S_r \geq \tau_r := \max\{W_{M,r}, W_{V,r}\}. \quad (27)$$

若核心 c 上有 n_c 个 Task，则该核从第一个 Task 起始到最后一个 Task 完成至少占用 $\sum_{r:m(r)=c} \tau_r + 100(n_c - 1)_+$ 周期。令 $\mathcal{P}(H(p))$ 为扩展商图的路径集合，在路径 $(r_1, \dots, r_h)$ 上，每个任务的最短计算时间和依赖释放必须依次经过。因此

$$T(p) \geq L_A(p) := \max \left\{ L_0(k), \frac{B_D(p)}{60}, \max_c \left[\sum_{r:m(r)=c} \tau_r + 100(n_c - 1)_+ \right], \right. \\ \left. \max_{(r_1, \dots, r_h) \in \mathcal{P}(H(p))} \left[\sum_{j=1}^h \tau_{r_j} + \sum_{j=1}^{h-1} \delta(r_j, r_{j+1}) \right] \right\}. \quad (28)$$

对路径中的数据依赖弧，跨核时取 $\delta = 1000$ ，同核时至少为 100 的任务间隔；对纯顺序弧，只取 100。若一对任务既有数据依赖又有顺序关系，同一间隔仅计一次。式 (28) 的每项分别由计算工作、带宽容量、同核串行和依赖链推出，取最大而不是相加，避免把可重叠资源时间重复计算。

这个界是针对所选划分和映射的条件界。例如合并任务会改变 $B_D(p)$ 、 n_c 和路径集；把方案 p_1 的 $L_A(p_1)$ 当作方案 p_2 的最优下界没有依据。我们用它寻找当前方案的主导限制：若同核负载项远大于其他项，优先考虑拆分或迁核；若关键路径项主导，优先处理路径上的边界与核心切换；若带宽项主导，检查重复输入及边界流量。这是一条有物理解释的候选分配规则，不是由下界直接求得最优划分。

4.5 合并与拆分的边际判断

考虑相邻子图 A, B 与一份大小为 b 的边界张量。若把它们合成同一 Task，理论上可删除这份边界上的部分 DDR 事件与释放等待；但这份“被删除的字节数”不一定等于 b 或 $2b$ ，因为还取决于共享消费者、输出写回以及已有 COPY。令 $\Delta B_{\text{boundary}}$ 为按场景 A 展开规则实际消失的边界字节， ΔB_{spill} 为合并后新增的溢出字节。一个仅用于排序的净搬运近似为

$$\widehat{\Delta B} = \Delta B_{\text{boundary}} - \Delta B_{\text{spill}}. \quad (29)$$

若合并删除一个实际落在关键链上的跨核 Task 边，还可能减少 1000 周期释放；若删除的是同核相邻 Task 边，最多直接省下该处 100 周期间隔。然而被删除的等待与传输可以重叠，也可能原本根本不控制最终输出，所以不能把 $\Delta B_{\text{boundary}}/60 + 1000$ 作为必得的 Makespan 改善。真正的差值为

$$\Delta T_{\text{merge}} = T^A(p_{\text{old}}) - T^A(p_{\text{merged}}), \quad (30)$$

只能在两个完整方案通过官方评估后确定。式 (29) 给出有依据的候选顺序，式 (30) 才给出是否接受的结论。拆分动作同理：它增加边界开销，却可能降低单核长尾与局部溢出。固定算子个数切块无法表达这种结构差异，故初始划分同时覆盖整分量与少量主导分量拆分。图 2 概括两种方法的候选构造路线。两条路线共享覆盖、无环与物理约束；图中的最终评价原则也用于搜索过程中的每次候选接纳。

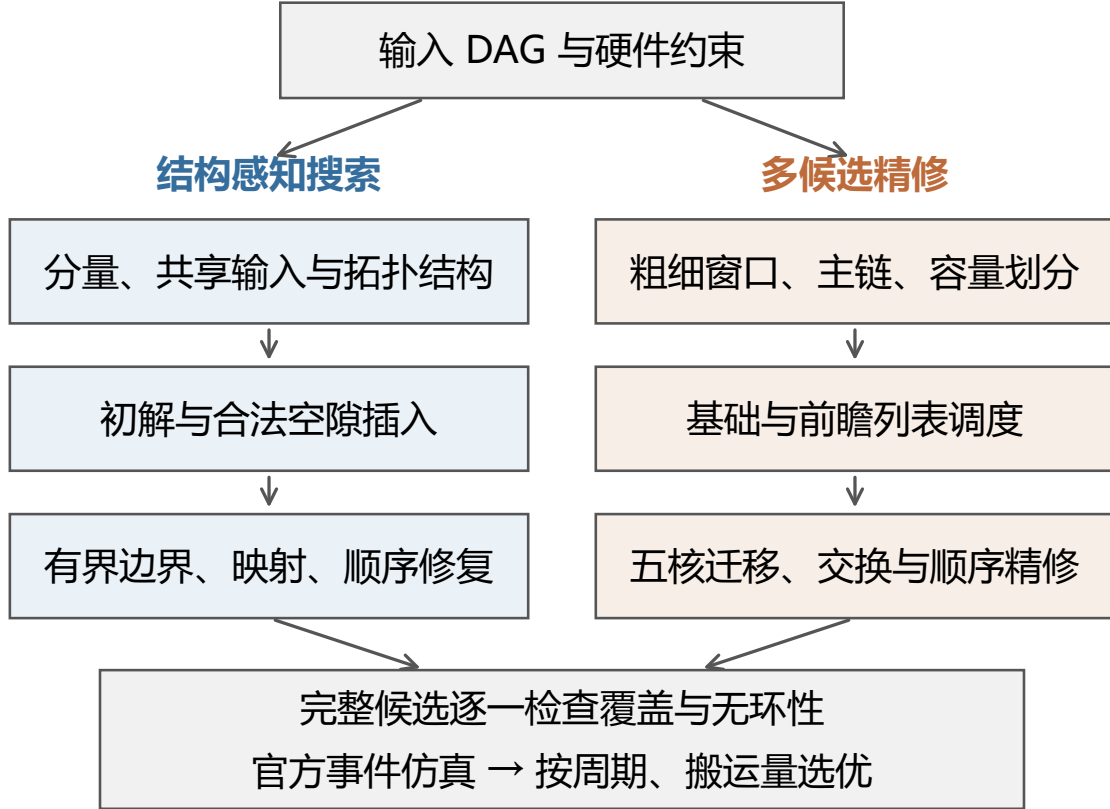


图 2 结构感知搜索与多候选精修的算法对照

4.6 结构感知搜索：结构感知初解与有界搜索

首先构造整图保底方案与依赖贪心方案。对没有计算依赖的弱连通分量，尝试保持完整并按估计的最长 **Pipe** 工作量分核；这样能释放天然并行度，且减少不必要的分量内切边。若一个主导分量明显压过其余分量，则在满足无环性的拓扑窗口中有限拆分，以免完整保留造成单核长尾。共享输入复用、局部核内时长和容量压力同时参与候选排序；局部估计只负责提出方案，不能替代官方全局仿真。

将候选子图调度到核心时，先按后续关键工作量排序，再为每个子图寻找最早合法插入空隙和预计完成时间较小的核心。该思想与最早完成时间列表调度相近^[3]，但本题必须附加式 (22)、共享 **DDR** 和真实容量条件。随后在固定的评价机会内，尝试少量关键区域的边界调整、核心迁移与顺序修复；每个被提出的完整方案均检查操作覆盖和扩展商图无环，并由原始评价语义选优。

当前采用方案对每个多核配置限制为 12 次评价机会、确定性种子 0。基础粒度搜索保留机会，结构新候选须通过事件成本准入；失败的候选不能因预测好就替代实测较优方案。当前研究版本启用了局部成本、关键路径、一般映射空隙插入、通信排序、划分保护、共享输入及有限区域修复。实验中测试过而未被采用的事件池整体重排序、扩大的自适应组合等，不计入本轮主结果。固定预算使评价调用数随图数线性增长；候选构造与图规模相关，但不枚举指数数量的分组、映射和任务排列。

4.6.1 初解生成的结构依据

求解先利用计算图的弱连通分量。两个互不相连的分量之间没有计算数据前驱；若把完整分量分别放到不同核心，不会产生分量间新的中间张量通信，因此它们是低成本的并行初解。设分量 C 的矩阵、向量工作量分别为 $W_M(C), W_V(C)$ ，并定义两管道负载向量 $\mathbf{w}(C) = (W_M(C), W_V(C))$ 。若某分量的某项工作量超过平均每核份额，

$$W_M(C) > \frac{W_M}{k} \quad \text{或} \quad W_V(C) > \frac{W_V}{k}, \quad (31)$$

则在“整个分量必须落在一个核心”的候选中，该核心相应 **Pipe** 至少要执行 $W_M(C)$ 或 $W_V(C)$ 周期，比理想均摊项更大。式 (31) 是考虑拆分的判据，不是“拆分必优”的证明：如果沿分量内部切断一条巨量共享张量，通信、同步或溢出可能使最终时间更差。实现只在有限拓扑窗口提出切分，并保留不拆的完整分量方案作对照。

分量分核用负载向量而非操作数量。对当前核心 c 的估计负载记 $\mathbf{L}_c = (L_{M,c}, L_{V,c})$ ；将分量 C 放到 c 后，一个用于初解的平衡指标为

$$\hat{L}_c(C) = \max\{L_{M,c} + W_M(C), L_{V,c} + W_V(C)\}. \quad (32)$$

优先选择能控制当前最大 **Pipe** 长尾的核心，再用共享输入字节数和依赖释放风险破同分。式 (32) 只近似双计算管道的最短可能占用，尚未计入搬运 **Pipe**、DDR 或生命周期。它比“每核平均分配相同算子数”更贴合计算时间，却不能单独决定最终方案。

另一类初解利用共享输入超边。对输入 x 的 **Task** 消费集合 $\mathcal{C}_x(\pi) = \{\pi(v) : v \in C(x)\}$ ，式 (26) 说明切到多个 **Task** 会增加读入机会。生成候选时，先根据超边交集和拓扑位置给相关操作形成小的局部簇，然后只有在商图无环且负载不过分集中时才尝试合并。这样让“共享输入节省”与“并行空间丢失”在候选池中同时可见。弱连通分量、拓扑窗口、共享输入簇对应不同图结构，保留多种候选能够降低单一固定粒度的偶然性。

4.6.2 关键工作量优先与合法空隙插入

固定一种划分后，给 **Task** r 一个向后看的关键度。以 τ_r 为式 (27) 给出的最短计算时长，从商图汇点反向递推

$$\text{rank}(r) = \tau_r + \max_{s \in \text{Succ}_\pi(r)} \{\text{rank}(s) + \mathcal{C}_{rs}\}, \quad \max \emptyset = 0. \quad (33)$$

其中 $\mathcal{C}_{rs}$ 可以取零作为保守结构排序，也可用经场景 A 展开规则估计的边界搬运与释放代价。 $\text{rank}(r)$ 大，表示推迟它更可能拖慢最终输出，因而在已就绪 **Task** 中优先处理。这是排序启发式，不把估计值误认为准确关键路径。**Task** 间序与核心映射仍由合法性检查和官方评价决定。

对就绪 **Task** r ，列表调度并非只能附加在某核队尾。核心 c 上已有按时间排序的 **Task** 区间 $[S_j, F_j]$ 。考虑相邻区间 $(j, j+1)$ 之间的空隙，候选插入位置须同时满足

$$S_r \geq F_j + 100, \quad F_r + 100 \leq S_{j+1}, \quad S_r \geq \max_{a \in \text{Pred}_\pi(r)} (F_a + \delta_{ar}). \quad (34)$$

还要检查插入后扩展商图仍无环。若放在队头或队尾，相应的单侧不等式删除。 F_r 在真实仿真前未知，调度器以局部成本估计做插入排序；被选中的完整方案最终由官方事件评价。与永远追加在队尾相比，合法空隙插入可把一个已经满足前驱的任务放进另一个任务等待 DDR 或跨核释放时留下的区间，而不改变已排 **Task** 的相对次序。若局部估计误差使其实际上不缩短完成时间，保底方案仍在。

候选核的比较遵循预计最早完成时间，但显式保留释放时刻和空隙位置。记 $\hat{d}_r(c)$ 为局部核内准备得到的任务时长代理， $\hat{S}_r(c, g)$ 为在核心 c 空隙 g 中满足式 (34) 的最早估计开始时刻，则

$$(c^*, g^*) \in \arg \min_{c, g \text{ legal}} [\hat{S}_r(c, g) + \hat{d}_r(c)]. \quad (35)$$

式 (35) 只产生一个或少量候选，不意味着真实全局完成时间等于该任务预计完成时间。多个任务共享 DDR；把一个任务提前可能推迟同时搬运的关键任务，所以每个候选在送官方评价前还要经过较完整的事件成本排序。

4.6.3 代理排序、机会成本与局部修复

在官方评估调用受限时，搜索对象不是全部合法方案，而是当前结构附近的有界候选池。代理评价固定一次成本状态，保留已准备的 **Pipe** 顺序、依赖和传输工作量；对各候选仅重放受到局部改变影响的释放与资源事件。设 $\hat{T}_\theta(p)$ 为在同一已冻结状态 θ 下的预测完成时间，比较候选 p_a, p_b 时必须使用同一 θ ：

$$p_a \prec_\theta p_b \iff \hat{T}_\theta(p_a) < \hat{T}_\theta(p_b). \quad (36)$$

若用候选 a 的旧资源占用预测 a ，却用另一套已更新的占用预测 b ，排名差异不能解释为方案差异。更重要的是 $\hat{T}_\theta$ 不是严格上界或下界；代理预测改善但官方变差的负例曾出现。因此代理只决定哪个合法候选优先占用宝贵评价机会，不能直接替换已验证方案。对于不同划分但同核内准备键的候选，复用已确定的核内子问题结果以降低主机计算秒数；这种工程复用不改变 NPU 仿真周期。

局部修复选择少量相互关联的瓶颈 Task，允许调整它们的边界、核心和局部顺序。相关性由关键链邻接、共享张量和在同一 DDR 拥塞窗口内搬运共同确定。区域外保持原相对顺序，区域内仅取有限种分配和优先级，逐项过滤覆盖重复、商图环与资源非法；对完全相同的完整方案按划分、映射、顺序的规范序列去重。保留至少一个结构不同的合法候选，比重复评价同一方案更有信息价值。区域规模及提案数设置硬上限，其作用是搜索联合改变而非组合穷举。

每个多核配置的完整官方评价机会封顶为 $B = 12$ 。基本划分与映射搜索先保留一部分机会，新初解和区域修复只能在剩余机会内竞争；同一指纹的重复方案不应假装是一次新探索。官方返回 (T, D) 后，采用规则为

$$p_{\text{best}} \leftarrow \arg \min_{p \in \{p_{\text{best}}, p_{\text{new}}\}} \text{lex}\{T^A(p), D(p)\}, \quad (37)$$

因此在一个配置内已评价方案的最优记录不会因一次失败提案而变差。相对上一轮不同完整算法版本的少数退步仍可能存在：版本间初解、排序、预算占用发生变化，新的有限候选池不一定包含旧版最优解。不能把式 (37) 误读为跨版本、跨核数或跨场景单调性的证明。

4.6.4 求解过程与复杂度边界

可将上述过程写为如下可复核的有限步骤：

1. 从原图生成整图保底、依赖贪心和完整分量等合法初解；对式 (31) 触发的分量有限地产生拓扑切分。
2. 对每种划分用式 (33) 排序就绪任务，按式 (34)、(35) 选取核心及合法空隙，得到完整 (π, m, σ) 。
3. 扩展商图做无环检查，并检查唯一覆盖与基本结构；对合法且不重复方案，用相同成本状态的代理排序。
4. 在至多 B 次完整官方评价内，轮流分配基础搜索与有限区域修复机会；每次按式 (37) 更新最优实测方案。
5. 输出每个算子子图编号与每核顺序，最终再作独立官方回放和逐例指标核对。

拓扑排序和无环检查对一个固定候选可在线性于商图节点、边数的时间完成；候选生成次数由结构档数、局部区域上限和评价预算限制。官方核内排程及事件仿真仍可能随大图、缓存溢出和同时在途搬运而耗时，故“组合搜索次数有界”并不自动保证每个大图从冷启动到最终复核都在十分钟内。本文所报最长约 364 秒仅对应已记录的热搜索阶段，求解器墙钟秒数与所输出方案的 NPU 执行周期必须分别解释。

4.6.5 超边流量、串并行收益与划分尺度

进一步说明为什么本问以共享张量超边而非普通边数参与初解。设一个输入张量 x 大小 b_x ，有三个消费算子 u_1, u_2, u_3 。若三者处于同一 Task，题目数据路径至多需要该 Task 按核内策略读取该张量；若分在三个 Task，场景 A 的 Task 边界不保留私有缓存，至少有三次独立的消费机会。相对共同读取一次，静态重复输入代理由式 (26) 给出 $2b_x$ 。但若用源到每个消费者的普通投影边表示，同一个张量可被计为三条边，简单地给每条边一个 b_x 权重会得到 $3b_x$ ，和“重复读取增量”不是同一量；若再把跨子图读入 $3b_x$ 叠加到 $2b_x$ ，甚至重复统计。对内部张量又有生产侧写出和消费侧读入，场景 A 的真实开销取决于官方构图细节，不能机械套用输入张量的 $r_x - 1$ 公式。因此超图权重只用于优先保留共享消费者，不作为正式 D 的替代。

对于分量 C ，可定义用于结构选择的三维轮廓

$$\mathbf{q}(C) = (W_M(C), W_V(C), R(C)), \quad R(C) = \sum_{x: C(x) \cap C \neq \emptyset} b_x, \quad (38)$$

其中 $R(C)$ 只表示潜在输入集合的字节规模。两分量没有原图依赖，分核不增加彼此的中间张量通信，但若它们共同消费原始输入，分到不同 Task 仍可能重复读入。这说明“弱连通分量完全独立”只能用于数据依赖和中间张量边的判断，不能宣称总输入搬运独立。轮廓中矩阵、向量工作量用于估计单核的两 Pipe 长尾； $R(C)$ 用于提醒共享输入代价。若只按 $W_M + W_V$ 做降序装箱，可能把一个 Cube 密集分量与另一 Cube 密集分量塞入同核，而让 Vector Pipe 和其它核心闲置。按式 (32) 比较添加后 $\max\{L_M, L_V\}$ ，可以把互补的 Cube/Vector 分量放在同核，利用两个计算 Pipe 的同时执行能力。

为了识别值得拆的主导分量，还要区分理论可并行余量和实际可拆边界。设完整分量 C 的算子投影图在某拓扑位置切成 C_1, C_2 ，其理想负载下界从 $\max\{W_M(C), W_V(C)\}$ 可能降为 $\max\{\max_j W_M(C_j), \max_j W_V(C_j)\}$ 。令这两个数的差为 G_{pipe} 。切开后需要传递的超边集合记为 $\partial(C_1, C_2)$ ，其潜在 DDR 工作量至少与其中需要跨 Task 的不同张量有关。 G_{pipe} 是释放并行的潜力，不是预计 Makespan 的真实节省；边界流量、100/1000 释放及局部溢出可抵消它。本文只在 G_{pipe} 足以提出合理候选、且商图保持无环时占用有限评价机会；如果整分量方案更快，保底机制选择整分量。由此，固定“每八算子一块”的规则被降为多种候选中的一类，而不作为普适划分尺度。

4.6.6 边界移动的局部合法性与非局部代价

设当前 $u \in V_a$ ，考虑把 u 移到相邻子图 V_b 。这一小动作需满足四层检查。第一， $V_a \setminus \{u\}$ 非空，且每个算子仍出现一次。第二，新的商图与核心顺序弧的并图 $H(p')$ 无环。第三，移动后的所有跨边数据都可按场景 A 插入合法的写出和读入事件，核内存储压力经官方排程处理后不违约。第四，候选必须在相同原图和同一核数下重新评价。前两项能用拓扑排序廉价地判定，后两项需要实际展开与仿真，不能仅凭原图局部邻接完成。尤其当 u 是共享张量的生产者时，它的移动可能改变多个消费 Task 的 COPY 结构；看似局部的划分变更可对多核 DDR 在途集合产生非局部影响。

可用集合差而非边计数，预估移动后的输入超边变化。记 $R_x(p) = \{\pi(v) : v \in C(x)\}$ 为张量 x 的消费者 Task 集合，移动 u 只可能改变包含 u 的那些 R_x 。代理差量为

$$\Delta \hat{D}_{\text{input}}(u : a \rightarrow b) = \sum_{x: u \in C(x)} b_x \left[(|R_x(p')| - 1)_+ - (|R_x(p)| - 1)_+ \right]. \quad (39)$$

若 a 中还有其它消费者而 b 中已有消费者，则可能 $|R_x|$ 不变，移动算子并不会改变该输入的重复读取代理；若 u 是 a 中唯一消费者而 b 中已有消费者，则 $|R_x|$ 减一。此增量可增量更新，因而适合从庞大候选集中筛出少量变体。但它仍不包含生产中间张量的写回、Pipe 排程和缓存溢出，所以不能代替式 (25)。

4.6.7 输出关键链与核心空闲的区别

令 $\mathcal{A}(z)$ 为最终输出 z 的算子祖先集合；若有多个输出，取最后完成的输出的祖先。某个算子或 Task 不在 $\mathcal{A}(z)$ 上，缩短它本身通常不能直接缩短当前实现时间，除非它在共享 Pipe 或 DDR 上抢占了关键祖先所需资源。这样可把候选分为两类：直接修改关键依赖路径的边界，或修改争用资源以间接帮助关键路径。单纯数核心忙碌百分比不能区分两类：一个核心全时忙碌但只做早早结束的非关键分支，与一个核心短时工作却在输出前形成最后释放点，对 Makespan 的贡献可能相反。因此我们同时观察最长 Task 链、跨核切换、Pipe 工作量和共享 DDR 时窗。

若某图有一个核心长期空闲，也不能简单强制把任务迁到它。考虑迁移一个 Task r 的估计净效应：释放原核心的计算负载 ΔC_{old} ，增加新核心的负载 ΔC_{new} ，并改变进入与离开 r 的 Task 边跨核状态。即使 $\Delta C_{\text{new}} = 0$ 、原核上有明显尾部队列，新增的关键链跨核释放仍可能使输出晚于原方案。一个必要的结构检查是：迁移后所有受影响的前驱和后继能否在合法顺序中释放；一个性能检查是完整仿真得到的 $T(p') < T(p)$ 。计算空闲只是候选触发信号，而不是接受条件。对问题一尤其如此，因为同核 Task 边界本就经 DDR，不存在“迁到同核就免传输”的捷径；迁核主要改变释放等待和并行排程。

4.6.8 多核加速比的两个上限直觉及其局限

若把全部计算视作一条完全可分的单类 Pipe，理想速度增益似乎至多为核数 k ；但题目以启发式整图单核结果作分母，该基准可能因整图规模与缓存压力而不是理论最优，故实测比值超过 k 不构成逻辑错误。另一方面，若只从固定计算链看， L_{op} 给出无论核数多少都无法突破的时间下界，因此对某个方案的参考单核时间 $T_{i,1}^0$ 有形式上的上限 $S_{i,k} \leq T_{i,1}^0 / L_{\text{op}}$ 。该上限往往很松，因为没有算入同步、DDR 和容量。本文不拿它预测成绩，只用来说明当关键串行链占主导时增加核心不可能无限加速，以及为何需要在分量和关键链之间平衡切图。

更实际的平均指标也不能由“每个图最优地利用五核”推断。若图 i 的 $T_{i,1}^0$ 很小、受固定 Task 释放主导，则 $S_{i,5}$ 可能低于其它图；若图 j 的整图核内溢出异常大，切成多核后即使新增通信仍可能得到超过五倍的比值。式 (12) 给每张图相同权重，要求算法在不同结构之间稳定，而不是只针对总周期占比最大的几张大图取得很好的成绩。我们在报告中并列列出大图周期贡献与百图逐图均值，目的不是制造两个“官方分数”，而是说明总目标与统计呈现的不同关注点。

4.7 结构感知搜索的百图结果与解释

表 5 和图 3 给出固定单核分母下的百图结果。五核的逐图平均加速比为 3.665903，总执行周期为 73 170 501；后二者分别衡量“每图等权速度”和“这批图的总工作量”，不可相互替代。二至五核的总周期随核数降低，但加速不是线性的：更多核心既释放并行，也可能增加 DDR 读取和跨核等待。

表 5 问题一结构感知搜索的百图汇总

核心数	平均加速比	总周期 (cycles)	总额外搬运 (bytes)
1	1.000000	312 479 373	1 241 686 120
2	1.834917	158 746 394	1 972 107 414
3	2.546238	109 880 862	1 848 970 300
4	3.161009	86 330 529	1 764 926 118
5	3.665903	73 170 501	1 767 728 284

与本工程上一轮相同百图方案相比，二至五核总周期分别下降 0.9958%、1.6807%、2.7519%、3.6678%；400 个多核配置中 68 胜、330 平、2 负。负例是第 6 图的三核和四核，不能把各阶段收益简单相加，或说每张图都改善。第 59 图在二、四、五核取得本批次最大

的逐图加速比，三核最大值来自第 97 图；这些仅为分布端点，不替代百图均值。由于单核分母本身是题目规定的核内启发式而非全局最优，少数多核比值超过核数可以出现，不构成评价错误。

五百份方案均通过官方结构及顺序校验，逐行指标与保存的官方评价相符；代表图还做了冷启动复现。大图采用增量无环检查、状态索引和已固定准备的复用，以减少求解器自身时间；这些工程措施不应被解释成 NPU 任务周期的额外收益。当前热基准搜索最长约 364 秒，但它不含所有配置的冷单核生成和额外最终复核，因此不能断言从零开始每图均在十分钟内结束。完整逐例周期与搬运见附录。

4.8 平均加速、总周期与执行效率的不同视角

表 5 中的平均加速比按式 (12) 逐图等权统计。为了评价这批图的总体执行负担，表 6 另把单核总周期 312 479 373 作为分母，列出合计周期缩减率与总周期比。数值均直接由同一 500 行结果表计算，不引入另一批算例。五核总周期较单核下降 76.5839%，其倒数形式的总周期比为 4.270565；规定的逐图平均加速比却为 3.665903。差异并非评估矛盾，而是式 (14) 的权重差异。论文主图仍采用规定的逐图等权均值，总周期比只用于说明大图的绝对时延贡献。并行效率定义为 $\eta_k = \bar{S}_k/k$ ，只是把相对核数的等权加速程度标准化。五核时

表 6 问题一结构感知搜索百图总周期视角与逐图等权视角

核数	总周期比	总周期下降率	平均加速比	平均并行效率
1	1.000000	0.0000%	1.000000	1.000000
2	1.968419	49.1978%	1.834917	0.917459
3	2.843802	64.8358%	2.546238	0.848746
4	3.619570	72.3724%	3.161009	0.790252
5	4.270565	76.5839%	3.665903	0.733181

$\eta_5 = 0.733181$ 并不意味着其余 26.6819% 的周期都由 DDR 独自造成；关键链、Pipe 不均、释放等待和核内溢出都可能参与。任务执行时间越短是首要目标，不能为了让 η_k 的比值好看而故意牺牲 Makespan。后文图 3 的两个子图分别展示逐图平均加速比与百图总周期，报告两个互补视角。

4.9 典型结果与反例

第 59 图的单核参考为 829 473 周期，五核方案为 150 782 周期，逐图加速比约 5.5011；其额外搬运则从单核 1 691 648 增至五核 4 325 376 字节。这个例子说明完成时间改善可以伴随更多逻辑搬运，也说明超过五倍的比值并不等于突破硬件计算吞吐上限。赛题的单核分母来自固定的核内启发式，不是单核全局最优；多核切图改变核内问题规模和溢出行为，可能使官方启发式在五核下比把原单核结果简单除以五更有效。第 97 图单核 11 491 509、三核 3 509 566 周期，加速比约 3.2743，是三核配置的分布端点之一。端点代表有利结构，不代表百图平均水平，也不能据此承诺每张图接近理想线性加速。

第 6 图呈现相反的警示：三核 32 925 周期、四核 31 129 周期、五核 31 729 周期，五核比四核慢。其核数、划分与映射共同改变，不能仅凭这三个总数断言“多一核导致 DDR 拥塞”；需结合时间线中的关键依赖与在途传输核查。该图在前后算法版本比较中三核和四核亦是两处退步，说明有限预算下的新候选机制可能挤掉旧版有价值的候选。故当前选优只保证同一候选池内不接受实测变差方案，跨版本必须再做同图同核配置对照。

4.10 额外搬运与大图求解代价

二核百图总额外搬运量为 1 972 107 414 字节，高于三核、四核和五核的合计值；五核也略高于四核。这一序列不单调，表明“核数增加必然增加割边搬运”过于简单。算例的实际划分粒度、完整分量分配、共享输入重复读取和核内溢出都随核数变动；如果把所有差值都归咎于跨核边数，便忽略了场景 A 下同核跨 Task 仍须 DDR 的规则。对本文采用方案，

搬运量是次级实测指标，不能由算子数或切边数推算为附录最终值。

大图还有另一种与 NPU 周期不同的成本：主机求解墙钟时间。官方核内启发式、合法性检查和全局事件评估对大型图更贵，若每生成一个局部候选就重新准备全部核内任务，求解过程会浪费大部分时间。对已有结构使用哈希键复用只读准备结果、对商图改变区域增量检查无环，减少的是找出方案的主机计算；它不改变候选的官方 $T^A(p)$ 。本轮 100 图多核热搜索记录的最长时长约 363.859 秒，含义仅限该阶段；冷启动单核基准和独立最终回放需另外计时。因此本文没有把“每张图端到端 5–10 分钟”作为已通过的全量实测结论。

4.11 多候选精修：多结构候选与模拟筛选

4.11.1 从图结构形成不同粒度的划分

单一粒度难以同时处理长链、宽层和大量独立分量。宽层中的细粒度 Task 可以分摊到多个核心，窄层或关键主链若切得过细，却会反复承担 DDR 中转与任务间隔。多候选精修因此构造一组有明确结构依据的划分，而不是只在结构感知搜索所得划分附近作扰动。所有划分仍覆盖每个非 COPY 算子且不重叠，并在加入每核序后检查扩展商图无环。

以原图拓扑层深 $d(v)$ 和层宽 w_d 描述并行结构：

$$d(v) = 1 + \max_{u \in \text{pred}(v)} d(u), \quad w_d = |\{v : d(v) = d\}|, \quad \max \emptyset = 0. \quad (40)$$

沿拓扑深度形成粗、细两档窗口；窄层主干采用较粗窗口，宽层及分支采用较细窗口。程序同时枚举若干层宽与深度阈值，保留完整弱连通分量、混合链、按张量字节划分和容量受控批次等候选。主导分量可被拆分，彼此独立的小分量则可按计算 Pipe 负载组合。窗口只是生成合法子图的候选规则，不能由层宽直接推导实际完成时间；被切断的张量还要按共同模型计入中转与等待。

为给任务放置提供较廉价的局部时间尺度，设子图 r 各 Pipe 的估计工作量为 $W_{r,p}$ ，使用“最长 Pipe 加总工作量补偿”或经核内成本校准的代理：

$$\hat{\tau}_r^{(a)} = \max_p W_{r,p} + 0.08 \sum_p W_{r,p}, \quad \hat{\tau}_r^{(b)} = \omega \max_p W_{r,p} + (1 - \omega) \sum_p W_{r,p}. \quad (41)$$

权重 ω 随候选设置变化，反映 Pipe 重叠程度的不同近似。该式是调度代理，既不宣称是总时长下界，也不以代理最优替代模拟最优。多候选精修保留这些不同代理，是为了在同一真实事件模型下检验它们提出的结构是否有效。

4.11.2 预计完成时间与后继代价引导的放置

对每个候选商图，从汇点反向计算任务的后续工作量，以关键任务优先的列表调度构造核心映射。基础放置采用 HEFT 型最早完成时间思想^[3]：在满足数据释放和同核顺序的可插入位置中选择预计完成较早者。带前瞻的候选还计入后继任务的估计代价，以避免当前任务局部完成较早，却把后继链放到不利的核心。此处的前瞻表仅作排序，跨核释放仍遵循式 (22)，共享 DDR 的最终争用由官方事件模拟计算。

与结构感知搜索的单个配置最多 12 次完整评价不同，多候选精修先保留基础候选，再追加结构层候选，按划分、映射和顺序的规范指纹去重。其五核扩展包括额外层结构及局部搜索；这些设置不能直接当成一至四核均已验证的算法收益。设候选集合为 $\mathcal{C}_0$ ，模拟阶段求

$$p_0 = \arg \min_{p \in \mathcal{C}_0 \cap \mathcal{P}_A(k)} \text{lex}(T_i^A(p), D_i(p)). \quad (42)$$

由于 $\mathcal{C}_0$ 并不保证包含结构感知搜索的最终解，多候选精修不存在逐图必优保证；即使百图均值提高，仍需检查逐图退步。

4.11.3 五核局部改进与算法步骤

五核局部搜索以已评价最优方案为起点，保持子图划分，尝试核心迁移、交换和保持拓扑合法的核心顺序调整。若修改不满足覆盖或无环条件，直接拒绝；若官方模拟的字典序目

标不改善，不替换当前最优。局部步数随图规模递减，超过 9000 个计算操作时跳过该扩展，以限制大图代价。历史运行设置了局部阶段 600 秒墙钟界限，并以已发生的构造评价成本决定是否进入局部搜索；这不是整个冷启动流程的 600 秒保证。

算法的完整步骤如下。

1. 读取图及硬件配置，计算拓扑层、弱连通分量、主链、Pipe 工作量与共享张量关系。
2. 生成完整分量、粗细窗口、混合链和容量批次等划分，分别用基础及带前瞻的列表调度映射到核心。
3. 去除重复与非法候选，逐一调用官方内层排程与共享 DDR 事件模拟，按式 (42) 保存实测最优。
4. 五核配置在规模和时间条件允许时，对固定划分执行迁核、交换与顺序邻域搜索，更新实测最优。
5. 输出子图归属与每核顺序；独立恢复并回放保存的方案，核对周期和额外搬运，不把恢复历史方案的时间作为全新搜索时间。

记结构候选数为 M ，局部提案数为 L ，一次构造、合法性检查和官方评价成本分别为 C_g, C_v, C_e ，则总成本可写为

$$O(M(C_g + C_v + C_e) + L(C_v + C_e)). \quad (43)$$

固定候选的商图无环检查为 $O(q + |E_\pi|)$ ； C_e 还受核内准备、溢出和在途事件数量影响，不能笼统写成只依赖节点数的常数。因此多候选精修的主要代价来自更丰富的候选及相应模拟，并未消除大图评价瓶颈。

4.12 两种方案的比较与适用范围

4.12.1 完整核数曲线

多候选精修的一至四核补跑及独立复核已完成，现与五核已核实组合共同组成 100 图、1 至 5 核的 500 组结果。图 3 将两方法按相同算例及固定单核分母对齐。多候选精修的五点平均加速比分别为 1.000000、1.860204、2.604827、3.272613 和 4.026824；多核均值均高于结构感知搜索，但三核总周期为 110359308，高于后者的 109880862。说明逐图等权的平均收益不能替代整批图的周期比较。五核沿用已恢复核对的交付组合，低核采用本次补跑结果；本图不构成全部配置等预算重新搜索的实验。

4.12.2 相同单核分母下的五核结果

采用相同的 100 张输入图及同一官方评价语义，将两种方案逐图配对。结构感知搜索取已交付的 r02 结果；多候选精修取 Allin2 最新交付组合，并恢复全部 100 份五核方案作官方回放。两批逐图单核分母逐项一致，恢复结果的完成周期及额外搬运均与保存的交付摘要一致。该核对支持输出结果比较，不代表重新实施了等预算的算法实验。多候选精修相对

表 7 问题一两种方案的百图五核配对比较

指标	结构感知搜索	多候选精修
逐图平均加速比	3.665903	4.026824
总完成周期 (cycles)	73170501	68905422
总额外搬运 (bytes)	1767728284	1807542424
配对图数	100	100

结构感知搜索为 66 胜、21 平、13 负，胜负按完成周期判断。平均加速比相对提高 9.85%，总完成周期下降 5.83%。前一指标对每图等权，后一指标更受大图影响，二者不可互换；多候选精修的总额外搬运反而增加约 2.25%，表明其周期收益并非来自总搬运量的整体减少，而与并行安排及关键等待有关。搬运作为次级指标单列，不替代完成周期。

图 4 按五核逐图周期改善率从小到大排列全部 100 个算例，改善率定义为“结构感知搜索周期减多候选精修周期”，再除以结构感知搜索周期。横轴是排序位置而非原始算例

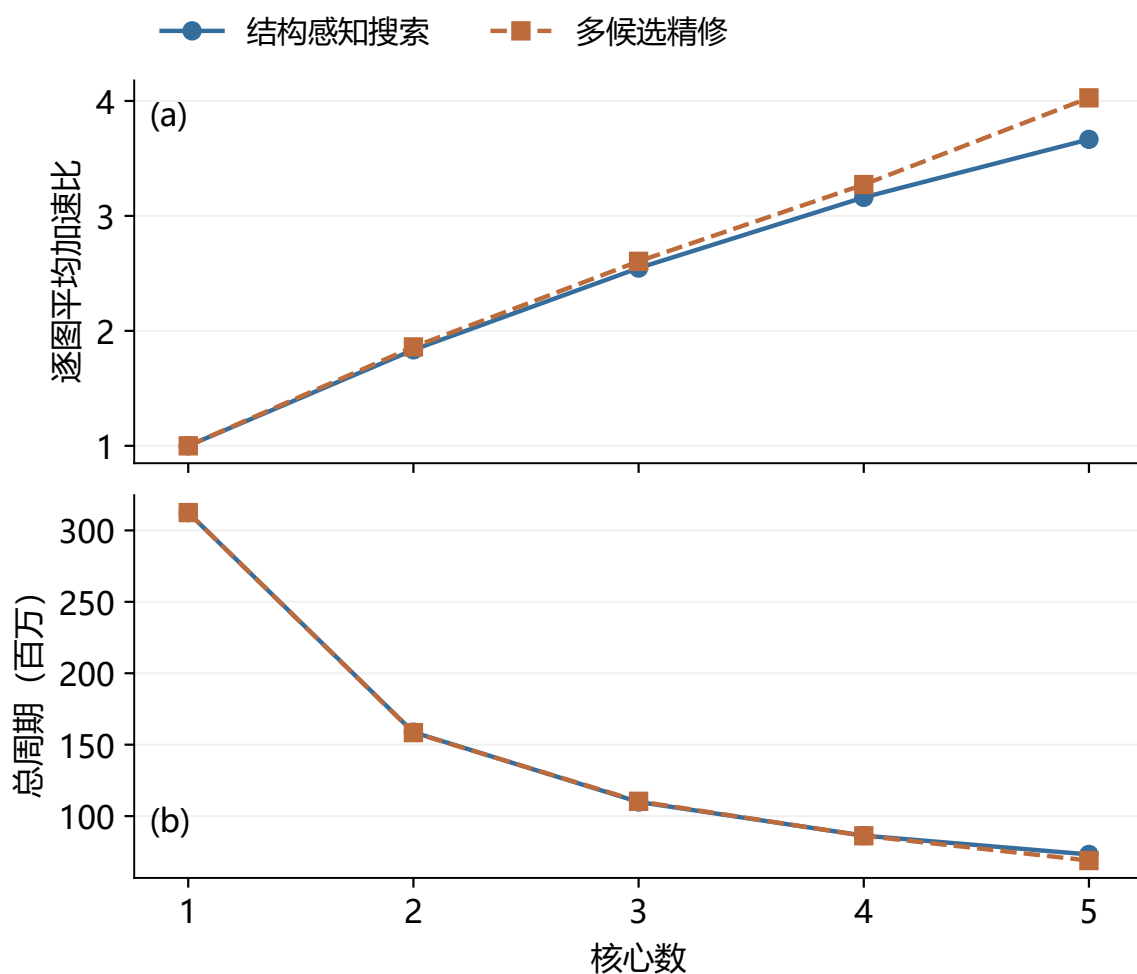


图3 两种方法在100图、1至5核下的平均加速比与总周期

编号，负值完整保留。多数改善集中于一部分图，仍有13个退步例，故不能把平均改善解释为逐图支配。

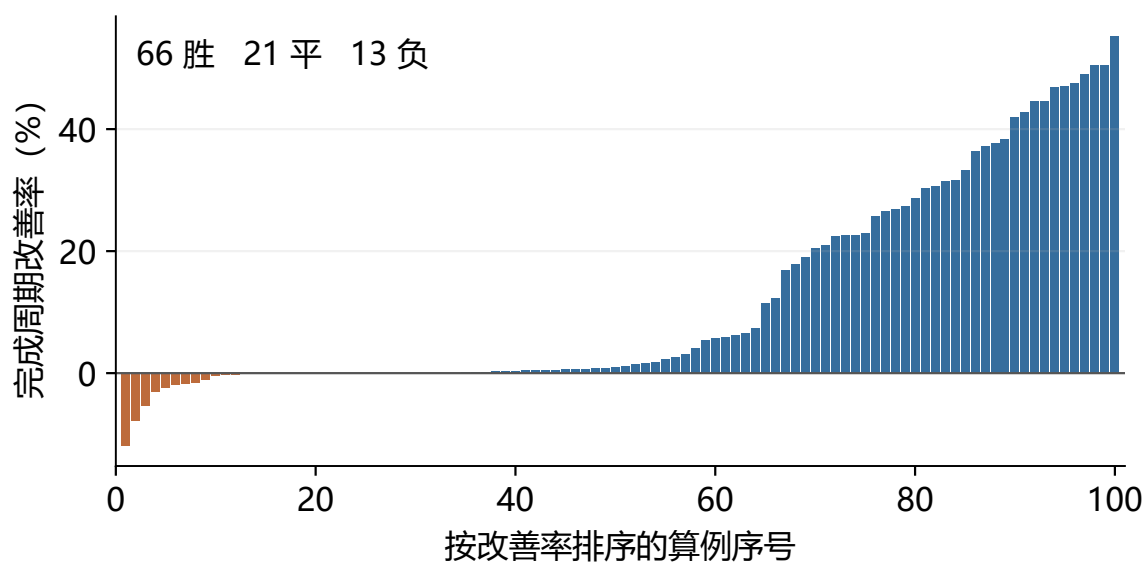


图4 五核逐图完成周期改善率分布

4.12.3 性能收益与计算预算的边界

结构感知搜索把评价机会限制为 12，适合昂贵模拟条件下的稳定基线。多候选精修覆盖更多层结构和映射候选，能改善整体五核结果，但历史全量搜索平均约 708.98 秒、最长约 3371.6 秒，49 张图的搜索评价时间超过 600 秒。这些历史统计来自全量搜索记录，并非本次恢复过程或最新交付组合的端到端计时；交付摘要还组合了不同阶段的已得方案。因此，当前对照能说明最终交付质量差异，不能单独归因于变宽划分、前瞻放置或局部搜索中的某一项。

若硬性限制在线求解资源，宜先采用结构感知搜索作为可复核基线，再在统一评价次数与统一墙钟预算下比较多候选精修。若允许离线充分搜索，可采用多候选精修已核实的五核组合。一至四核结果现已独立复核并纳入完整核数曲线，但仍需等预算消融和端到端计时，才能判断不同机制的独立收益。两种方案均为有限候选启发式，未证明全局最优。

5 问题二：同核驻留与跨核同步下的调度

5.1 场景 B 的模型扩展

问题二仍选择 (π, m, σ) ，但场景 B 不再把每个子图独立清空为一个 Task：映射到同一核心的全部子图组成该核的 Task，核内合法顺序决定缓存复用与张量生命周期。对于同一张量，若生产和消费能够在同核满足依赖，则中间结果可以驻留；跨核消费必须按实际生产核与消费核构造搬运，并在源 COPY_OUT 完成后经过 500 cycles 才能释放目标 COPY_IN。对于跨核数据事件 e ，有

$$a_e^{\text{in}} \geq f_e^{\text{out}} + 500. \quad (44)$$

这里的前驱是具体搬运事件，而非“源子图全部运行完”的粗粒度时刻。对多消费者共享张量，外部输入按消费核心而非每个算子独立读取；跨核中间张量按实际源核—目标核关系生成，避免把每条逻辑边简单等同一次物理 COPY。

合法集合 $\mathcal{P}_B(k)$ 除保留覆盖、子图商图和核上顺序的无环性外，还要求由官方核内展开得到的物理实例满足 L1/UB 在每个时刻的容量约束。四条 Pipe 各自串行，但不同 Pipe 在依赖允许时可重叠；同一 DDR 池服务所有在途外存搬运。张量能否驻留依赖实际产生、消费和溢出时刻，不能只按静态张量大小之和判断。令 $\mathcal{E}_B$ 表示题目的事件模拟，问题二的目标为

$$\min_{p \in \mathcal{P}_B(k)} \text{lex} (T_i^B(p), D_i(p)), \quad T_i^B(p) = \mathcal{E}_B(G_i, p; k). \quad (45)$$

这仍是依赖官方事件过程的组合优化模型，不把启发式估价器当作精确求解的线性目标。与问题一相比，映射会同时改变通信、驻留、释放和并行，所以切图后固定映射、或仅按分组时长排序，均可能错过有效方案。处理器互连与片上存储的耦合也是相关体系结构研究关注的问题^[4]；本题仍以给定的缓存及 COPY 语义为准，不移植其他硬件的通信规则。

5.2 从独立 Task 到每核联合 Task

场景 B 保留同一个划分 π 、核心映射 m 和核上顺序 σ 决策，但改变了这些决策的物理含义。对每个核心 c ，把映射到 c 的全部子图按 σ_c 展开到一个 Task 的内部：子图之间仍有次序，然而私有 L1/UB 状态在这段次序中连续存在。若生产与最后消费同在 c ，中间张量可在容量允许时直接驻留；外层切出两个子图并不自动产生一次 DDR 往返。相反，两个核心之间的依赖须经源端写出、目标端读入与同步释放。由此，场景 A 的“每跨一个子图边界就增加传输”在 B 中不再成立，决策的风险从 Task 边数转向跨核心的逻辑张量集合与同核生命周期长度。

记 $c_p(x)$ 为内部张量 x 的生产核心， $\mathcal{C}(x; p) = \{m(\pi(v)) : v \in C(x)\}$ 为消费核心的去重集合。目标核心数

$$n_{\text{remote}}(x; p) = |\mathcal{C}(x; p) \setminus \{c_p(x)\}| \quad (46)$$

给出跨核心目的地的数目，而不是跨核算子边条数。若同一核心有三个消费算子，集合里仍只出现一次。对于原始外部输入，不存在内部生产核心；其各消费核心可共享一次该核

心的加载机会。这个去重关系用于静态候选估价，但是否产生源端一次或多次 COPY、原图已经有多少基础 COPY、是否因容量溢出再读，均由官方 Task 构造及核内展开确定。因此问题二沿用

$$D^B(p) = B_{\text{all}}^B(p) - B_{\text{orig}}, \quad (47)$$

作为可交付的额外搬运定义；不直接把 $b_x n_{\text{remote}}(x; p)$ 宣称为总额外字节。对按边计费的朴素代理，令 $\mathcal{E}_x^{\text{remote}}$ 为张量 x 的跨核消费者边集合，则目标核心去重的读入字节代理满足

$$b_x n_{\text{remote}}(x; p) \leq b_x |\mathcal{E}_x^{\text{remote}}|. \quad (48)$$

不等号可严格成立，说明朴素边权和可能高估共享消费者的通信，从而错误惩罚应考虑划分。

5.3 跨核释放与事件路径

对一个源核心 a 到目标核心 b 的张量传递，设 e^{out} 为源端对应 COPY_OUT， e^{in} 为目标端 COPY_IN。在场景 B，后者的发射条件是

$$s_{e^{\text{in}}} \geq f_{e^{\text{out}}} + \delta_B, \quad \delta_B = 500 \text{ cycles}. \quad (49)$$

等待从实际写出完成开始计，不是从源子图的名义结束开始计。若源核还有与这份张量无关的矩阵计算，这段计算可能在写出完成后继续；目标核只需等待它自身需要的 COPY 就绪，而不是盲等源核整个 Task 完成。一个目标操作若依赖多个数据来源，其就绪时刻取相关发射与完成约束的最大值。把多条前驱的 500 周期全部相加，会夸大并行等待；把所有源 Task 一起拉长 500，则会错误阻塞无关 Pipe。表 2 所示三个场景在此处产生的差别，必须在事件层而非仅在子图时长层表达。

场景 B 的完整事件依赖包括：原算子数据边、各 Pipe 的顺序边、跨核 COPY 写出到读入的释放边，以及官方核内启发式插入的内存复用和换入换出边。若把所有源到汇路径记为 $\Gamma_B(p)$ ，则在真实事件服务时长确定后仍可使用式 (8) 解释总时间；但搜索前无法仅由子图大小求得这些路径。特别地，跨核释放 δ_B 可以与另一核心的计算并行，只有位于决定最终输出的路径时才直接增加 Makespan。

5.4 真实物理驻留与容量约束

令 $\mathcal{R}_{c,r}^{\text{phys}}(t)$ 表示时刻 t 实际驻留于核心 c 、私有池 r 的张量实例。场景 B 与 A 一样必须满足

$$\sum_{x \in \mathcal{R}_{c,r}^{\text{phys}}(t)} b_x \leq C_r, \quad r \in \{\text{L1}, \text{UB}\}, \quad \forall c, t. \quad (50)$$

不同的是，场景 B 的同核子图共享一段 Task 生命周期。为了估计不溢出时的压力，可定义某张量在核心 c 的理想驻留区间 $I_{x,c} = [t_{x,c}^{\text{first}}, t_{x,c}^{\text{last}}]$ ，并定义同时存活字节

$$L_{c,r}^{\text{ideal}}(t) = \sum_{x: x \text{ 属池 } r} b_x \mathbf{1}\{t \in I_{x,c}\}. \quad (51)$$

如果 $L_{c,r}^{\text{ideal}}(t) > C_r$ ，这份完全驻留的理想日程不可能直接执行；核内调度必须改变操作顺序、及时释放，或把部分张量换出并在再次使用时换入。反过来， $L_{c,r}^{\text{ideal}}(t) \leq C_r$ 也不足以断言不会有搬运开销：原始输入读入、输出写回、Pipe 冲突和启发式实际排程仍在。式 (51) 是压力诊断而非赛题评价器的替身。

若同核聚集更多子图，跨核消费核心集合可能缩小，式 (46) 的通信代理下降；但某些中间张量的 $t^{\text{last}} - t^{\text{first}}$ 可能变长，驻留峰值上升，触发换出换入。这正是“合并后反而更慢”的一种可检验机制。以生命周期面积

$$A_{c,r} = \int_0^{T(p)} L_{c,r}^{\text{ideal}}(t) dt \quad (52)$$

描述整体占用强度有助于排序候选，但它不能取代峰值约束 (50)：相同的面积可能由短时高峰或长时低占用形成，前者更容易引发容量违规。最终物理实例仍以官方核内输出和全局内存审计为准。

5.5 场景 B 的严格下界与局部判断

式 (10) 对 B 继续成立。对固定方案 p ，将所有源到汇的算子依赖链记为 Γ_c 。链 γ 上的固定计算周期和为 $C(\gamma)$ ，穿越不同核心的依赖次数为 $n_\times(\gamma; p)$ 。每次跨核依赖须经过式 (49) 的 500 周期释放，且链上先后发生，故有条件下界

$$T^B(p) \geq L_{\text{sync}}(p) := \max_{\gamma \in \Gamma_c} \{C(\gamma) + 500 n_\times(\gamma; p)\}. \quad (53)$$

这仍忽略写出、读入本身的服务与共享带宽竞争，因此是保守界。多个平行分支上的同步不能相加，但沿同一条连续路径发生的释放可以累积。结合实际展开 DDR 字节数得到

$$T^B(p) \geq \max \{L_0(k), L_{\text{sync}}(p), B_D^B(p)/60\}. \quad (54)$$

后两项随划分和映射变化，不能用于声称“当前方案离全局最优只差某百分比”。它们可用于判别：如果 $L_{\text{sync}}(p)$ 接近实测时间，优先消除链上核心切换；如果带宽项接近实测时间，优先检查多核心重复读取和在途拥塞。与问题一相比，跨核同步而非每个子图边界成为关键链的主要显式释放代价。

考虑移动一个边界子图 S 至相邻核心。该动作可能同时改变三项： S 的生产张量跨核目标数、消费张量的读入核心集合，以及本核张量的最后使用时刻。令 Δ_{comm} 、 Δ_{live} 、 Δ_{crit} 分别表示由这些因素构成的代理变化，则可用

$$\hat{\Delta}(S, c) = \alpha \hat{\Delta}_{\text{crit}} + \beta \hat{\Delta}_{\text{comm}} + \gamma \hat{\Delta}_{\text{live}} \quad (55)$$

为有限候选排序；权重不是赛题的真实目标系数，也不用于公布收益。对任意迁移、合并或拆分，真实判别始终是

$$\Delta T = T^B(p_{\text{old}}) - T^B(p_{\text{new}}). \quad (56)$$

尤其不能从“删除 b 字节跨核搬运与一次同步”推断必节省 $b/60 + 500$ 周期：传输可能原本与别的计算重叠，迁移可能令另一条链成为新的瓶颈，或推高 L1/UB 溢出。式 (55) 说明为什么值得提出该动作；式 (56) 才决定是否采用。

5.6 保护基础搜索的限时算法组合

本轮采用结构初解与自适应有界邻域的组合。对输入图先构造弱连通分量、共享输入消费者、拓扑层次和各 Pipe 工作量索引；给定迁移种子时，必须在场景 B 下重新评价。初始化保留整图候选、原有结构菜单和四次普通联合修复机会，再启动扩展搜索。这里的“四次”只描述初始化阶段，不能把整个新算法误写成固定十二次评价。

令 f 表示重映射、粒度调整、顺序扰动、联合修复、局部插入、拓扑前沿或分量装箱的一类候选。定义最大分量占比

$$\chi(G) = \frac{\max_{C \in \text{WCC}(G)} |C|}{|\mathcal{O}_c|}. \quad (57)$$

当 $\chi(G) > 0.5$ 时，提高重映射与粒度类的先验份额；存在多个独立分量时才开放分量装箱。这些规则只依赖图结构，不按算例编号查答案。前期优先保留普通联合搜索：前 32 次提议且位于自适应阶段剩余时间的前 40% 时采用该基础邻域；随后在不同候选类间按收益与耗时调整抽样。令 G_f 为累计相对周期改善、 t_f 为该类累计墙钟耗时、 w_f 为结构先验，则抽样权重为

$$\tilde{w}_f = w_f \left[1 + \min \left(8, \frac{100G_f}{\max(0.1, t_f)} \right) \right], \quad P(f) = \frac{\tilde{w}_f}{\sum_g \tilde{w}_g}. \quad (58)$$

未获收益的候选类仍保留正权重。由于 t_f 取实测秒数，相同随机种子在不同主机和并发负载下不保证产生完全相同的搜索轨迹；已保存方案的官方重放则与搜索轨迹无关。

每个多核搜索设 590 秒上限，其中预留 20% 给最终原版评价和物理内存审计，自适应提议至多 384 次。每个提议可以被合法性检查、重复指纹或剩余时间检查提前拒绝，提议数不等于实际官方评价数。停止条件还参考最近四次评价的最大耗时，剩余时间不足其 1.3 倍时不再启动下一项。该经验预留并不是任意图上的严格完成时间证明；正式结果只接纳完整复核成功的输出。

为减少重复生成成本，在同一不可变图对象内，缓存上下文构造、顺序构造和局部代理函数的确定性结果。缓存键包含函数名及完整实参的摘要，返回值反序列化为独立对象，避免后续修改污染缓存；总载荷上限为 32 MiB，过大对象绕过缓存。缓存不改变外层随机数调用，不复用旧方案的全局 DDR 时长。划分、映射或顺序变化后，正式评价重新计算完整核内排程、物理内存和全局事件。

候选 p 仅在官方评价合法且 $(T(p), D(p))$ 字典序优于 incumbent 时替换。对实际评价的合法集合 $\mathcal{A}$ ，最终返回

$$p^* \in \arg \min_{p \in \mathcal{A}} \text{lex}(T^B(p), D(p)). \quad (59)$$

这一有限集合择优性质由逐次更新直接归纳得到，既不证明全局最优，也不保证另一台机器在同一秒数内访问同一个 $\mathcal{A}$ 。全量实验固定规则、种子与配置，并单独披露历史初解成本。

5.7 典型算例的执行时间线

图 5 采用第 12 图的五核配置，把已声明输入初解与最终方案放在同一场景 B 官方评价器下比较。初解完成周期为 13406，最终为 11971，下降 10.70%。图中按每核分别合并同类计算、搬运操作的实际执行区间，保留二者重叠；空白只表示该类操作当时未执行，不单凭空白判定等待原因。虚线标明全图完成时刻。该图揭示末段完成时刻与活动分布的变化，不把全部改善归因于某一种局部操作。

5.8 百图结果与验证

问题二五核逐图平均加速比为 4.264407，百图总周期为 65 342 192。

总额外搬运为 1 063 835 222 字节。各配置保留完整官方结果与物理内存审计。历史初解作为显式输入，其生成不包含在本轮限时搜索内。

表 8 问题二百图汇总

核数	平均加速比	总周期	额外搬运
1	1.000000	312 479 373	1 241 686 120
2	2.019635	153 426 507	1 157 663 840
3	2.867797	103 587 831	1 037 566 148
4	3.640804	79 392 460	1 089 899 086
5	4.264407	65 342 192	1 063 835 222

算法在每个核数独立搜索，增加核心数并不保证每个图都更快。全量结果证明与给定官方仿真一致，不构成全局最优证书或未见图泛化检验。

6 问题三：共享 FIFO 缓存下的多处理器调度

6.1 缓存状态与双带宽事件模型

问题三继承场景 B 的合法方案集 $\mathcal{P}_B(k)$ 、跨核释放与核内内存约束，新增容量为 C 的共享只读 L2。它不是每核各有一份，也不改变已生成的逻辑 COPY 数量。对时刻 t 按进入先后

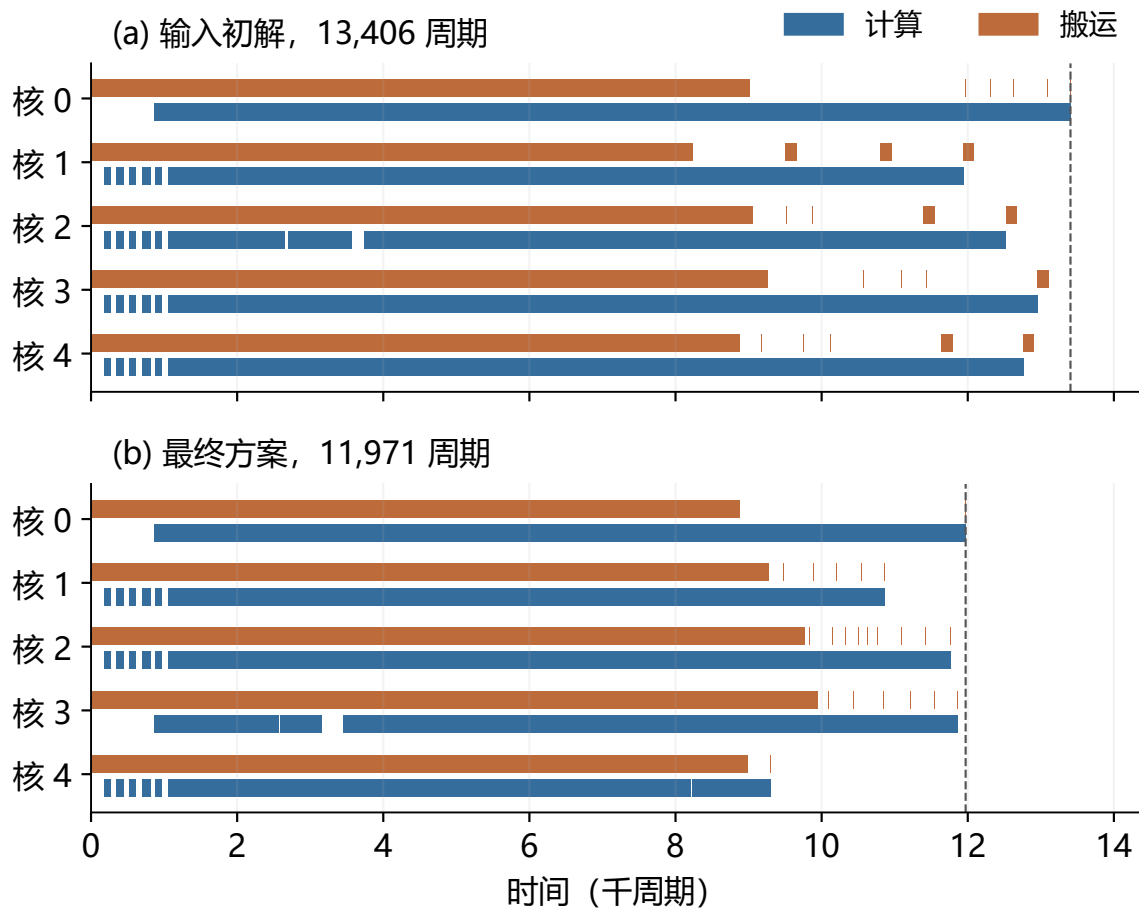


图 5 第 12 图五核初解与最终方案的官方执行时间线

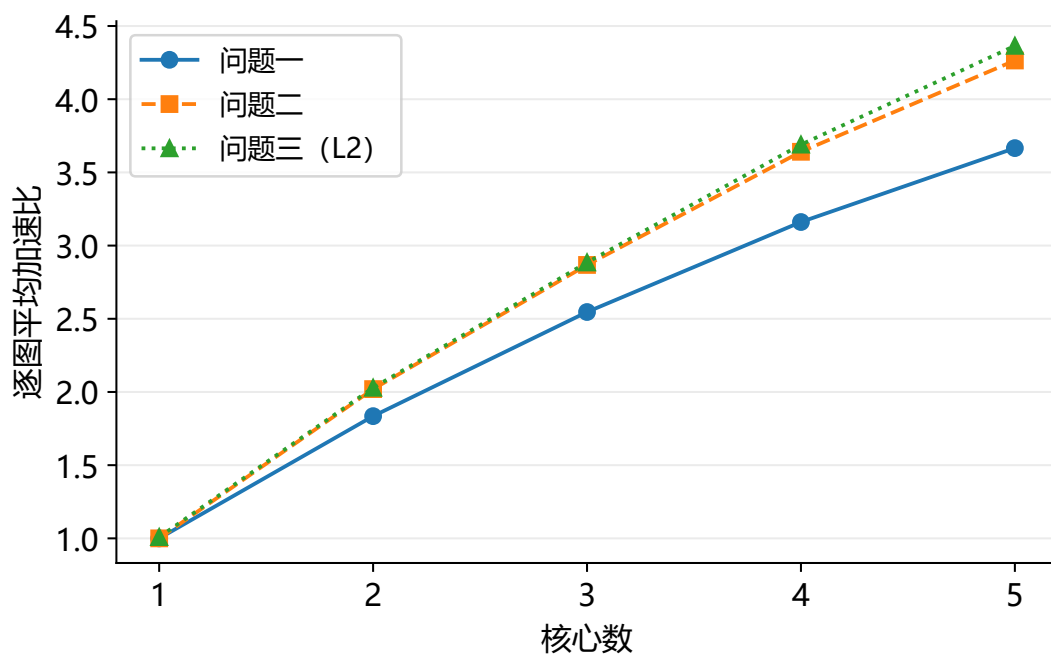


图 6 三问逐图平均加速比（问题一取结构感知搜索；L2 单核沿用无 L2 参考）

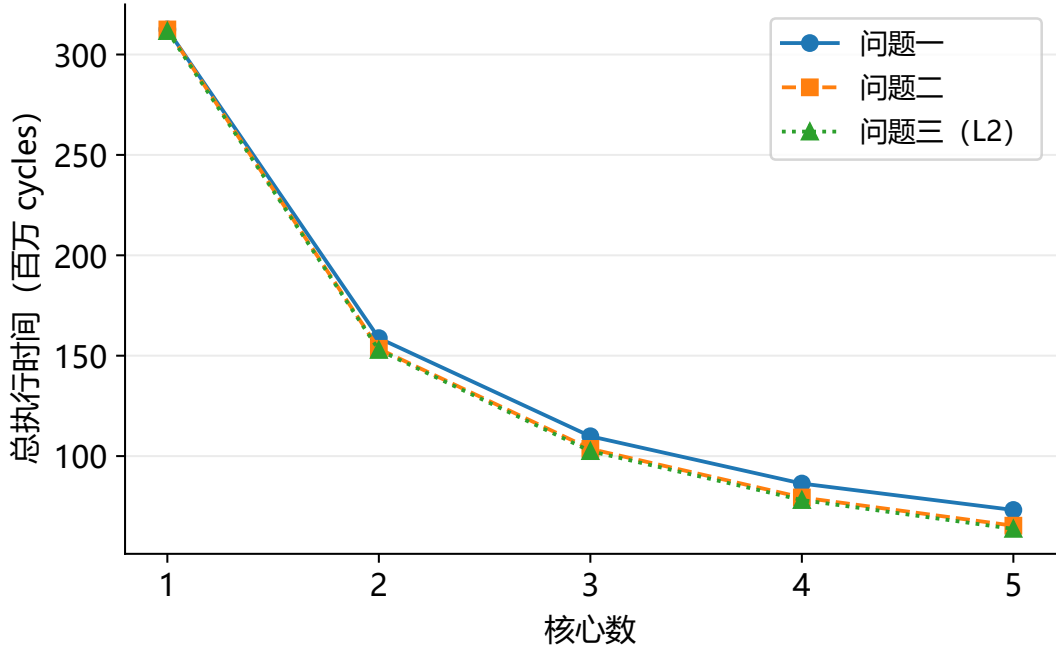


图 7 三间百图总周期（问题一取结构感知搜索）

排序的逻辑张量队列记为 $Q(t)$ ，有

$$\sum_{x \in Q(t)} s_x \leq C, \quad C = 1\,048\,576 \text{ bytes}. \quad (60)$$

正字节 **COPY_IN** 事件 e 在发射时刻 a_e 查询逻辑张量标识。若标识已在 $Q(a_e^-)$ 中则 $h_e = 1$ ，改由 L2 服务；否则 $h_e = 0$ ，仍由 DDR 服务。命中不刷新 FIFO 次序。搬运在完成时按官方处理顺序进行填充：未驻留且大小不超过 C 的张量，从队首淘汰至有足够空间后入队；超容量张量旁路。两个同时发射的冷读不能因为较早一个尚在途中而合并为一次命中。完成时的重新插入、相同时间事件次序等均保持官方语义。

设 $A_D(t), A_L(t)$ 分别为 DDR 和 L2 池的在途操作集合， $b_D = 60$ 、 $b_L = 250$ bytes/cycle。每条搬运按所属池的独占带宽形成初始服务工作量，活跃集合不变的时间段内，各自池内公平分享服务：

$$\dot{w}_e(t) = -\frac{1}{|A_r(t)|}, \quad e \in A_r(t), \quad r \in \{D, L\}. \quad (61)$$

事件到达或完成后须结算剩余工作并重算完成时刻；实际评价还依官方整数周期取整。两池独立，不能把 $60 + 250$ 当作共同带宽，也不能给每个核心各分配一条 250 bytes/cycle 的 L2 通道。L2 命中只改变 COPY 的服务池，不能绕过 (44) 中的跨核同步，不能占用别的 Pipe 插队，亦不能使超出 L1/UB 的物理实例合法。

于是问题三在相同合法方案集上求

$$\min_{p \in \mathcal{P}_B(k)} \text{lex}(T_i^{L2}(p; C, b_L), D_i(p)). \quad (62)$$

字节命中率为

$$H_i = \frac{\sum_{e: \text{COPY_IN}, s_e > 0} h_e s_e}{\sum_{e: \text{COPY_IN}, s_e > 0} s_e}. \quad (63)$$

分母为零时只在统计程序中定义为零并标注“无有效读取”。官方命中率按字节而非按次数统计。 D_i 是插入额外逻辑 COPY 的字节量，不因命中自动扣减；同一方案分别在无 L2 和 L2 下评价，该字段应相同。命中字节可作为实际 DDR 服务减量的辅助诊断，但 H_i 越

高未必总周期越短，因为同步、计算、L2 带宽竞争以及关键路径变化仍可能占主导。

6.2 缓存微事件及状态转移的完整定义

问题三的方案空间仍是满足场景 B 合法性条件的 $\mathcal{P}_B(k)$ ，并未新增“选择某张量是否命中”的自由变量。命中由方案诱导的 COPY 发射时刻与全核共享 FIFO 状态内生决定。为区别实际缓存与外层预测，以离散事件序号 j 表示官方确定的同刻处理顺序，记

$$Q_j = (x_1, \dots, x_{m_j}), \quad u_j = \sum_{r=1}^{m_j} b_{x_r} \leq C, \quad Q_0 = \emptyset. \quad (64)$$

队列从左至右按进入缓存的先后排序， $C = 1048576$ bytes。对正字节 COPY_IN 事件 e ，在发射时读取其逻辑张量标识 ℓ_e ，确定

$$h_e = \mathbf{1}\{\ell_e \in Q_{j(e)^-}\}, \quad r(e) = \begin{cases} L, & h_e = 1, \\ D, & h_e = 0, \end{cases} \quad (65)$$

其中 L, D 分别是 L2 与 DDR 服务池。查询命中不把张量移到队尾，这与最近最少使用策略不同。服务池在发射时已经确定；即使传输在途期间该张量被别的完成事件淘汰，本次请求也不临时改道。对于非正字节读取与 COPY_OUT，按官方特殊规则执行，但不把它们计入“正字节读入命中率”的分子或分母。

填充发生在读入完成时。令事件完成的逻辑张量为 x 。若标识无效、 $b_x > C$ 或 x 已在 Q_j 中，队列保持；否则选择最短的队首淘汰前缀长度

$$h^* = \min \left\{ h \in \{0, \dots, m_j\} : u_j - \sum_{r=1}^h b_{x_r} + b_x \leq C \right\}, \quad (66)$$

再定义

$$Q_{j+1} = (x_{h^*+1}, \dots, x_{m_j}, x), \quad u_{j+1} = u_j - \sum_{r=1}^{h^*} b_{x_r} + b_x. \quad (67)$$

对于 $b_x \leq C$ ，取 $h = m_j$ 总能容纳，故式 (66) 的集合非空。式 (67) 是 FIFO：只删除队首前缀，幸存者相对次序不变，新张量放队尾。原先命中的在途读取若在完成前被淘汰，完成时若缓存中仍无该逻辑张量，也按相同规则重新插入；若已由别的完成事件插入，则不刷新队列。两个几乎同时发射的冷读在尚无完成填充时均可未命中，不能为了提高命中率私自合并。

这些规则给出三个可证明的不变量。首先， Q_0 不含重复标识，插入前先判驻留，故归纳可得所有 Q_j 中标识唯一。其次， $b_x > C$ 的张量旁路，其他新入张量按式 (66) 淘汰到足够空间，所以 $0 \leq u_j \leq C$ 恒成立。第三，命中只读取、不重排，淘汰只删前缀，故幸存张量的 FIFO 相对顺序保持。由发射时的二值判定，每个有合格逻辑键的正字节读取恰好计入命中或未命中之一。这是实现缓存事件账本时的局部校验条件，也说明只按访问频次估算 L2 收益会遗漏顺序与在途状态。

6.3 两个共享服务池及命中率

问题三保留共享 DDR 总带宽 $b_D = 60$ bytes/cycle，并新增全核共同的 L2 读带宽 $b_L = 250$ bytes/cycle。两条带宽属于不同池；一个 L2 命中请求不能同时按 DDR 与 L2 服务，也不能把两者相加为一个 310 bytes/cycle 的通用资源。对事件 e 的正字节 b_e ，定义独占参考服务工作量

$$q_e^r = \left\lceil \frac{b_e}{b_r} \right\rceil, \quad r \in \{D, L\}. \quad (68)$$

在活跃集合不变时各池分别按式 (61) 分享，事件到达或完成后结算剩余工作，再由官方的整数事件顺序决定下一次状态转移。整数上取整、零字节请求及同刻多个完成事件的处理

可以改变下一次发射与 FIFO 次序，故这里的连续服务式是物理解释，精确主结果仍取官方事件仿真。

令 $R(p) = \sum_{e \in \mathcal{R}_{in}(p)} b_e$ 为固定方案下所有合格正字节读入量， $X(p)$ 为其余必须经 DDR 的逻辑搬运量， $H(p, C, b_L) = \sum_e h_e b_e$ 为命中字节。固定方案展开的 COPY 集合不因只读 L2 本身改变，因此两池的字节工作量可写成

$$D_D = X + R - H, \quad D_L = H, \quad H_B = \begin{cases} H/R, & R > 0, \\ 0(\text{标记为无有效读取}), & R = 0. \end{cases} \quad (69)$$

H_B 是字节命中率，不是命中请求次数除以请求总次数。式 (69) 还说明：命中会改变由哪个服务池承担读取，却不自动删除方案中的逻辑 COPY 事件。因此同一个 p 在 B 与 L2 两种硬件环境下评价，逻辑额外搬运 $D(p)$ 应相同，虽然实际 DDR 服务字节可减少。若缓存感知搜索最终改用另一个 p_L ，其划分和通信本身改变， $D(p_L)$ 则可能升高。

仅由共享带宽的容量限制可得

$$T^{L2}(p) \geq \max \left\{ \frac{X + R - H}{b_D}, \frac{H}{b_L} \right\}. \quad (70)$$

两项取最大而非相加，因为两个服务池可并行运行，计算 Pipe 也可与两者重叠。若只观察 H_B 增加，式 (70) 的 DDR 项下降，但 L2 项上升，且真实关键路径可能转移；“命中率升高必然缩短 Makespan”不成立。对于完成时间的更强条件界，还可与式 (10)、固定方案每核计算 Pipe 负载取最大；这些界不能反推出真实完成时间等于各资源忙碌时长之和。

6.4 乐观命中上限与容量、带宽包络

对固定方案 p ，设逻辑张量 x 被正字节读取 $m_x \geq 1$ 次、每次大小为 b_x 。空缓存必须经历至少一次冷读； $b_x > C$ 的张量根本不能驻留。因此无需知道访问次序便有乐观上限

$$0 \leq H \leq H_{\max}(p, C) = \sum_{x: b_x \leq C} (m_x - 1)b_x. \quad (71)$$

等号通常难达到：并发冷读可能多次未命中，不同张量之间的 FIFO 淘汰也可能让已读张量再次变冷。式 (71) 只回答“最多有多少重复读取字节有资格转向 L2”，并不保证这些字节真实命中。

为了说明硬件收益的理论边界而非预测实测，将 H 放宽为区间 $[0, H_{\max}]$ 内可任意选择的连续变量 z 。对固定 p 的两池负载，构造

$$\Phi(p; C, b_L) = \min_{0 \leq z \leq H_{\max}(p, C)} \max \left\{ \frac{X + R - z}{b_D}, \frac{z}{b_L} \right\}. \quad (72)$$

第一项随 z 下降，第二项随 z 上升，其无约束交点解由

$$\frac{X + R - z}{b_D} = \frac{z}{b_L} \implies z_0 = \frac{(X + R)b_L}{b_D + b_L}.$$

由于 $z_0 \geq 0$ ，受上限截断后的最优放宽值为

$$z^* = \min\{H_{\max}(p, C), z_0\}, \quad \Phi(p; C, b_L) = \max \left\{ \frac{X + R - z^*}{b_D}, \frac{z^*}{b_L} \right\}. \quad (73)$$

真实 H 必在放宽区间中，故 $\Phi(p; C, b_L) \leq T^{L2}(p)$ 。容量 C 增大时 $H_{\max}$ 不减，放宽可行域扩大；带宽 b_L 增大时，对于每个 z 的第二项不减，所以 Φ 对两参数均不减。这是乐观包络的单调性，不是 FIFO 真实周期的单调性证明。真实队列与完成顺序会随容量、带宽改变，甚至使某个固定方案在带宽增大时略慢。

全空间最优值也必须与有限候选和方案条件界分开。定义

$$T^* = \min_{p \in \mathcal{P}_B(k)} T^{L2}(p), \quad U_A = \min_{p \in \mathcal{A}} T^{L2}(p), \quad (74)$$

其中 $\mathcal{A} \subset \mathcal{P}_B(k)$ 是实际完成官方评价的有限集合。由式 (10) 和集合包含关系，严格成立

$$L_0(k) \leq T^* \leq U_{\mathcal{A}}. \quad (75)$$

若还定义全空间放宽下界 $L^* = \inf_{p \in \mathcal{P}_B(k)} \max\{L_0(k), \Phi(p; C, b_L)\}$ ，则 $L^* \leq T^*$ 。但只在有限候选集上取 $\min_{p \in \mathcal{A}} \Phi(p; C, b_L)$ 不能替代 L^* ，更不能用 $T(p) - \Phi(p)$ 宣称剩余优化潜力。本文没有求得全空间 L^* 或 T^* ，只把这些式子用于解释何种资源可能限制已选方案。

6.5 同一张量并发读取的时序例子

缓存状态不能仅由“这一轮请求的张量集合”决定；发射与完成顺序是必需的状态变量。设某个合格张量 x 尚未驻留，两个核心的读取请求 e_1, e_2 在 $t_1 < t_2$ 发射，而第一请求的完成时刻为 $f_1 > t_2$ 。在两个发射瞬间 $x \notin Q(t_1^-)$ 且 $x \notin Q(t_2^-)$ ，所以按官方发射时查询规则，两次均为未命中：

$$h_{e_1} = h_{e_2} = 0, \quad t_1 < t_2 < f_1. \quad (76)$$

若代理只统计“ x 在这一批被读取两次”，便可能误用式 (71) 给它一次命中。式 (71) 确实允许一次命中的乐观上限，但实际顺序 (76) 实现的是零次。第二个 DDR 请求在 t_2 加入还会降低第一个请求的瞬时带宽份额，两个请求彼此影响，即使它们来自不同核心且不具有算子依赖。第一个完成后， x 才可能入 FIFO；第二个请求已经选定 DDR 池，不能中途转为 L2。第二个完成时若 x 仍在队列，按规则不刷新其入队位置；若中间被其它事件淘汰，才可能重新插入。这个例子证明缓存状态和传输在途状态必须同时进入事件模型。

再设缓存只能容纳两个大小均为 b 的张量，初始队列 $Q = (x, y)$ ，依次发生对 x 的命中、一个新张量 z 的读入完成、之后对 x 的查询。由于 FIFO 命中不刷新顺序，第一次命中后仍是 (x, y) ； z 填充时删除队首 x ，队列变成 (y, z) ；最后一次 x 查询为未命中。若把该机制错误实现成 LRU，第一次命中会形成 (y, x) ，填充时淘汰 y ，最后一次查询反而命中。这两个序列仅有替换语义不同，却可能给出不同 DDR 工作量和 Makespan。因而问题三即使沿用问题二同一划分，也必须重新按 FIFO 规则执行官方仿真，不能用 LRU 的经验命中率系数乘原 DDR 时间。

还需区分缓存队列大小和实际占用的物理通道。一次 L2 命中读取在逻辑上仍是 COPY_IN，其目的端私有存储在执行时仍必须符合 L1/UB 容量；共享 L2 的一兆字节不能被算作每核额外的一兆字节 UB。对张量 x ，若它在 L2 中驻留并被多个核心命中，这只说明源端读取可从 L2 带宽池服务，不能证明这些核心的计算都可同时持有私有副本，也不删除跨核源写出后的 500 周期释放。缓存提升的是某些读入的服务途径，不直接修改核内计算 Pipe 或算子依赖。由此可在理论上解释为什么一次命中有时只降低 DDR 忙碌时长，却没有缩短最终输出控制路径。

6.5.1 真实张量的 FIFO 生命周期

图 8 摘取第 39 图最终五核方案中张量 1000000005 的官方缓存事件。该 4 KiB 张量在 39485 周期填入，在 51610 周期命中；命中不改变 FIFO 先后，随后在 63967 周期被新填充挤出，77594 周期的再次读取仍然未命中，直到 77801 周期重新填入。图中节点等距排列只表达事件顺序，数值是实际发生周期。该例说明“曾经命中”不等于后续一直驻留，也解释了缓存邻域为何必须结合读取时刻和淘汰事件，而不能只统计复用次数。

6.6 共享缓存下的组合搜索与同方案对照

问题三采用与问题二相同的有界结构候选族，并增加由实际 FIFO 时间线引导的缓存邻域。先在 L2 规则下重新评价声明的场景 B 种子和已有 L2 锚点，之后围绕重复读取的消费者、合法迁核与顺序插入生成候选。缓存候选按当前最优方案指纹组织有限菜单，不能直接更改 FIFO 淘汰规则或手工指定命中事件。

自适应阶段前 16 次提议且处于预留探索时间的前 40% 时，交替尝试缓存邻域和普通联合邻域；随后先补充未探索的类别，再依据结构先验、实测收益和耗时分配机会。缓存类初始权重为 2，拓扑前沿为 1，存在独立分量时开放装箱类。总时限、最终复核预留、提议上限及生成准备复用与问题二一致。命中率只解释数据服务路径，最终仍按 (T^{L2}, D) 选

case039 · 五核 · 张量 1000000005 (4 KiB)



标注为真实周期；节点间距仅表示事件顺序

图 8 第 39 图一个只读张量的真实 FIFO 事件链

择，不能以命中更多代替周期更短。

本轮初解是显式保存的历史输入：二至四核与五核的场景 B 种子来自自己登记的不同证据批次，均在 L2 下重新评价；不是将本轮新问题二的结果自动传给问题三。因此，跨问最终成绩的比较只表示各自选定方案的综合差别，不代表单独的硬件收益。

为隔离硬件作用，对最终选定的同一问题三方案 $p_{L,i,k}$ 额外执行无 L2 官方重放，定义

$$R_{i,k}^{\text{same}} = \frac{T_i^B(p_{L,i,k})}{T_i^{L2}(p_{L,i,k})}, \quad \bar{R}_k^{\text{same}} = \frac{1}{100} \sum_{i=1}^{100} R_{i,k}^{\text{same}}. \quad (77)$$

同时报告 $T_i^B(p_{B,i,k})/T_i^{L2}(p_{L,i,k})$ 的逐图均值作为跨方案综合比较。前者保持划分、核心与顺序不变，后者同时改变方案；两者不得互称。附录为最终问题三方案列出无 L2 与只读 L2 两种配置的周期、额外搬运量和命中率。相同方案两次评价的逻辑额外搬运应相等，服务池变化只影响物理传输及完成时刻。

题目前两问的单核加速比固定为 1；本问若展示相对无 L2 单核基准的 L2 曲线，单核点可以大于 1。该变化来自同一整图方案启用缓存，不能冒称单核并行收益。所有平均值均先按图计算比值，再对图取算术平均。

6.7 百图成对结果

五核相对固定无 L2 单核参考的平均加速比为 4.364503，总周期为 63 759 268。固定最终方案只切换 L2 的逐图同核时间比为 1.030244；相对本轮问题二所选方案的综合时间比为 1.027332。后者不能单独归因于缓存硬件。

表 9 问题三百图成对结果

核数	L2 平均加速比	L2 总周期	同方案硬件比	跨方案综合比
1	1.008622	311 669 310	1.008622	1.008622
2	2.028605	152 849 324	1.004722	1.004040
3	2.884632	102 478 047	1.006767	1.006464
4	3.689639	78 107 717	1.018935	1.014140
5	4.364503	63 759 268	1.030244	1.027332

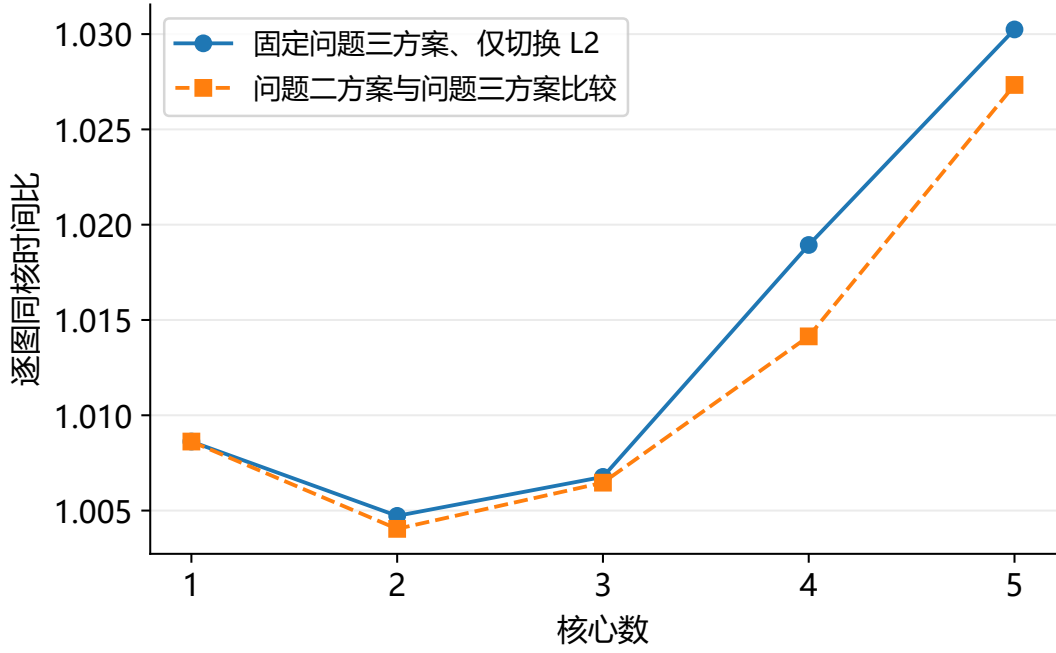


图 9 固定问题三方案的 L2 效应与跨方案综合比较

问题三五核总额外逻辑搬运为 1 150 596 866 字节。附录的同方案无 L2 与 L2 两次评价采用同一逻辑搬运口径，不从额外搬运中扣除命中字节。命中率并不单独决定周期：FIFO 淘汰、并发冷读、双带宽竞争与数据释放的改变均会影响关键路径。

7 模型检验与敏感性分析

本节三问横向表图中的问题一统一采用结构感知搜索 (r02)，以保持原三问实验的版本对应。多候选精修的完整核数与五核逐图比较另见问题一，本节跨问曲线仍以结构感知搜索为问题一参照。

7.1 逐配置账本与指标重算

以 $\mathcal{I} = \{1, \dots, 100\} \times \{1, \dots, 5\}$ 为逐问结果索引，每问必须恰有 500 份方案，既不能缺项，也不能用同一配置重复填数。合并本机和服务端结果后，依次核对输入与源码身份、方案文件哈希、官方结果哈希、节点覆盖及核上顺序，再从官方完整记录提取周期和额外搬运；问题三还核对最终同方案无 L2 和 L2 两次的逻辑搬运相等。任何缺项、非法方案或哈希冲突均中止表格生成。

题目要求的平均与辅助总周期比分别为

$$\bar{S}_k = \frac{1}{100} \sum_i \frac{T_{i,1}^0}{T_{i,k}}, \quad R_k^\Sigma = \frac{\sum_i T_{i,1}^0}{\sum_i T_{i,k}}. \quad (78)$$

前者给每张图等权，后者更受大周期图影响。计算百分比降幅时同样先说明参照：对单个配置为 $1 - T_{i,k}/T_{i,1}^0$ ，总周期降幅为 $1 - 1/R_k^\Sigma$ ；二者的聚合顺序不可互换。方案搜索以每配置周期最小为主，赛题规定的平均加速比是结果摘要，不是牺牲大图完成时间来抬高小图均值的理由。

7.2 五核分布与核数稳定性

按逐图加速比排序并作线性插值，报告给定百图的经验分位数。这些统计描述本次测试集，不是对未知图分布的置信区间。均值相同也可能对应完全不同的低分位表现，后续优化应分别检查低加速图与绝对周期占比大的图。本轮问题二有 9 个、问题三有 9 个相邻核数比较出现较多核反而更慢。它们说明当前有限候选池未保证核数间的嵌入保底，不证明物理

表 10 平均加速比和总周期口径对照

问题	五核平均加速比	五核总周期比	总周期下降率
1	3.665903	4.270565	76.5839%
2	4.264407	4.782199	79.0891%
3	4.364503	4.900925	79.5957%

表 11 五核逐图加速比的经验分布

问题	最小	四分位	中位数	四分之三位	最大
1	1.0017	2.4421	4.3104	4.9096	5.5011
2	1.6782	3.5791	4.7117	4.9690	5.8331
3	1.6897	3.7604	4.7947	5.0099	6.0115

最优值必然随核数增加。核心映射、子图边界与全局事件竞争共同变化，不能只由两个周期数字判断是哪一种机制导致退步。

7.3 成对缓存检验与负例

对每个最终问题三方案定义 $\delta_{i,k} = T_i^{L2}(p_{L,i,k}) - T_i^B(p_{L,i,k})$ 。 $\delta < 0$ 表示同方案启用 L2 更快， $\delta > 0$ 为反例；不能把它与问题三搜索相对另一个问题二方案的胜负混在一起。FIFO 命中

表 12 固定最终方案的缓存负例统计

核数	同方案启用 L2 后较慢的图数
1	0
2	2
3	3
4	1
5	1

和填充分别发生在发射及完成事件，带宽改变会使后续事件重排，因此容量或带宽变化没有一般的逐方案单调保证。保留的历史机制实验中，第 44 图整图单核方案在 L2 容量 0、64、128、256、512、1024、2048 KiB 下分别为 154407、154272、154407、154407、134726、107814、107814 周期。这是同一历史方案的事后容量扫描，不是本轮新算法在各容量下重新搜索的成绩；主实验参数始终固定为题面数值。

7.4 复用正确性、时间预算与复现范围

生成准备复用仅作用于不可变图上的确定性函数，以完整参数指纹识别重复状态，不省略新方案的全局评价。最终所选方案重新调用原版官方评价，并比较完整结果，而非只比较一个 **Makespan** 数字；物理缓存审计还检查整个执行过程中各实例的生命周期和容量。格式合法、事件评价成功与物理容量通过是不同检查，不能互相代替。

主机墙钟耗时由图解析、初解生成、候选构造、核内准备、全局事件仿真和最后复核组成。历史种子已经在磁盘上时，590 秒搜索含最终复核但不包含此前求种子的成本；图输入冷启动必须把前序阶段累计计入。两机分片运行记录主机和并发信息，禁止用整批耗时除以任务数冒充逐配置时间。具体冷启动压力结果见表 13。

全量覆盖证明给定输入、参数与官方语义下可复核，不等于全局最优证明。自适应候选权重使用实测耗时，相同种子在不同硬件上未必走相同轨迹；保存方案的独立重放才是对最终成绩的确定性复现。对缓存开关、候选顺序或搜索预算的独立归因，仍须保持相同初解和总时间，不能用增大计算机会后的成绩直接宣称某个组件有效。

8 模型评价与结论

8.1 模型特点

本文以同一计算图表示为基础，按三个场景逐步扩展状态与约束：场景 A 把每个子图视为独立 Task 并令跨图数据经 DDR；场景 B 引入同核复用和跨核同步；场景 C 进一步显式记录共享 L2 的 FIFO 状态与带宽竞争。这样的分层建模保留了问题之间可比较的划分、映射和顺序决策，又避免将某场景不存在的通信或缓存收益误用于另一场景。

求解采用结构化初解、关键路径优先调度与有界局部修复。连通分量、共享输入、双计算管道负载和缓存压力共同决定候选的结构；关键链等待与资源空隙决定局部调度；官方评估程序负责最终合法性和完成时间裁决。通过保留原方案、冻结预算及记录逐组结果，算法可区分预测收益与实际收益，适合在 100 图、1 至 5 核的固定配置上复现。

8.2 模型优点的边界化表述

第一，三个问题共用统一的算子、张量与方案表示，场景变化由 Task 构造和事件服务规则承担。这样可以复用覆盖、无环与核心顺序检查，而不把场景 A 的每子图 DDR 规则硬塞给 B，也不把 C 的 L2 命中率写成独立优化变量。统一不是“用一套固定成本系数解释三问”，而是对相同的决策 p 使用相应的正式评价器。式 (2) 把这种共性与差异分开，降低了跨问迁移时的概念错误。

第二，模型将静态结构、局部预测和官方结果分为三个证据层。静态结构可以证明一个商图是否无环、某一分量是否主导算子数，不能证明其周期；局部预测可以决定有限候选的优先次序，不能直接替代已验证方案；官方结果可以确定指定参数下的 Makespan 与额外搬运，却不能证明全局最优。这样划界，使得每一个正面结论都可追溯到足够强的证据。尤其对共享 DDR 和 FIFO，线性边权代理易失真，因此最后一层正式评价至关重要。

第三，方案的主要目标和次要目标遵循字典序。这比把搬运字节随意乘上一个大常数并加到周期里更符合题意，因为二者量纲不同，常数若取值不当会让一字节差异压倒真实周期改善。定义

$$(T_a, D_a) \prec (T_b, D_b) \iff [T_a < T_b] \vee [T_a = T_b \wedge D_a < D_b]. \quad (79)$$

由此，只要 T_a 比 T_b 少一周，即使 D_a 略高，仍优先 a ；两者周期完全相同才看搬运。由于官方周期是离散整数，这一判别可直接执行而不需要人为调节权重。在论文结果中某些方案的逻辑搬运上升而周期下降，正是式 (79) 主次关系的体现，而非模型自相矛盾。

第四，百图全量结果和逐例附录相互约束。平均加速比、总周期、逻辑额外搬运、Q3 同方案缓存效应分别从相同的五百行账本聚合；负例和相邻核数非单调也保留。这样的证据链允许读者从主文曲线一路追到一个算例、一种方案与一份官方评估记录。与只报告若干开发图相比，它更能检查候选规则是否在多种结构下稳定；但全量覆盖仍只覆盖赛题给定图集，不自动成为对未见图分布的统计保证。

8.3 模型缺点、已观察到的失效及可操作改进

成本预测仍是最主要的算法弱点。候选之间的共享 DDR 竞争不是独立子图时长的简单和：把一个 Task 提前可能改变所有同刻传输的服务份额，延后另一个更关键的读取；对问题三，命中改变的是事件的服务池，并可能改变完成顺序和后续 FIFO 队列。即便代理保留四条 Pipe 的局部依赖，如果冻结的背景状态与候选修改后的全局状态不同，排名仍可能错。问题一过去已有代理看好而官方接近四胜五负的审计现象，因此新邻域若仅依赖代理选择，会迅速耗尽有限预算而难以稳定提升。可操作的下一步是：固定同一完整候选池，逐对保存代理值与官方值，核对前二覆盖率、选择遗憾及误差最大的三类事件；仅对被证实误排的关键区域补充更精确的局部事件重放，最后仍用同预算正式评价验证结果。

模型的另一个缺点是缺少紧的、与划分无关的全局最优下界。式 (10) 严谨但忽略 DDR 与同步；条件带宽界和场景 A/B 路径界较具体，却随方案本身改变。若把它们直接拿去计算“当前距最优只差某百分比”，推理无效。未来可以尝试在聚合图上构造更强的资源割集下界，例如对每一种合法切分都必经的张量集合求最小必传量，再把它与算子链、Pipe

工作量耦合。但这样的下界必须对所有可行方案成立，且求解下界本身不应比原题穷举还难。在尚未证明前，本文只用同版本官方对照评价启发式质量。

有限候选预算是求解效率与搜索质量的有意取舍，也造成可见的失败：问题二、三都有相邻核数退步，问题三有四组 L2 相对无 L2 的负例；在大量配置上增加一个搜索算子只带来持平，说明该算子可能重复了现有候选或挤掉有效基础搜索。可以把已验证的少核方案作为多核保底，但需确认增加一个空闲核心不改变评价语义，并从原预算中为这项新保底安排机会。对有主导分量的图，可在现有整分量方案取得实测收益后，才开放一次结构不同的细切候选；对很多小分量的图，保留完整分量分配和两 Pipe 负载均衡，不必固定给每张图相同的拆分机会。这种按结构和实测改善调整机会的策略应在正式规则冻结后做全百图对照，不能根据已见结果事后给个别图额外试验次数再与旧版比较。

大图端到端墙钟时间仍未全量证明满足题目建议的五至十分钟。现有最长热搜索约 364 秒只是部分阶段；第 67 图等大图的单核整图核内基准本身可能耗时很长。若仅把搜索次数削到很少而牺牲周期质量，虽然墙钟变短，也不符合主目标；若只重跑相同候选或重复核内准备，则增加墙钟而未扩大信息量。工程上可优先缓存相同输入与硬件下的官方核内准备结果、对不受局部动作影响的子图复用 Pipe 序、对规范相同的完整方案只作一次正式评价，并把冷启动、搜索与最终复核分别计时。这些操作要通过独立官方重放检查缓存键正确，不能为了提速跳过结果核验。

8.4 可推广的求解思想与必须重新校准的部分

对其它具有多核、私有片上存储和共享带宽的计算设备，可迁移的是四步：先把工作负载表示为带共享张量的有向依赖图；再针对硬件通信与同步语义生成结构不同的合法划分；用执行资源和关键输出的事件信息排序有限候选；最后用可信评价器按主次目标选择方案。这一框架尤其适用于“图规模大而精确整数规划不可直接求解、官方仿真较昂贵”的离散调度问题。并非只有神经网络算子可以抽象成类似的图，工作流或数据流作业也可能共享这一方法学。

不能直接移植的是所有数值常数、FIFO 状态方程和具体切图阈值。例如目标设备若提供跨 Task 的私有缓存保持，场景 A 中同核边界必须经 DDR 的前提就消失；若有网络互联而非共享 DDR，通信的拥塞资源会从一个全局池变成多条链路；若 L2 为组相联 LRU，完成时的 FIFO 淘汰式也不适用。对新设备应重新建立 Task 事件生成规则、服务池状态和存储容量，验证评价器与实际设备的关系，再把本文的有界结构搜索嵌入新规则。保持逻辑框架，不等于保证原来在一百图上的 3.665903、4.264407 或 4.364503 继续成立。

8.5 主次目标、帕累托候选与可交付选择

字典序使最终选择没有权重任意性，但搜索阶段仍有理由保留少量非最优的结构候选。假设候选 p_a 的周期较短而搬运较多，另一个候选 p_b 的周期较长而搬运较少；根据式 (79) 最终只能选择 p_a ，但 p_b 可能是后续局部修复更好的出发点：它较少的 DDR 字节可为迁核或拆分腾出通信预算。保留候选是搜索状态的策略，不改变最终评分规则。若一次只保留当前字典序最优，所有局部动作都必须从 p_a 出发，可能因邻域狭窄而无法到达 p_b 附近的优质方案；若无限保留帕累托前沿，又会超过题目建议的求解时间。因此可仅保存结构差异显著的少数候选，并把它们的后续官方评价机会纳入总预算。

对两个方案定义弱支配： $p_a \preceq p_b$ 当且仅当 $T(p_a) \leq T(p_b)$ 且 $D(p_a) \leq D(p_b)$ ，并至少一个严格。若 p_b 被支配，它在当前目标和两个单项指标上均不优于 p_a ；但这仍不足以证明从 p_b 出发的所有后继都不优，因为局部结构相异可产生不同的可达邻域。若两方案互不支配，选择 T 更短者用于交付，同时仍可在固定小容量的工作池中保存另一者。这个区别防止把“保留多候选”误写成“改变赛题评分方式”。当前交付版的核心保证是已评价的最短周期方案输出；未来若引入有限帕累托工作池，必须比较相同总预算下的最终 T ，而不能把更多评估次数带来的改善算作方法优势。

历史分布诊断与当前搬运账本也提醒，对于不同场景的方案不能把 (T, D) 直接放入一个共同的帕累托前沿。场景 A 和 B 的 Task/COPY 构造不同，场景 C 虽和 B 使用相同的逻

辑 D 定义, 但读取走 L2 或 DDR 的物理服务不同; 若把 Q1 的 D 与 Q3 的 D 当作同一资源消耗严格比较, 会忽略它们对应的带宽池差异。字典序仅在同一场景、同一图、同一核数、同一固定参数下用于候选选择。跨场景的表格回答的是“各题规定的模型分别能达到什么数值”, 而非在同一硬件上公平选择 A/B/C 模式的工程结论。

8.6 复杂度与质量之间的可审查权衡

若图有 n 个可切算子、 q 个候选子图和 e 条投影依赖, 单次商图拓扑检验是 $O(q + e_\pi + e_\sigma)$; 对固定划分, 反向关键度递推也是同阶。一个朴素空隙插入会查看至多 $q + k$ 个位置, 每次检查受影响顺序与依赖; 若不复用局部状态, 最坏可能多次重建商图。与这些多项式阶段相比, 官方核内启发式可能在大图上插入大量换入换出, 而全局事件仿真随同时在途搬运的到达、完成次数增长。求解时间因此更接近

$$t_{\text{solve}} = t_{\text{parse}} + t_{\text{seed}} + \sum_{j=1}^{B_{\text{distinct}}} (t_{\text{legal}}^{(j)} + t_{\text{inner}}^{(j)} + t_{\text{event}}^{(j)}) + t_{\text{replay}}, \quad (80)$$

而不是仅用 $O(n + e)$ 概括。 B_{distinct} 是真正不同并需要正式评估的候选数, 名义槽位可能因重复而更大; 缓存可以减少重复 t_{inner} , 但不能省略最后一次独立回放。式 (80) 使五至十分钟的要求可以被分阶段审计: 若 t_{inner} 主导, 应优先优化核内准备复用; 若 t_{event} 主导, 可优化同等语义下的事件队列实现; 若 B_{distinct} 过多, 则应校准排序和去重。盲目降低所有阶段的精度可能牺牲合法性或主目标。

搜索预算 B 选得过低时, 结构候选尚未覆盖关键布局; 选得过高时, 后续提案的边际收益可能接近零而 CPU 成本继续增长。可定义第 b 次新增正式评价的实际增益 $g_b = T_{b-1}^{\text{best}} - T_b^{\text{best}} \geq 0$ 。由于 **incumbent** 只接受更好方案, g_b 非负, 但这并不保证 $g_{b+1} \leq g_b$: 一次晚出现的结构候选仍可能带来大幅改善。由此不能用“最近三次均持平”作严格的搜索收敛证明。合理的停止规则是在预定硬预算内, 结合过去一段候选的结构新颖性和实际收益, 决定剩余机会投向何种邻域; 若没有这些日志, 就应保守维持基础搜索份额。对于本题一百图, 所有这类自适应规则还须固定版本后全量对照, 而不能在看见最终分数后临时修改某张图的预算。

当算法在同一图上对多个核数重复求解时, 计算资源充足的本机或服务器可把不同 (i, k) 配置并行运行; 这不改变每配置的候选预算, 也不改变 NPU 方案周期。但是若多个进程同时争用主机内存、磁盘或官方评价器的缓存, 观察到的墙钟秒数会随部署方式变化。因此提交中的“单用例十分钟”应在明确的主机硬件、并发数和冷/热状态下报告; 把百图并行墙钟时长除以一百不能代表任何单个配置的求解时间。本文的全量成绩是方案执行周期的统计, 与运行批次的服务器调度方式分离。

8.7 结论

三问分别保存了 100 图、1 至 5 核的 500 份合法方案。五核逐图平均加速比分别为 3.665903、4.264407、4.364503; 问题三最终方案在同核下仅切换 L2 的平均时间比为 1.030244。目标是降低实际完成周期, 加速比用于按题目口径归纳结果; 额外搬运在周期相同时进一步优化。

8.8 冷启动与计时边界

另选 11 张预先指定的代表图、长依赖图及大图, 从原始图依次构造三问方案, 不读取历史初解。冷启动指新进程且不复用算法缓存, 不清空操作系统页缓存。下表把前序初解生成累计计入; 这些重新生成的初解与全量热启动证据不保证相同, 因此计时压力实验不替换正文成绩。尚未完成 100 图全部核数的图输入端到端冷启动核验, 不能声称十分钟目标全面满足。

8.9 问题一两方案比较的结论

问题一的两种方案使用相同物理约束与官方评价目标, 区别在有限搜索空间的构造。结构感知搜索保留每配置 12 次评价机会的资源约束, 多候选精修扩大结构候选并增加五核局

表 13 从原始图开始的冷启动压力核验

算例	问题	本阶段秒数	累计秒数	完整复核
1	1	2.07	2.07	通过
1	2	17.35	19.42	通过
1	3	16.44	35.86	通过
14	1	2400.12	2400.12	未完成
48	1	4.47	4.47	通过
48	2	180.12	184.60	通过
48	3	164.27	348.87	通过
67	1	89.34	89.34	通过
67	2	476.21	565.56	通过
67	3	477.41	1042.98	通过
72	1	1728.07	1728.07	通过
72	2	513.94	2242.05	通过
72	3	474.27	2716.37	通过
76	1	1992.21	1992.21	通过
76	2	506.79	2499.06	通过
76	3	473.86	2972.97	通过
3	1	55.20	55.20	通过
3	2	478.31	533.66	通过
3	3	475.28	1008.95	通过
18	1	8.67	8.67	通过
18	2	97.50	106.17	通过
18	3	74.50	180.67	通过
33	1	13.75	13.75	通过
33	2	138.12	151.89	通过
33	3	126.28	278.19	通过
64	1	1.47	1.47	通过
64	2	30.08	31.56	通过
64	3	24.14	55.72	通过
71	1	1.31	1.31	通过
71	2	34.48	35.81	通过
71	3	29.36	65.17	通过

部搜索；最新五核交付组合平均加速比为 4.026824。其相对结构感知搜索的 66 胜、21 平、13 负表明均值改善不意味着逐图支配。评价次数、历史搜索时间和组合来源不同，限制了因果解释；低核配置已补齐；等预算消融及全流程冷启动计时仍需补充。这些限制不影响已保存方案的周期与搬运核对，但应限制对算法普遍优势的表述。

参考文献

- [1] 中国研究生数学建模竞赛组委会. 通用神经网络处理器下的多核调度问题 [Z]. 2026.
- [2] Popp M, Schlag S, Schulz C, Seemaier D. Multilevel Acyclic Hypergraph Partitioning[EB/OL]. arXiv:2002.02962, 2020.
- [3] Topcuoglu H, Hariri S, Wu M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260–274. DOI: 10.1109/71.993206.
- [4] Liu Y, Xue Y, Cheng Y, et al. Scaling Deep Learning Computation over the Inter-core Connected Intelligence Processor with T10[C]//Proceedings of the 30th Symposium on Operating Systems Principles. 2024. DOI: 10.1145/3694715.3695955.
- [5] Belady L A, Nelson R A, Shedler G S. An anomaly in space-time characteristics of certain programs running in a paging machine[J]. Communications of the ACM, 1969, 12(6): 349–353. DOI: 10.1145/363011.363155.

A 程序与复现说明

本附录列出支撑材料文件并直接引用当前工作区源程序，保证论文附录与实际计算代码一致。

A.1 附录目录

附录 1: 附录目录	
附录 2: 核心代码	主程序与各子问题求解程序。
附录 3: 支撑材料	数据、校验和工具文件。
附录 4: 复现说明	运行命令、依赖环境与哈希。

A.2 支撑材料文件列表

1. `q1_experimental.py`: 模型计算源程序，683 行，下文展示核心实现。
2. `q1_submit.py`: 模型计算源程序，84 行，下文展示核心实现。
3. `q23_portfolio_reuse.py`: 模型计算源程序，75 行，下文展示核心实现。
4. `q23_selective_reuse.py`: 模型计算源程序，75 行，下文展示核心实现。
5. `q23_submit_current.py`: 模型计算源程序，24 行，下文展示核心实现。
6. `q2_timed_portfolio.py`: 模型计算源程序，187 行，下文展示核心实现。
7. `q3_timed_combination.py`: 模型计算源程序，101 行，下文展示核心实现。

A.3 复现入口（英文别名）

```
python q1_submit.py GRAPH -n K --output DIR --verify-final
```

A.4 核心实现 1: `q1_experimental.py`

程序 1 `q1_experimental.py`

```
1  """可消融的文献启发优化。官方评价不变；关闭特性用于共同搜索框架对照。"""
2  from collections import defaultdict
3  import copy
4  import json
5  import math
6  import random
7  import time
8
9  from q1_solver import Graph, greedy_partition, multilevel, mutate,
    validate, topological
10
11  FEATURES = frozenset({'local_cost', 'critical', 'adaptive', 'portfolio'})
12  EXTRA_FEATURES = frozenset({'insertion', 'comm_rank', 'lookahead', '
    calibrated', 'joint', 'budget_adapt', 'ddr', 'guarded_joint', 'beam', '
    partition_guard', 'shared_input', 'local_repair', 'repair_move', '
    region', 'region_gap', 'fluid_rank', 'boundary_refine', 'exact_region',
    'wide_region', 'uphill_region', 'phase_rank', 'event_rank', '
    chain_joint', 'late_chain', 'resource_init', 'init_components', '
    init_batches', 'init_depth', 'component_guard', 'component_slot', '
    component_followup', 'component_local_rank', 'memory_route', '
    hybrid_route', 'fast_contractions', 'bounded_routes', 'opportunity_gate
    ', 'event_opportunity', 'event_seed_rank', 'preparation_reuse'})
13
14
15  class CostGraph(Graph):
16      def __init__(self, raw, settings, waits, enabled=False):
```

```

17     super().__init__(raw, settings, waits)
18     self.enabled = enabled
19     self.local_observations = {}
20     self.observation_conflicts = 0
21     self.cost_hits = 0
22     self.insertion = False
23     self.beam = False
24     self.placement_stats = []
25     self.guided_repair_stats = []
26     self.region_stats = []
27     self.boundary_stats = []
28     self.chain_stats = []
29     self.communication_rank = False
30     self.calibrated = False
31     self.learn_calibration = False
32     self.ratios = defaultdict(list)
33     self.fast_costs = False
34
35     def schedule(self, mapping, cores):
36         if self.beam:
37             from q1_beam import schedule
38             plan, score, info = schedule(self, mapping, cores)
39             self.placement_stats.append(info)
40             return plan, score
41         if self.insertion or self.communication_rank:
42             from q1_insertion import schedule
43             return schedule(self, mapping, cores, self.insertion, self.
communication_rank)
44         return super().schedule(mapping, cores)
45
46     def observe(self, plan, result):
47         groups = defaultdict(list)
48         for u, s in plan['node_to_subgraph'].items():
49             groups[s].append(int(u))
50         mapping = {int(u): s for u, s in plan['node_to_subgraph'].items()}
51         base = super().costs(mapping)[0] if self.calibrated or self.
learn_calibration else {}
52         for s, profile in result['step3_by_task'].items():
53             key = tuple(sorted(groups[int(s)]))
54             value = profile['local_makespan']
55             if base and key not in self.local_observations and base[int(s)
] > 0:
56                 loads = defaultdict(int)
57                 for u in key:
58                     loads[self.ops[u]['pipe']] += self.ops[u]['cycles']
59                 pipe = max.loads, key=lambda p: (loads[p], p))
60                 self.ratios[pipe].append(max(.5, min(4., value/base[int(s)
])))
61             if key in self.local_observations and self.local_observations[
key] != value:
62                 self.observation_conflicts += 1
63                 # Per-Graph cache: graph, hardware and official evaluator

```

```

        remain fixed.
64         # This is a local estimate, never a cached concurrent task
        duration.
65         self.local_observations[key] = value
66
67     def costs(self, mapping):
68         if self.fast_costs:
69             from q1_fast_costs import costs
70             duration, traffic, view = costs(self, mapping)
71         else:
72             duration, traffic, view = super().costs(mapping)
73         if not self.enabled:
74             return duration, traffic, view
75         groups = view[0]
76         longest = {}
77         for u in self.order:
78             longest[u] = self.ops[u]['cycles'] + max(
79                 (longest[p] for p in self.pred[u] if mapping[p] == mapping
80                  [u]), default=0)
81         for s, nodes in groups.items():
82             key = tuple(sorted(nodes))
83             if key in self.local_observations:
84                 duration[s] = self.local_observations[key]
85                 self.cost_hits += 1
86             else:
87                 if self.calibrated:
88                     from statistics import median
89                     loads = defaultdict(int)
90                     for u in nodes:
91                         loads[self.ops[u]['pipe']] += self.ops[u]['cycles']
92
93                 pipe = max(loads, key=lambda p: (loads[p], p))
94                 if self.ratios[pipe]:
95                     duration[s] *= median(self.ratios[pipe][-64:])
96                 duration[s] = max(duration[s], max((longest[u] for u in
97                  nodes), default=0))
98         return duration, traffic, view
99
100     def task_slacks(g, plan, result):
101         """冻结已观察的 Task 持续时间，反推数据/同核联合图的余量。
102
103         DDR 竞争在方案改变后会变化，所以此余量只用于候选排序。
104         """
105         mapping = {int(u): s for u, s in plan['node_to_subgraph'].items()}
106         groups, pred, succ, _ = g.view(mapping)
107         pred = {s: set(v) for s, v in pred.items()}
108         succ = {s: set(v) for s, v in succ.items()}
109         owner = {s: c for c, seq in enumerate(plan['core_schedules']) for s in
110                  seq}
111         delays = {(a, b): g.cross if owner[a] != owner[b] else 0
112                   for a in groups for b in succ[a]}

```

```

110     for seq in plan['core_schedules']:
111         for a, b in zip(seq, seq[1:]):
112             pred[b].add(a)
113             succ[a].add(b)
114             delays[a, b] = max(g.same, delays.get((a, b), 0))
115     tasks = {t['task_id']: t for c in result['per_core_timeline'] for t in
116              c['tasks']}
117     latest = {}
118     for s in reversed(topological(pred, succ)):
119         latest_end = min((latest[v] - delays[s, v] for v in succ[s]),
120                          default=result['makespan'])
121         latest[s] = latest_end - tasks[s]['duration']
122     return {s: max(0, latest[s] - tasks[s]['start']) for s in groups}
123
124 def adaptive_partition(g, cores, scale=1.0, max_nodes=256):
125     """局部工作量/数据驻留代理驱动分组；不拆分或改写原始算子。"""
126     total = defaultdict(int)
127     for o in g.ops.values():
128         total[o['pipe']] += o['cycles']
129     target = max(g.cross, max(total.values(), default=0) * scale / max(1,
130                                cores * 8))
131     # First obtain a communication-aware topological sequence using the
132     # old constructor.
133     base = greedy_partition(g, cores, max_nodes)
134     rank = {u: -base[u] for u in g.ops}
135     order = topological(g.pred, g.succ, rank)
136     mapping, block, count = {}, 0, 0
137     loads, live, remaining = defaultdict(int), set(), {}
138     resident = defaultdict(int)
139     for u in order:
140         if count >= max_nodes or (count >= 4 and (
141             max(loads.values(), default=0) >= target or
142             any(resident[p] > cap for p, cap in g.capacity.items()))):
143             block += 1
144             count = 0
145             loads, live, remaining = defaultdict(int), set(), {}
146             resident = defaultdict(int)
147         mapping[u] = block
148         count += 1
149         loads[g.ops[u]['pipe']] += g.ops[u]['cycles']
150         for i in g.incident[u]:
151             size, pos, ps, cs, final = g.tensors[i]
152             if i not in live:
153                 live.add(i)
154                 resident[pos] += size
155                 remaining[i] = set(cs)
156                 remaining[i].discard(u)
157             # End-of-Task outputs stay resident in this conservative
158             # grouping proxy.
159             if not remaining[i] and not final:
160                 live.remove(i)

```

```

157         resident[pos] -= size
158     return mapping
159
160
161 def directed_candidates(g, plan, result, cores, critical=False, adaptive=
False, preserve=False):
162     mapping = {int(u): s for u, s in plan['node_to_subgraph'].items()}
163     groups, _, succ, order = g.view(mapping)
164     tasks = [t for c in result['per_core_timeline'] for t in c['tasks']]
165     slack = task_slacks(g, plan, result) if critical else {}
166     tasks.sort(key=lambda t: (slack.get(t['task_id'], 0), -t['duration'],
-t['end'], t['task_id']))
167     for task in tasks[:4]:
168         s = task['task_id']
169         nodes = sorted(groups[s], key=g.pos.get)
170         if len(nodes) > 1:
171             weights = [g.ops[u]['cycles'] for u in nodes]
172             total, prefix = sum(weights), 0
173             choices = []
174             for i, w in enumerate(weights[:-1], 1):
175                 prefix += w
176                 choices.append((abs(total / 2 - prefix), i))
177             cut = min(choices)[1]
178             cuts = [cut]
179             if adaptive:
180                 # Favor low-traffic boundaries near a balanced work cut.
181                 def crossing(i):
182                     left, right = set(nodes[:i]), set(nodes[i:])
183                     tids = {t for u in nodes for t in g.incident[u]}
184                     return sum(g.tensors[t][0] for t in tids
185                               if g.tensors[t][2] & left and g.tensors[t
186                               ][3] & right)
187                 nearby = range(max(1, cut - 3), min(len(nodes), cut + 4))
188                 cuts = sorted(set([cut, min(nearby, key=lambda i: (
crossing(i), abs(i-cut), i))]))
189                 for i in cuts:
190                     trial = dict(mapping)
191                     for u in nodes[i:]:
192                         trial[u] = max(groups) + 1
193                     try:
194                         from q1_partition import repair
195                         from q1_guided_repair import guided_repair
196                         yield 'split', (guided_repair(g, plan, trial, cores)
197                         if preserve == 'mobile' else
198                         repair(g, plan, trial) if preserve
199                         else g.schedule(trial, cores)[0])
200                     except ValueError:
201                         continue
202                 if adaptive:
203                     # Local merge alternatives retain the rest of the graph's
granularity.
204                     for b in sorted(succ[s])[:2]:

```

```

202         trial = {u: s if v == b else v for u, v in mapping.items()}
203     }
204     try:
205         from q1_partition import repair
206         from q1_guided_repair import guided_repair
207         yield 'merge', (guided_repair(g, plan, trial, cores)
208         if preserve == 'mobile' else
209         repair(g, plan, trial) if preserve
210         else g.schedule(trial, cores)[0])
211     except ValueError:
212         continue
213     positions = {s: i for i, s in enumerate(order)}
214     finish = [max((t['end'] for t in c['tasks']), default=0) for c in
215     result['per_core_timeline']]
216     for task in tasks[:3]:
217         s = task['task_id']
218         source = next(i for i, seq in enumerate(plan['core_schedules']) if
219         s in seq)
220         for target in sorted(range(cores), key=lambda i: (finish[i], i)):
221             if target == source:
222                 continue
223             trial = copy.deepcopy(plan)
224             trial['core_schedules'][source].remove(s)
225             trial['core_schedules'][target].append(s)
226             trial['core_schedules'][target].sort(key=positions.get)
227             yield 'migrate', trial
228
229 def elite_update(entries, entry, limit=4):
230     """非支配时间/搬运候选 + 最优时间兜底；只控制搜索存档，不作最优证明。"""
231     pool = entries + [entry]
232     pool.sort(key=lambda e: (e[0], json.dumps(e[1], sort_keys=True)))
233     chosen = [pool[0]]
234     for e in pool[1:]:
235         if any(x[1] == e[1] for x in chosen):
236             continue
237         if not any(x[0][0] <= e[0][0] and x[0][1] <= e[0][1] and x[0] != e
238         [0] for x in pool):
239             chosen.append(e)
240         if len(chosen) == limit:
241             return chosen
242         # Preserve structurally different runner-up solutions when the
243         # frontier is small.
244         for e in pool:
245             if len(chosen) == limit:
246                 break
247             if not any(x[1]['node_to_subgraph'] == e[1]['node_to_subgraph']
248             for x in chosen):
249                 chosen.append(e)
250     return chosen

```

```

246 def solve_experimental(raw, settings, waits, cores=4, evaluation_budget
    =12, seed=0,
247                        features=(), on_evaluation=None, cache_dir=None,
    single_reference=None,
248                        evaluator_backend='official', on_proposal=None,
    initial_plan=None, on_incumbent=None):
249     from multicore_cut_evaluate_problem_1 import evaluate_scene_a
250     if evaluator_backend == 'counter':
251         from q1_fast_evaluator import evaluate_scene_a, BACKEND_ID
252         # Keep derived-backend evidence separate from the original cache.
253         if cache_dir is not None:
254             from pathlib import Path
255             cache_dir = Path(cache_dir) / BACKEND_ID.replace(':', '-')
256     elif evaluator_backend != 'official':
257         raise ValueError('Unknown evaluator backend')
258     features = frozenset(features)
259     if 'late_chain' in features and 'chain_joint' not in features:
260         raise ValueError('late_chain requires chain_joint')
261     if 'chain_joint' in features and 'event_rank' not in features:
262         raise ValueError('chain_joint requires event_rank')
263     if 'event_rank' in features and features & {'phase_rank', 'wide_region',
    'uphill_region'}:
264         raise ValueError('event_rank must be an isolated ablation')
265     if features & {'wide_region', 'uphill_region', 'phase_rank', '
    event_rank'} and 'exact_region' not in features:
266         raise ValueError('wide/uphill region require exact_region')
267     if 'exact_region' in features and ('region' not in features or
    features & {'fluid_rank', 'boundary_refine', 'region_gap'}):
268         raise ValueError('exact_region requires region and isolated
    ranking')
269     if 'guarded_joint' in features and features & {'calibrated', 'joint',
    'budget_adapt', 'ddr', 'portfolio', 'adaptive', 'lookahead'}:
270         raise ValueError('guarded_joint must preserve the base search
    prefix; incompatible feature combination')
271     if features - (FEATURES | EXTRA_FEATURES) or evaluation_budget < 1 or
    cores < 1:
272         raise ValueError('未知特性或无效预算/核数')
273     if features & {'shared_input', 'local_repair', 'repair_move', 'region'
    } and 'partition_guard' not in features:
274         raise ValueError('shared_input/local_repair/repair_move require
    partition_guard')
275     if features & {'fluid_rank', 'boundary_refine'} and ('region' not in
    features or 'region_gap' in features):
276         raise ValueError('fluid_rank/boundary_refine require region and
    exclude region_gap for isolated ablation')
277     if 'region_gap' in features and 'region' not in features:
278         raise ValueError('region_gap requires region')
279     if 'region' in features and ('local_cost' not in features or features
    & {'local_repair', 'repair_move'}):
280         raise ValueError('region requires local_cost and excludes other
    repair modes')
281     if {'local_repair', 'repair_move'} <= features:

```

```

282         raise ValueError('Choose local_repair or repair_move, not both')
283     if 'partition_guard' in features and (evaluation_budget < 8 or
features & {'guarded_joint', 'budget_adapt', 'portfolio', 'adaptive', '
joint', 'ddr', 'lookahead', 'beam', 'calibrated'})):
284         raise ValueError('partition_guard requires budget >= 8 and base-
only features')
285     structural = features & {'init_components', 'init_batches', '
init_depth'}
286     if structural and ('partition_guard' not in features or
evaluation_budget < 12 or 'resource_init' in features):
287         raise ValueError('Structural seeds require partition_guard, budget
>= 12 and exclude resource_init')
288     if features & {'component_guard', 'component_slot'} and (structural !=
{'init_components'} or 'component_guard' not in features):
289         raise ValueError('Component routing requires only init_components
and component_guard')
290     if features & {'event_seed_rank', 'preparation_reuse'} and '
event_opportunity' not in features:
291         raise ValueError('event ranking/reuse require event opportunity')
292     if 'event_opportunity' in features and ('bounded_routes' not in
features or 'opportunity_gate' in features):
293         raise ValueError('event_opportunity requires bounded_routes and
excludes local gate')
294     if 'opportunity_gate' in features and 'bounded_routes' not in features
:
295         raise ValueError('opportunity_gate requires bounded_routes')
296     if 'bounded_routes' in features and ('component_local_rank' not in
features or features & {'memory_route', 'hybrid_route'}):
297         raise ValueError('bounded_routes requires local rank and excludes
separate routes')
298     if features & {'memory_route', 'hybrid_route'} and ('
component_local_rank' not in features or {'memory_route', 'hybrid_route'
} <= features):
299         raise ValueError('Choose one route with component_local_rank')
300     if 'component_local_rank' in features and 'component_followup' not in
features:
301         raise ValueError('component_local_rank requires component_followup
')
302     if 'component_followup' in features and 'component_slot' not in
features:
303         raise ValueError('component_followup requires component_slot')
304     from q1_search_tools import EvaluationCache, replay, diagnose,
ranked_joint, choose_arm
305     g = CostGraph(raw, settings, waits, bool({'local_cost', 'calibrated'}
& features))
306     g.fast_costs = evaluator_backend == 'counter'
307     g.fast_contractions = 'fast_contractions' in features
308     g.insertion = 'insertion' in features
309     g.beam = 'beam' in features
310     g.communication_rank = 'comm_rank' in features
311     g.calibrated = 'calibrated' in features
312     g.learn_calibration = 'guarded_joint' in features

```

```

313     shared_ranker = None
314     if 'preparation_reuse' in features:
315         from q1_exact_placement import PartitionEvaluator
316         from q1_local_rank import LocalRank
317         from q1_fast_evaluator import evaluator as counter_engine
318         shared = PartitionEvaluator(raw, g.bandwidth, g.capacity, g.cross,
319                                   g.same,
320                                   backend_engine=counter_engine() if
321                                   evaluator_backend=='counter' else None,
322                                   cache_on_evaluate=False)
323         shared_ranker = LocalRank(raw,g,event=True,preparer=shared,
324                                   reuse_scores=True)
325         persistent = EvaluationCache(cache_dir, raw, settings, waits) if
326         cache_dir else None
327         started = time.perf_counter()
328         rng = random.Random(seed)
329         cache, history, rejected, archive = {}, [], [], []
330         proposal_ledger = []
331         protected_grain_ledger = []
332         best = None
333
334     def evaluate(plan, label, observe_improving_only=False):
335         nonlocal best, archive
336         if on_proposal is not None:
337             on_proposal(copy.deepcopy(plan), label)
338         key = json.dumps(plan, sort_keys=True)
339         import hashlib
340         record = {'candidate': label, 'plan_sha256': hashlib.sha256(key.
341 encode()).hexdigest()}
342         proposal_ledger.append(record)
343         if key in cache:
344             record.update(status='duplicate', evaluation_id=cache[key])
345             return None
346         if len(history) >= evaluation_budget:
347             record['status'] = 'budget_exhausted'
348             return None
349         try:
350             validate(g, plan)
351         except (ValueError, RuntimeError) as exc:
352             rejected.append({'candidate': label, 'reason': str(exc)})
353             record.update(status='infeasible', reason=str(exc))
354             return None
355         t = time.perf_counter()
356         mapping = {int(u): s for u, s in plan['node_to_subgraph'].items()}
357         predicted = replay(g, plan, g.costs(mapping)[0])
358         def compute():
359             if shared_ranker is not None:
360                 return shared_ranker.preparer.evaluate(plan)
361             return evaluate_scene_a(raw, plan, g.bandwidth, g.capacity, g.
362 cross, g.same)
363         if label == 'single_task' and single_reference is not None:
364             from q1_single_reference import reuse

```

```

359         result = reuse(single_reference, raw, settings, waits, cores)
360         cache_hit = True
361     else:
362         result, cache_hit = persistent.run(plan, compute) if
persistent else (compute(), False)
363         diagnostic = diagnose(g, plan, result, predicted)
364         score = (result['makespan'], result['data_movement_bytes']['
added_copy_bytes'])
365         entry = (score, plan, result)
366         improved = best is None or score < best[0]
367         if improved:
368             best = entry
369         if 'portfolio' in features:
370             archive = elite_update(archive, entry)
371         else:
372             archive = [best]
373         if g.enabled and (not observe_improving_only or improved):
374             g.observe(plan, result)
375         if observe_improving_only:
376             record['cost_observation_applied'] = bool(g.enabled and
improved)
377             cache[key] = len(history)
378             record.update(status='evaluated', evaluation_id=len(history),
cache_hit=cache_hit)
379             history.append({'candidate': label, 'makespan': score[0], '
added_copy_bytes': score[1],
380                             'evaluation_seconds': time.perf_counter() - t,
381                             'cache_hit': cache_hit, 'fixed_single_reference':
label == 'single_task' and single_reference is not None,
382                             'timing_diagnostic': diagnostic,
383                             'best_makespan': best[0][0]})
384         if on_evaluation:
385             on_evaluation(history[-1])
386         if improved and on_incumbent:
387             on_incumbent(plan, result)
388         return entry
389
390     fallback = {'node_to_subgraph': {str(u): 0 for u in sorted(g.ops)},
391                 'core_schedules': [[0] if g.ops else []] + [[] for _ in
range(cores-1)]]
392     evaluate(fallback if initial_plan is None else initial_plan,
393             'single_task' if initial_plan is None else 'provided_initial'
)
394     # A multicore incumbent is not the prescribed whole-graph single-core
reference.
395     baseline = best[0][0] if initial_plan is None else None
396     reward_scale = max(1, best[0][0])
397     if cores > 1 and len(history) < evaluation_budget:
398         if 'resource_init' in features:
399             from q1_resource_init import resource_partition
400             initial = resource_partition(g, cores)
401     else:

```

```

402         initial = greedy_partition(g, cores)
403         evaluate(g.schedule(initial, cores)[0], 'resource_initial' if '
resource_init' in features else 'greedy')
404         if len(history) < evaluation_budget:
405             coarse = multilevel(g, initial, cores)
406             if 'shared_input' in features:
407                 from q1_partition import shared_coarsen
408                 coarse = shared_coarsen(g, coarse, cores)
409             evaluate(g.schedule(coarse, cores)[0], 'multilevel')
410     structural_info = {}
411     pending_component = []
412     followup_unlocked = False
413     if structural and cores > 1:
414         from q1_structural_seeds import initial_candidates, canonical_key
415         if 'component_guard' in features:
416             from q1_structural_seeds import guarded_component_candidate
417             if features & {'memory_route', 'hybrid_route', 'bounded_routes'
}:
418                 from q1_memory_routes import routed_candidates
419                 route = 'auto' if 'bounded_routes' in features else ('
memory' if 'memory_route' in features else 'hybrid')
420                 options = {}
421                 if 'event_seed_rank' in features or shared_ranker is not
None:
422                     options = dict(ranking='event' if 'event_seed_rank' in
features else 'local', ranker=shared_ranker)
423                 proposals, structural_info = routed_candidates(raw,
settings, waits, cores, route, **options)
424                 elif 'component_local_rank' in features:
425                     proposals, structural_info = guarded_component_candidate(
raw, settings, waits, cores, max_candidates=2, ranking='local')
426                 elif 'component_followup' in features:
427                     proposals, structural_info = guarded_component_candidate(
raw, settings, waits, cores, max_candidates=2)
428                 else:
429                     proposals, structural_info = guarded_component_candidate(
raw, settings, waits, cores)
430                 else:
431                     proposals, structural_info = initial_candidates(raw, settings,
waits, cores,
432                         {feature.removeprefix('init_') for feature in structural})
433                 # At most four new official opportunities. Four grain attempts and
at
434                 # Least one normal search opportunity survive, in addition to old
init.
435                 limit = max(0, min(4, evaluation_budget - len(history) - 5))
436                 if 'component_guard' in features:
437                     limit = min(limit, 2 if 'component_followup' in features else
1)
438                 structural_info.update(limit=limit, evaluated=0, ledger=[])
439                 if 'component_slot' in features:
440                     pending_component = proposals

```

```

441         proposals = []
442         known = {canonical_key(json.loads(key)): index for key, index in
cache.items()}
443         for kind, plan in proposals:
444             record = {'candidate': 'structural_' + kind}
445             structural_info['ledger'].append(record)
446             key = canonical_key(plan)
447             if key in known:
448                 record.update(status='equivalent_duplicate', evaluation_id
=known[key])
449                 continue
450             if structural_info['evaluated'] >= limit:
451                 record['status'] = 'seed_limit'
452                 continue
453             if 'component_guard' in features and structural_info['
selected_compute_lower_bound'] > best[0][0]:
454                 record['status'] = 'compute_bound_exceeds_incumbent'
455                 continue
456             entry = evaluate(plan, record['candidate'],
observe_improving_only='component_guard' in features)
457             record.update(proposal_ledger[-1])
458             if entry:
459                 structural_info['evaluated'] += 1
460                 known[key] = len(history) - 1
461             arms = ['grain', 'directed', 'random'] + (['local_reschedule'] if g.
enabled else [])
462             if 'budget_adapt' in features:
463                 arms = ['grain', 'directed', 'split', 'boundary', 'migrate', '
reorder', 'local_reschedule']
464             if 'joint' in features:
465                 arms.append('joint')
466             if 'ddr' in features:
467                 arms.append('ddr')
468             queues = {}
469             pulls, rewards = defaultdict(int), defaultdict(float)
470             grain_index, directed, directed_key = 0, iter(()), None
471             refreshes = 0
472             protected_grain_attempts = []
473             guarded_start = max(3, math.ceil(.75 * evaluation_budget))
474             guarded_attempts = 0
475             region_attempts = 0
476             late_region_start = None
477             protected_score = None
478             for attempt in range(evaluation_budget * 80 if cores > 1 else 0):
479                 if len(history) >= evaluation_budget:
480                     break
481                 if 'budget_adapt' in features:
482                     arm = choose_arm(arms, pulls, rewards, attempt)
483                 elif 'portfolio' in features:
484                     arm = next((a for a in arms if not pulls[a]), None)
485                 if arm is None:
486                     arm = max(arms, key=lambda a: (rewards[a] / pulls[a] +

```

```

487         math.sqrt(2 * math.log(1 + sum(pulls.values()))
/ pulls[a]), -arms.index(a)))
488     else:
489         arm = arms[attempt % len(arms)]
490         if ('partition_guard' in features and grain_index < 4
491             and evaluation_budget - len(history) <= 4 - grain_index +
int('late_chain' in features and region_attempts == 0)):
492             # Reserve only the remaining opportunities; retain normal
ordering until needed.
493             arm = 'grain'
494             if 'late_chain' in features and region_attempts == 0 and
grain_index >= 4 and len(history) == evaluation_budget-1:
495                 arm = 'random'
496             if 'guarded_joint' in features and len(history) >= guarded_start:
497                 if protected_score is None:
498                     protected_score = best[0][0]
499                     g.calibrated = True
500                     if guarded_attempts % 4 == 0:
501                         arm = 'joint'
502                         guarded_attempts += 1
503                 pulls[arm] += 1
504                 before = best[0][0]
505                 parent = archive[rng.randrange(len(archive))] if 'portfolio' in
features and arm == 'random' else best
506                 label = arm
507                 grain_record = None
508                 try:
509                     if arm == 'grain':
510                         scales = [.5, 1., 2., .25, 4.]
511                         scale = scales[grain_index % len(scales)]
512                         grain_index += 1
513                         if 'partition_guard' in features and grain_index <= 4:
514                             protected_grain_attempts.append(scale)
515                             grain_record = {'scale': scale, 'status': '
constructing'}
516                             protected_grain_ledger.append(grain_record)
517                             mapping = (adaptive_partition(g, cores, scale) if '
adaptive' in features else
518                                     greedy_partition(g, cores, max(1, int(32 *
scale))))
519                             trial = g.schedule(mapping, cores)[0]
520                             label = f'grain_{scale}'
521                     elif arm == 'directed':
522                         current_key = json.dumps(best[1], sort_keys=True)
523                         if directed_key is None or ('critical' in features and
directed_key != current_key):
524                             if 'lookahead' in features:
525                                 from itertools import chain
526                                 from q1_lookahead import candidates
527                                 directed = chain(candidates(g, best[1], best[2],
cores),
528                                                 directed_candidates(g, best[1],

```

```

best[2], cores,
529                                     'critical' in
    features, 'adaptive' in features, ('mobile' if 'repair_move' in
features else 'local_repair' in features)))
530         else:
531             directed = directed_candidates(g, best[1], best
[2], cores,
532                                     'critical' in
    features, 'adaptive' in features, ('mobile' if 'repair_move' in
features else 'local_repair' in features))
533             directed_key = current_key
534             refreshes += 1
535         try:
536             kind, trial = next(directed)
537         except StopIteration:
538             directed_key = None
539             continue
540         label += '_' + kind
541         elif (arm == 'random' and 'region' in features and
region_attempts < 1
542             and (pulls['random'] >= 2 or 'late_chain' in features)
543             and ('late_chain' not in features or len(history) ==
evaluation_budget-1)):
544             if 'late_chain' in features:
545                 late_region_start = {'evaluation_index':len(history),
'grain_attempts':list(protected_grain_attempts), 'incumbent_time':best
[0][0]}
546                 from q1_region import candidates as region_candidates
547                 region_attempts += 1
548                 current_key = json.dumps(best[1], sort_keys=True)
549                 if 'region' not in queues or queues['region'][0] !=
current_key:
550                     if 'exact_region' in features:
551                         from q1_ranked_region import ranked_candidates
552                         queue = ranked_candidates(g, raw, best[1], best
[2], cores, 'wide_region' in features, 'uphill_region' in features, '
phase_rank' in features, 'event_rank' in features, 'chain_joint' in
features)
553                     elif features & {'fluid_rank', 'boundary_refine'}:
554                         from q1_boundary_refine import ranked_candidates
555                         queue = ranked_candidates(g, best[1], best[2],
cores, 'fluid_rank' in features, 'boundary_refine' in features)
556                     else:
557                         queue = region_candidates(g, best[1], best[2],
cores, 'region_gap' in features)
558                     queues['region'] = (current_key, queue)
559                 try:
560                     kind, trial = next(queues['region'][1])
561                 except StopIteration:
562                     continue
563                 label = 'region_' + kind
564                 elif arm == 'local_reschedule':

```

```

565         trial = g.schedule({int(u): s for u, s in best[1]['
node_to_subgraph'].items()} , cores)[0]
566         elif arm in {'joint', 'ddr'}:
567             current_key = json.dumps(best[1], sort_keys=True)
568             if arm not in queues or queues[arm][0] != current_key:
569                 queues[arm] = (current_key, ranked_joint(g, best[1],
best[2], cores, arm == 'ddr'))
570             try:
571                 kind, trial = next(queues[arm][1])
572             except StopIteration:
573                 continue
574             label += '_' + kind
575         else:
576             trial = parent[1]
577             kind = arm if arm in {'split', 'boundary', 'migrate', '
reorder'} else rng.choice(['merge', 'split', 'boundary', 'migrate', '
reorder'])
578             for _ in range(rng.randint(1, 3)):
579                 trial = mutate(g, trial, cores, rng, kind, preserve=('
mobile' if 'repair_move' in features else 'local_repair' in features))
580                 label += '_' + kind
581             except (ValueError, RuntimeError) as exc:
582                 rejected.append({'candidate': label, 'reason': str(exc)})
583                 if grain_record is not None:
584                     grain_record.update(status='infeasible', reason=str(exc))
585             continue
586             # Evaluator errors propagate. Only construction/legality failures
are rejected above.
587             entry = None
588             # Construct the original random proposal first, preserving RNG
draws.
589             # A structural probe replaces one ordinary random opportunity, not
a
590             # directed/grain/local-reschedule/region opportunity.
591             ordinary_random = arm == 'random' and not label.startswith('
region_')
592             duplicate_opportunity = False
593             if pending_component and followup_unlocked:
594                 known = {canonical_key(json.loads(key)): index for key, index
in cache.items()}
595                 duplicate_opportunity = canonical_key(trial) in known
596             if pending_component and (ordinary_random or duplicate_opportunity
):
597                 kind, component_plan = pending_component.pop(0)
598                 import hashlib
599                 record = {'candidate': 'structural_' + kind, '
replaces_candidate': label,
600                           'replaced_plan_sha256': hashlib.sha256(json.dumps(
trial, sort_keys=True).encode()).hexdigest(),
601                           'replaced_exact_evaluation_id': cache.get(json.dumps
(trial, sort_keys=True)),
602                           'replacement_at_evaluation': len(history),

```

```

603         'incumbent_plan_sha256': hashlib.sha256(json.dumps(
best[1], sort_keys=True).encode()).hexdigest()}
604     if 'component_followup' in features:
605         record.update(probe_index=structural_info['evaluated'] +
1,
606                       opportunity='equivalent_duplicate' if
duplicate_opportunity else 'ordinary_random',
607                       incumbent_makespan=best[0][0])
608     if duplicate_opportunity:
609         record['replaced_equivalent_evaluation_id'] = known[
canonical_key(trial)]
610         structural_info['ledger'].append(record)
611         known = {canonical_key(json.loads(key)) for key in cache}
612         if canonical_key(component_plan) in known:
613             record['status'] = 'equivalent_duplicate'
614         elif structural_info.get('candidate_compute_bounds', {}).get(
kind, structural_info['selected_compute_lower_bound']) > best[0][0]:
615             record['status'] = 'compute_bound_exceeds_incumbent'
616         else:
617             allowed = True
618             if 'opportunity_gate' in features and structural_info.get(
'route'):
619                 from q1_opportunity import compare_local_profiles
620                 prediction = next(x['local_prediction'] for x in
structural_info['ranked'] if x['candidate'] == kind)
621                 allowed, decision = compare_local_profiles(g,
prediction, best[1], best[2])
622                 record['opportunity_comparison'] = decision
623                 if not allowed:
624                     record['status'] = 'screened_no_predicted_gain'
625                 if 'event_opportunity' in features and structural_info.get(
('route')):
626                     from q1_opportunity import compare_event_profiles
627                     allowed, decision = (compare_event_profiles(raw, g,
component_plan, best[1], shared_ranker)
628                                         if shared_ranker is not None else
compare_event_profiles(raw, g, component_plan, best[1]))
629                     record['event_comparison'] = decision
630                     if not allowed:
631                         record['status'] = 'screened_no_event_gain'
632                     if allowed:
633                         entry = evaluate(component_plan, record['candidate'],
observe_improving_only=True)
634                         record.update(proposal_ledger[-1])
635                         if entry:
636                             structural_info['evaluated'] += 1
637                         if 'component_followup' in features and not followup_unlocked:
638                             # Strict primary-objective improvement, not merely fewer
bytes.
639                             followup_unlocked = entry is not None and entry[0][0] <
record['incumbent_makespan']
640                             structural_info['followup_unlocked'] = followup_unlocked

```

```

641         record['unlocked_followup'] = followup_unlocked
642         if not followup_unlocked:
643             structural_info['followup_skip_reason'] = '
first_probe_did_not_improve_makespan'
644             pending_component.clear()
645         if entry is None:
646             entry = evaluate(trial, label)
647         if grain_record is not None:
648             grain_record.update(proposal_ledger[-1])
649         if entry:
650             # Count-based reward keeps seeded runs reproducible across
machine speeds.
651             rewards[arm] += max(0., before - entry[0][0]) / reward_scale
652         if 'component_followup' in features and pending_component:
653             structural_info['followup_skip_reason'] = '
no_unprotected_opportunity_before_budget_end'
654             stats = {'method': 'experimental', 'features': sorted(features), 'seed
': seed,
655                     'evaluator_backend': evaluator_backend,
656                     'official_calls_semantics': 'full global scoring calls,
original or equivalent derived backend',
657                     'evaluation_budget': evaluation_budget, 'evaluations':
history,
658                     'official_calls': sum(not r['cache_hit'] for r in history),
659                     'cache_hits': sum(r['cache_hit'] for r in history),
660                     'budget_unit': 'unique evaluated candidates including cache
hits; hits do not buy extra search',
661                     'shared_input_stats': getattr(g, 'shared_input_stats', {}),
662                     'preparation_reuse_stats': shared_ranker.stats() if
shared_ranker else None,
663                     'resource_init_stats': getattr(g, 'resource_init_stats', {}),
664                     'structural_seed_stats': structural_info,
665                     'protected_grain_attempts': protected_grain_attempts,
666                     'protected_grain_ledger': protected_grain_ledger,
667                     'proposal_ledger': proposal_ledger,
668                     'protected_prefix_evaluations': guarded_start if '
guarded_joint' in features else 0,
669                     'protected_prefix_makespan': protected_score,
670                     'budget_exhausted': len(history) == evaluation_budget,
671                     'singlecore_makespan': baseline,
672                     'speedup': (baseline / best[0][0] if best[0][0] else 1.) if
baseline is not None else None,
673                     'rejected_proposals': rejected, 'local_cost_entries': len(g.
local_observations),
674                     'local_cost_hits': g.cost_hits, 'local_cost_conflicts': g.
observation_conflicts,
675                     'placement_stats': g.placement_stats,
676                     'guided_repair_stats': g.guided_repair_stats,
677                     'boundary_stats': g.boundary_stats,
678                     'chain_stats': g.chain_stats,
679                     'late_region_start': late_region_start,
680                     'region_stats': g.region_stats, 'region_attempts':

```

```

        region_attempts,
681         'critical_refreshes': refreshes, 'arm_pulls': dict(pulls), '
        arm_rewards': dict(rewards),
682         'elite_scores': [e[0] for e in archive], 'elapsed_seconds':
        time.perf_counter() - started}
683     return best[1], best[2], stats

```

A.5 核心实现 2: q1_submit.py

程序 2 q1_submit.py

```

1  """Portable scene-A baseline entry; standard library only, explicit graph
   input."""
2  import argparse
3  import gzip
4  import json
5  import platform
6  import time
7  from pathlib import Path
8  from q1_io import PROCESSED, official, verify, sha, write_json
9
10 from q1_ablation import CONFIGS
11
12 ADOPTED_POLICY = 'routes_gate_reuse'
13 BASELINE_FEATURES = tuple(CONFIGS[ADOPTED_POLICY])
14
15
16 def main():
17     entry_started = time.perf_counter()
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('graph', type=Path)
20     parser.add_argument('-n', '--cores', type=int, choices=range(1, 6),
21                         required=True)
22     parser.add_argument('--config', type=Path, default=PROCESSED / 'data/
23                         config.txt')
24     parser.add_argument('--output', type=Path, required=True, help='New
25                         output directory')
26     parser.add_argument('--backend', choices=['official', 'counter'],
27                         default='counter')
28     parser.add_argument('--budget', type=int, default=12)
29     parser.add_argument('--seed', type=int, default=0)
30     parser.add_argument('--verify-final', action='store_true',
31                         help='Additional original official replay, outside
32                         search budget; time is recorded')
33     parser.add_argument('--final-check-backend', choices=['official', '
34                         counter'], default='official',
35                         help='Experimental counter check is an accelerated
36                         replay, not an independent original official replay')
37     args = parser.parse_args()
38     if args.cores > 1 and args.budget < 8:
39         parser.error('Protected baseline partition budget requires --
40         budget >= 8')
41     verify()

```

```

34     mod = official()
35     from evaluation_validation import read_evaluation_config,
validate_graph
36     from q1_experimental import solve_experimental
37     raw = json.loads(args.graph.read_text(encoding='utf-8-sig'))
38     validate_graph(raw)
39     settings = read_evaluation_config(str(args.config))
40     waits = mod.read_scene_a_config(str(args.config))
41     args.output.mkdir(parents=True, exist_ok=False)
42     def phase(name):
43         write_json(args.output / 'phase.json', {'phase': name,
44             'entry_elapsed_seconds': time.perf_counter() - entry_started,
45             'final_check_backend': args.final_check_backend})
46         print('PHASE', name, flush=True)
47     phase('search_started')
48     started = time.perf_counter()
49     if args.cores == 1:
50         from singlecore_evaluate import build_singlecore_plan
51         plan = build_singlecore_plan(raw)
52         evaluator = mod.evaluate_scene_a
53         if args.backend == 'counter':
54             from q1_fast_evaluator import evaluate_scene_a as evaluator
55             result = evaluator(raw, plan, settings['bandwidth'], settings['
capacity'],
56                                     waits['task_cross_core_wait_cycles'], waits['
task_same_core_wait_cycles'])
57             stats = {'evaluations': [], 'evaluation_count': 1,
58                     'singlecore_makespan': result['makespan'], 'mode': '
fixed_whole_graph'}
59         else:
60             def progress(row):
61                 print(row['candidate'], row['makespan'], flush=True)
62             plan, result, stats = solve_experimental(raw, settings, waits,
args.cores,
63                 args.budget, args.seed, BASELINE_FEATURES, on_evaluation=
progress,
64                 evaluator_backend=args.backend)
65             solve_seconds = time.perf_counter() - started
66             phase('search_finished')
67             verification_seconds = None
68             if args.verify_final:
69                 phase('final_check_started')
70                 check_start = time.perf_counter()
71                 checker = mod.evaluate_scene_a
72                 if args.final_check_backend == 'counter':
73                     from q1_fast_evaluator import evaluate_scene_a as checker
74                     checked = checker(raw, plan, settings['bandwidth'], settings['
capacity'],
75                                     waits['task_cross_core_wait_cycles'], waits['
task_same_core_wait_cycles'])
76                     if checked != result:
77                         raise RuntimeError('Full final result differs from ' + args.

```

```

final_check_backend + ' replay')
78     verification_seconds = time.perf_counter() - check_start
79     phase('final_check_finished')
80     write_json(args.output / (args.graph.stem + '_multicore_res.json'),
plan)
81     write_json(args.output / 'search.json', stats)
82     (args.output / 'evaluation.json.gz').write_bytes(gzip.compress(
83         json.dumps(result, ensure_ascii=False, separators=(',', ':')).
encode(), mtime=0))
84     meta = {'algorithm': ADOPTED_POLICY, 'features': sorted(
BASELINE_FEATURES),
85         'seed': args.seed, 'budget': args.budget, 'cores': args.cores,
86         'backend': args.backend, 'python': platform.python_version(),
87         'input_sha256': sha(args.graph.read_bytes()), 'config_sha256':
sha(args.config.read_bytes()),
88         'solve_seconds': solve_seconds, 'verification_seconds':
verification_seconds,
89         'final_check_backend': args.final_check_backend if args.
verify_final else None,
90         'original_official_replayed': bool(args.verify_final and args.
final_check_backend == 'official'),
91         'total_seconds': time.perf_counter() - started,
92         'makespan': result['makespan'], 'added_copy_bytes': result['
data_movement_bytes']['added_copy_bytes'],
93         'singlecore_makespan': stats['singlecore_makespan'],
94         'speedup': stats['singlecore_makespan'] / result['makespan'],
95         'source_sha256': {p.name: sha(p.read_bytes()) for p in sorted(
Path(__file__).parent.glob('*.py'))}}
96     write_json(args.output / 'run.json', meta)
97     phase('complete')
98     print(json.dumps({k: meta[k] for k in ('makespan', 'speedup', '
solve_seconds')}, ensure_ascii=False))
99
100
101 if __name__ == '__main__':
102     main()

```

A.6 核心实现 3: q23_portfolio_reuse.py

程序 3 q23_portfolio_reuse.py

```

1  """Instrument frozen full portfolios; generation reuse is the only arm
switch.
2
3  Search receives 80% of the wall budget; remaining time is reserved for
original
4  replay and physical audit in BOTH arms. Original adaptive time policies
remain.
5  """
6  import argparse, contextlib, gzip, json, time, traceback
7  from pathlib import Path
8  from unittest.mock import patch
9  from q1_io import sha, write_json

```

```

10 from q2_timed_portfolio import atomic, objective
11 from q2_evaluator import key
12 from q23_cache_search import read, run as supervised_run
13
14 def worker(args, deadline):
15     out=Path(args['output']); inner=out/'search'; inner.mkdir(); records=[];
16     generation=None
17     started=time.monotonic(); q=args['problem']
18     try:
19         import q2_solver, q2_timed_portfolio, q3_timed_combination, q3_solver
20         from q23_selective_reuse import GenerationReuse
21         from q2_evaluator import evaluate as q2_evaluate
22         from q2_physical import PhysicalScorer
23         settings, delay, cache, provenance=q3_solver.load(); raw=read(args['
graph'])
24         score=q2_evaluate if q==2 else q3_solver.evaluate
25         def observed(*a, **kw):
26             t=time.monotonic(); h=key(a[1])
27             r=score(*a, **kw)
28             records.append(dict(plan_sha256=h, result_sha256=sha(json.dumps
(r, sort_keys=True, separators=(',', ':')).encode()), makespan=r['makespan'
], added=objective(r)[1], seconds=time.monotonic()-t))
29             atomic(out/'scored.json', records); return r
30         source_args=dict(args, output=str(inner), arm='protected' if q==2
else 'combined')
31         search_end=deadline-.2*args['seconds']
32         source_graph=q2_solver.SceneBGraph
33         with contextlib.ExitStack() as stack:
34             def graph_factory(*a, **kw):
35                 nonlocal generation
36                 g=source_graph(*a, **kw)
37                 if args['reuse'] and generation is None:
38                     generation=stack.enter_context(GenerationReuse(g))
39                 return g
40             stack.enter_context(patch.object(q2_solver, 'SceneBGraph',
graph_factory))
41             if q==2:
42                 stack.enter_context(patch.object(q2_timed_portfolio, '
evaluate', observed))
43                 q2_timed_portfolio.worker(source_args, search_end)
44             else:
45                 stack.enter_context(patch.object(q3_solver, 'evaluate',
observed))
46                 q3_timed_combination.worker(source_args, search_end)
47                 search_seconds=time.monotonic()-started
48                 if (inner/'error.txt').exists(): raise RuntimeError((inner/'error.
txt').read_text(encoding='utf-8'))
49                 pointer=read(inner/'verified.json'); plan=read(inner/pointer['plan'
])
50                 with gzip.open(inner/pointer['evaluation'], 'rt', encoding='utf-8')
as f: expected=json.load(f)
51                 t=time.monotonic()

```

```

51         if q==2:
52             result=q2_evaluate(raw,plan,settings,delay);assert json.loads(
53             json.dumps(result))==expected
54             PhysicalScorer(raw,settings,delay).actual_lifetimes(plan,
55             result)
56         else:
57             result=q3_solver.evaluate(raw,plan,settings,delay,cache);
58             assert json.loads(json.dumps(result))==expected
59             q3_solver.audit(raw,plan,result,settings,delay)
60             final_seconds=time.monotonic()-t
61             write_json(out/'final.plan.json',plan)
62             with gzip.open(out/'final.evaluation.json.gz','wt',encoding='utf-8
63             ') as f:json.dump(result,f)
64             atomic(out/'verified.json',dict(plan='final.plan.json',evaluation=
65             'final.evaluation.json.gz',makespan=result['makespan'],added=objective(
66             result)[1],plan_sha256=key(plan),official_original=True))
67             atomic(out/'details.json',dict(search_seconds=search_seconds,
68             final_verify_seconds=final_seconds,scored=records,generation_stats=dict
69             (generation.stats) if generation else {},complete=True,
70             input_sha256=sha(Path(args['graph']).read_bytes()),
71             provenance=provenance,policy='Unmodified full protected/combined,
72             including measured-time adaptive weights; sequence divergence is
73             allowed'))
74         except Exception:(out/'error.txt').write_text(traceback.format_exc(),
75             encoding='utf-8')
76
77 def run(args):
78     import q23_cache_search
79     with patch.object(q23_cache_search,'worker',worker):return
80     supervised_run(args)
81
82 if __name__=='__main__':
83     p=argparse.ArgumentParser();p.add_argument('--problem',type=int,
84     required=True,choices=[2,3]);p.add_argument('--graph',required=True);p.
85     add_argument('--output',required=True)
86     p.add_argument('--migration');p.add_argument('--seed-plan');p.
87     add_argument('--anchor');p.add_argument('--reuse',action='store_true')
88     p.add_argument('--seconds',type=float,default=590);p.add_argument('--
89     cores',type=int,default=5);p.add_argument('--seed',type=int,default=0);
90     p.add_argument('--max-proposals',type=int,default=96)
91     a=vars(p.parse_args());a['cache_mib']=32
92     if a['problem']==3 and not a['seed_plan']:p.error('Q3 needs --seed-
93     plan')
94     print(json.dumps(run(a)))

```

A.7 核心实现 4: q23_selective_reuse.py

程序 4 q23_selective_reuse.py

```

1  """Experimental exact reuse; graph must remain immutable within the
2  context.
3  Memoizing deterministic inner operations preserves all outer RNG

```

```

        consumption.
4 No whole proposal or global simulation is memoized.
5 """
6 import hashlib,pickle,time
7 from collections import Counter,OrderedDict
8 from q23_preparation_reuse import PreparationReuse
9
10 class GenerationReuse:
11     def __init__(self,graph,max_bytes=32*1024**2):
12         import q2_solver
13         self.module=q2_solver;self.graph=graph;self.limit=max_bytes
14         self.entries=OrderedDict();self.size=0;self.stats=Counter();self.
original={}
15     def wrap(self,name,fn):
16         def call(g,*args,**kwargs):
17             if g is not self.graph:return fn(g,*args,**kwargs)
18             t=time.perf_counter();k=(name,hashlib.sha256(pickle.dumps((
args,kwags),protocol=5)).digest())
19             self.stats[name+'_calls']+=1
20             if k in self.entries:
21                 self.stats[name+'_hits']+=1;b=self.entries.pop(k);self.
entries[k]=b
22                 value=pickle.loads(b)
23             else:
24                 self.stats[name+'_misses']+=1;value=fn(g,*args,**kwargs);b
=pickle.dumps(value,protocol=5)
25                 if len(b)<=self.limit//4:
26                     while self.entries and self.size+len(b)>self.limit:
27                         _,old=self.entries.popitem(last=False);self.size-=
len(old);self.stats['evictions']+=1
28                         self.entries[k]=b;self.size+=len(b)
29                 else:self.stats['oversize']+=1
30                 self.stats[name+'_seconds']+=time.perf_counter()-t
31                 self.stats['peak_payload_bytes']=max(self.stats['
peak_payload_bytes'],self.size)
32                 return value
33             return call
34     def __enter__(self):
35         for n in ['context','order_plan','proxy']:
36             self.original[n]=getattr(self.module,n);setattr(self.module,n,
self.wrap(n,self.original[n]))
37         return self
38     def __exit__(self,*exc):
39         for n,fn in self.original.items():setattr(self.module,n,fn)
40         self.entries.clear();self.size=0
41
42 class SelectivePreparationReuse(PreparationReuse):
43     """Separate stage budgets, second-use admission, oversized-entry
bypass.
44
45     A stage with no hit in 128 probes stops caching for this invocation.
46     Hashes include complete ordered inputs and settings. Snapshot hits are

```

```

isolated.
47 """
48 def wrap(self,name,fn):
49     entries=OrderedDict();history=OrderedDict();size=0;calls=hits=0;
disabled=False
50     limit=self.limit//3
51     def call(*args,**kwargs):
52         nonlocal size,calls,hits,disabled
53         self.stats['calls']+=1;self.stats[name+'_calls']+=1
54         if disabled:
55             self.stats['bypassed']+=1;return fn(*args,**kwargs)
56         calls+=1;k=hashlib.sha256(pickle.dumps((args,kwargs),protocol
=5)).digest()
57         if k in entries:
58             hits+=1;self.stats['hits']+=1;self.stats[name+'_hits']+=1
59             b=entries.pop(k);entries[k]=b;return pickle.loads(b)
60         self.stats['misses']+=1;value=fn(*args,**kwargs)
61         if k in history:
62             history.move_to_end(k);b=pickle.dumps(value,protocol=5)
63             if len(b)<=limit//4:
64                 while entries and size+len(b)>limit:
65                     _,old=entries.popitem(last=False);size-=len(old);
self.stats['evictions']+=1
66                     entries[k]=b;size+=len(b)
67                     self.stats[name+'_peak_payload_bytes']=max(self.stats[
name+'_peak_payload_bytes'],size)
68                 else:self.stats['oversize']+=1
69             else:
70                 history[k]=None;self.stats['first_use_bypass']+=1
71                 if len(history)>4096:history.popitem(last=False)
72                 if calls>=128 and hits==0:
73                     disabled=True;entries.clear();history.clear();size=0;self.
stats[name+'_disabled']=1
74                 return value
75     return call

```

A.8 核心实现 5: q23_submit_current.py

程序 5 q23_submit_current.py

```

1 """Explicit warm-start entry for the adopted Q2/Q3 combinations; Appendix
B output."""
2 import argparse,json,time
3 from pathlib import Path
4 from q1_io import write_json,official
5 from q23_portfolio_reuse import run
6
7 def main():
8     entry_started=time.monotonic()
9     p=argparse.ArgumentParser(description=__doc__);p.add_argument('graph',
type=Path);p.add_argument('--problem',type=int,choices=[2,3],required=
True);p.add_argument('-n','--cores',type=int,choices=range(2,6),
required=True);p.add_argument('--output',type=Path,required=True);p.

```

```

    add_argument('--migration',type=Path);p.add_argument('--seed-plan',type
    =Path);p.add_argument('--anchor',type=Path);p.add_argument('--seconds',
    type=float,default=590);p.add_argument('--max-proposals',type=int,
    default=384);a=p.parse_args()
10 seed=a.migration if a.problem==2 else a.seed_plan
11 if seed is None:p.error('Q2 requires --migration; Q3 requires --seed-plan
    . Historical seed generation is excluded; no implicit case lookup.')
12 official();from stub_multicore_cut_and_schedule import
    validate_multicore_plan
13 raw=json.loads(a.graph.read_text(encoding='utf-8-sig'))
14 for f in [seed,a.anchor] if a.problem==3 else [seed]:
15     if f:
16         plan=json.loads(f.read_text(encoding='utf-8-sig'));
            validate_multicore_plan(raw,plan)
17         if len(plan['core_schedules'])!=a.cores:p.error('Initial plan core
            count differs from requested cores')
18     args={k:str(v) if isinstance(v,Path) else v for k,v in vars(a).items()};
            args.update(seed=0,reuse=True,cache_mib=32);t=time.monotonic();s=run(
            args)
19     if not(s['valid'] and s['complete'] and not s['error'] and not s['
            deadline_stop'] and s['worker_exitcode']==0):raise RuntimeError('No
            completely verified final solution; inspect summary.json')
20     plan=json.loads((a.output/s['plan']).read_text(encoding='utf-8'));
            validate_multicore_plan(raw,plan)
21     target=a.output/(a.graph.stem+'_multicore_res.json');write_json(target,
            plan)
22     write_json(a.output/'delivery.json',dict(solution=target.name,
            elapsed_including_export=time.monotonic()-entry_started,solver_summary=
            'summary.json',seed_generation_included=False,algorithm='protected' if
            a.problem==2 else 'combined',generation_reuse=True))
23     print(target)
24 if __name__=='__main__':main()

```

A.9 核心实现 6: q2_timed_portfolio.py

程序 6 q2_timed_portfolio.py

```

1  """Bounded scene-B portfolio. Every accepted result is an ORIGINAL
    official replay.
2
3  The parent owns the end-to-end deadline. Optional migration is a declared
    warm
4  input, never advertised as cold seed generation. Fixed T1 is an external
    metric.
5  No Q1 task-duration cache is valid for scene B's whole-core preparation.
6  """
7  import argparse, copy, gzip, json, math, multiprocessing as mp
8  import os, random, time, traceback
9  from pathlib import Path
10 from collections import defaultdict
11 from q1_io import PROCESSED, write_json, sha
12 from q2_evaluator import load, evaluate, key
13

```

```

14
15 def atomic(path, value):
16     path = Path(path); tmp = path.with_suffix(path.suffix + '.tmp')
17     write_json(tmp, value); os.replace(tmp, path)
18
19
20 def objective(result):
21     return result['makespan'], result['data_movement_bytes']['
    added_copy_bytes']
22
23
24 def family_weights(m):
25     """Feature-based priors only, no case IDs or saved answers."""
26     fraction = max(map(len, m.components)) / max(1, len(m.ids))
27     return dict(remap=2 if fraction > .5 else 1,
28                 granularity=2 if fraction > .5 else 1,
29                 order=1, joint=1, insert=1)
30
31
32 def generate(family, m, g, incumbent, n, rng, step):
33     from q2_solver import place, order_plan, owner_of, context, proposal,
    DEFAULTS
34     from q2_physical import candidate_pool
35     mapping = {int(u): s for u, s in incumbent['node_to_subgraph'].items()
    }
36     if family == 'packing':
37         # Only partition/placement construction is inherited from A;
    accepted
38         # costs always include B's whole-core scheduling and tensor
    lifetimes.
39         from q1_timed_candidates import packing
40         return packing(g, m.components, n, rng, step)
41     if family == 'frontier':
42         from q1_timed_candidates import frontier_mapping
43         mapping = frontier_mapping(g, n, rng, step)
44         owners = place(g, mapping, n, True, (.25, 1., 4.)[step%3])
45         return order_plan(g, mapping, owners, n, True, weight=.5)
46     if family == 'remap':
47         # Log-spaced trade-offs between parallel work and marginal tensor
    traffic.
48         weight = (0., .25, .5, 1., 2., 4.)[step % 6]
49         owners = place(g, mapping, n, True, weight)
50         return order_plan(g, mapping, owners, n, step % 2 == 0, weight=.5)
51     if family == 'granularity':
52         # Relative topological windows scale with graph size and core
    count.
53         count = n * (2, 4, 8, 16, 32)[step % 5]
54         width = max(1, math.ceil(len(m.ids) / count))
55         mapping = {u: i // width for i, u in enumerate(g.order)}
56         owners = place(g, mapping, n, True, (.25, 1., 4.)[(step // 5) %
    3])
57     return order_plan(g, mapping, owners, n, True, weight=.5)

```

```

58     if family == 'order':
59         groups, _, _, _, _, ranks = context(g, mapping)
60         priority = {s: ranks[s] * rng.uniform(.5, 1.5) for s in groups}
61         return order_plan(g, mapping, owner_of(incumbent), n, step % 2 ==
0,
62                             weight=(.1, .5, 1., 2.)[step % 4],
random_priority=priority)
63     if family == 'insert':
64         pool = candidate_pool(g, incumbent, rng, 'move', limit=4)
65         return pool[step % len(pool)][plan] if pool else incumbent
66     return proposal(g, incumbent, n, rng, 'CLJ', DEFAULTS, joint=step % 2
== 0)[0]
67
68
69 def worker(args, deadline):
70     folder = Path(args['output']); rows = []; best = result = None; seen =
set()
71     started = time.perf_counter(); rng = random.Random(args['seed'])
72     counters = defaultdict(int); gain = defaultdict(float); spent =
defaultdict(float)
73     def assess(label, plan):
74         nonlocal best, result
75         h = key(plan); t = time.perf_counter()
76         if h in seen:
77             rows.append(dict(name=label, plan_sha256=h, status='duplicate'
, seconds=0))
78             return 0.
79         seen.add(h)
80         try:
81             r = evaluate(raw, plan, settings, delay)
82             before = result['makespan'] if result else None
83             accepted = result is None or objective(r) < objective(result)
84             improvement = max(0., (before-r['makespan'])/before) if before
else 0.
85             if accepted:
86                 # Immutable payloads first, atomic pointer last: timeout
cannot
87                 # expose a new plan with an old official evaluation.
88                 name = 'accepted_%03d' % len(rows)
89                 write_json(folder/(name+'.plan.json'), plan)
90                 with gzip.open(folder/(name+'.evaluation.json.gz'), 'wt',
encoding='utf-8') as f:
91                     json.dump(r, f)
92                     atomic(folder/'verified.json', dict(plan=name+'.plan.json'
,
93                     evaluation=name+'.evaluation.json.gz', makespan=r['
makespan'],
94                     added_copy_bytes=objective(r)[1], plan_sha256=h,
official_original=True))
95                     best, result = copy.deepcopy(plan), r
96                     rows.append(dict(name=label, plan_sha256=h, status='ok',
accepted=accepted,

```

```

97         makespan=r['makespan'], added_copy_bytes=objective(r)[1],
seconds=time.perf_counter()-t))
98     except (ValueError, RuntimeError) as e:
99         improvement = 0.; rows.append(dict(name=label, plan_sha256=h,
100             status='invalid', error=str(e), seconds=time.perf_counter
101             (-t))
102         atomic(folder/'search.json', dict(evaluations=rows, elapsed=time.
perf_counter()-started,
103             counts=dict(counters), gain=dict(gain), spent=dict(spent)))
104         return improvement
105     try:
106         settings, delay, provenance = load(args.get('config'))
107         raw = json.loads(Path(args['graph']).read_text(encoding='utf-8'))
108         from q1_structural_seeds import GraphModel, _component_plan
109         from q2_solver import SceneBGraph
110         from q2_reserve_shared import menu
111         from q2_bottleneck_j import pool
112         m = GraphModel(raw, settings, dict(task_cross_core_wait_cycles=
delay, task_same_core_wait_cycles=0))
113         g = SceneBGraph(raw, settings, delay); n = args['cores']
114         migration = (json.loads(Path(args['migration']).read_text(encoding
='utf-8'))
115             if args.get('migration') else _component_plan(m, n, '
pipe', True))
116         atomic(folder/'identity.json', dict(input_sha256=sha(Path(args['
graph']).read_bytes()),
117             provenance=provenance, seed_mode='warm_Q1' if args.get('
migration') else 'cold_components',
118             migration_sha256=sha(Path(args['migration']).read_bytes()) if
args.get('migration') else None,
119             weights=family_weights(m), arguments=args))
120         assess('migration_B' if args.get('migration') else '
components_bootstrap', migration)
121         whole=dict(node_to_subgraph={str(u):0 for u in g.ops},
core_schedules=[[0]]+[[ ] for _ in range(n-1)])
122         seeds=[('whole',whole)]; metadata={}
123         # Both arms keep the adopted seed menu and ordinary repair
opportunities.
124         _, more = menu(raw, settings, delay, n, migration, metadata);
seeds += more
125         for label, plan in seeds:
126             if time.monotonic() >= deadline: return
127             assess(label, plan)
128         if best is None: raise RuntimeError('No verified fallback')
129         for item in pool(g, best, None, budget=4):
130             if time.monotonic() >= deadline: return
131             assess('ordinary_J', item['plan'])
132         if args['arm'] == 'baseline': return
133         weights = family_weights(m)
134         if args['arm']=='protected':
135             weights['frontier']=1
136             if len(m.components)>1: weights['packing']=2

```

```

136     families = list(weights)
137     adaptive_started=time.monotonic()
138     protected_until=adaptive_started+.4*max(0,deadline-
adaptive_started)
139     # Finite proposal cap, not a combinatorial enumeration. Initial
round
140     # explores every family; subsequent draws retain minimum
exploration.
141     for iteration in range(args['max_proposals']):
142         remaining = deadline-time.monotonic()
143         recent = [r['seconds'] for r in rows if r['status']=='ok'
][-4:]
144         if remaining < max(.5, 1.3*max(recent, default=.1)): break
145         protected=args['arm']=='protected' and iteration<32 and time.
monotonic()<protected_until
146         family = ('joint' if args['arm']=='ordinary_extended' or
protected else
147             families[iteration] if iteration<len(families) else rng.
choices(
148                 families, [weights[f]*(1+min(8.,100*gain[f]/max(.1,spent[f
]))) for f in families])[0])
149         step=counters[family]; counters[family]+=1; t=time.
perf_counter()
150         try:
151             candidate=generate(family,m,g,best,n,rng,step)
152             gain[family]+=assess(family,candidate)
153         except (ValueError,RuntimeError) as e:
154             rows.append(dict(name=family,status='generation_invalid',
error=str(e),seconds=time.perf_counter()-t))
155             spent[family]+=time.perf_counter()-t
156             atomic(folder/'search.json',dict(evaluations=rows,counts=dict(
counters),gain=dict(gain),spent=dict(spent)))
157         except Exception:
158             (folder/'error.txt').write_text(traceback.format_exc(),encoding='
utf-8')
159
160
161 def run(graph, output, cores=5, seconds=590, arm='portfolio', seed=0,
migration=None, max_proposals=96):
162     if not 2 <= cores <= 5 or not 0 < seconds <= 590: raise ValueError('
cores 2..5, seconds (0,590]')
163     if arm not in ('baseline','portfolio','ordinary_extended','protected')
: raise ValueError(arm)
164     started=time.monotonic(); folder=Path(output); folder.mkdir(parents=
True,exist_ok=False)
165     args=dict(graph=str(Path(graph).resolve()),output=str(folder.resolve()
),cores=cores,
166         seconds=seconds,arm=arm,seed=seed,migration=str(Path(
migration).resolve()) if migration else None,
167         max_proposals=max_proposals)
168     ctx=mp.get_context('spawn'); process=ctx.Process(target=worker,args=(
args,started+seconds))

```

```

169     process.start(); process.join(max(0,seconds-(time.monotonic()-started)
170 ))
171     stopped=process.is_alive()
172     if stopped: process.terminate(); process.join(2)
173     if process.is_alive(): process.kill(); process.join(2)
174     pointer=folder/'verified.json'
175     summary=dict(arguments=args,seconds=time.monotonic()-started,
176     deadline_stop=stopped,
177         status='ok' if pointer.exists() else '
178 no_verified_solution',
179         worker_error=(folder/'error.txt').exists(),
180 worker_exitcode=process.exitcode,
181         source_sha256={p.name:sha(p.read_bytes()) for p in Path(
182 __file__).parent.glob('q*.py')})
183     if pointer.exists(): summary.update(json.loads(pointer.read_text(
184 encoding='utf-8')))
185     atomic(folder/'summary.json',summary); return summary
186
187 if __name__=='__main__':
188     p=argparse.ArgumentParser();p.add_argument('graph');p.add_argument('--
189 output',required=True)
190     p.add_argument('--cores',type=int,default=5);p.add_argument('--seconds
191 ',type=float,default=590)
192     p.add_argument('--arm',choices=['baseline','portfolio','
193 ordinary_extended','protected'],default='portfolio');p.add_argument('--
194 seed',type=int,default=0)
195     p.add_argument('--migration');p.add_argument('--max-proposals',type=
196 int,default=96)
197     print(json.dumps(run(**vars(p.parse_args()))),ensure_ascii=False,indent
198 =2))

```

A.10 核心实现 7: q3_timed_combination.py

程序 7 q3_timed_combination.py

```

1  """Q2 structural portfolio adapted to actual FIFO-L2 events, bounded wall
2  time."""
3  import argparse,copy,gzip,json,multiprocessing as mp,random,time,traceback
4  from pathlib import Path
5  from collections import defaultdict
6  from q1_io import write_json,sha
7  from q2_timed_portfolio import atomic,objective,generate,family_weights
8  from q2_evaluator import key
9
10 def worker(args,deadline):
11     from q3_solver import load,evaluate,audit_cache,cache_diagnostic
12     from q3_neighborhood import pool
13     from q2_solver import SceneBGraph
14     from q1_structural_seeds import GraphModel
15     out=Path(args['output']);rows=[];seen=set();best=result=None
16     try:

```

```

17     s,d,c,provenance=load();raw=json.loads(Path(args['graph']).
read_text(encoding='utf-8'))
18     atomic(out/'identity.json',dict(problem=3,cache=c,provenance=
provenance,
19         input_sha256=sha(Path(args['graph']).read_bytes()),seed_sha256
=sha(Path(args['seed_plan']).read_bytes()),
20         anchor_sha256=sha(Path(args['anchor']).read_bytes()) if args['
anchor'] else None,
21         warm_start=True,arguments=args))
22     def assess(p,name):
23         nonlocal best,result
24         h=key(p);started=time.monotonic()
25         if h in seen:return 0.
26         seen.add(h)
27         try:
28             r=evaluate(raw,p,s,d,c);audit_cache(r)
29             gain=max(0.,(result['makespan']-r['makespan'])/result['
makespan']) if result else 0.
30             accept=result is None or objective(r)<objective(result)
31             if accept:
32                 stem=f'accepted_{len(rows):03}'
33                 write_json(out/(stem+'.plan.json'),p)
34                 with gzip.open(out/(stem+'.evaluation.json.gz'),'wt',
encoding='utf-8') as f:json.dump(r,f)
35                 atomic(out/'verified.json',dict(plan=stem+'.plan.json'
,evaluation=stem+'.evaluation.json.gz',
36                     makespan=r['makespan'],added_copy_bytes=objective(
r)[1],plan_sha256=h,official_problem=3))
37                 best,result=copy.deepcopy(p),r
38                 rows.append(dict(name=name,status='ok',accepted=accept,
makespan=r['makespan'],
39                     added_copy_bytes=objective(r)[1],hit_bytes=r['
cache_stats']['hit_bytes'],
40                     plan_sha256=h,seconds=time.monotonic()-started))
41             except (ValueError,RuntimeError) as e:
42                 gain=0.;rows.append(dict(name=name,status='invalid',error=
str(e),plan_sha256=h,seconds=time.monotonic()-started))
43                 atomic(out/'search.json',dict(evaluations=rows));return gain
44                 assess(json.loads(Path(args['seed_plan']).read_text(encoding='utf
-8')), 'latest_Q2_in_L2')
45                 if args['anchor'] and time.monotonic()<deadline:
46                     assess(json.loads(Path(args['anchor']).read_text(encoding='utf
-8')), 'previous_Q3_anchor')
47                 if args['arm']=='seed_only':return
48                 g=SceneBGraph(raw,s,d);m=GraphModel(raw,s,dict(
task_cross_core_wait_cycles=d,task_same_core_wait_cycles=0))
49                 weights=family_weights(m) if args['arm']=='combined' else {'joint'
:1}
50                 weights['cache']=2
51                 if args['arm']=='combined':
52                     weights['frontier']=1
53                 if len(m.components)>1:weights['packing']=2

```

```

54         rng=random.Random(args['seed']);counts=defaultdict(int);gains=
defaultdict(float);spent=defaultdict(float)
55         menus={};protected_until=time.monotonic()+.4*max(0,deadline-time.
monotonic())
56         for it in range(args['max_proposals']):
57             costs=[r['seconds'] for r in rows if r['status']=='ok'][-4:]
58             if deadline-time.monotonic()<max(.5,1.3*max(costs,default=.1))
:break
59             unexplored=[f for f in weights if not counts[f]]
60             if it<16 and time.monotonic()<protected_until:family='joint'
if it%2 else 'cache'
61             elif unexplored:family=unexplored[0]
62             else:family=rng.choices(list(weights),[weights[f]*(1+min
(8,100*gains[f]/max(.1,spent[f])) for f in weights])[0]
63             index=counts[family];counts[family]+=1;started=time.monotonic
()
64             try:
65                 if family=='cache':
66                     h=key(best)
67                     if h not in menus:menus[h]=iter(pool(g,best,
cache_diagnostic(result),budget=24))
68                     item=next(menus[h],None)
69                     if item is None:continue
70                     candidate=item['plan']
71                 else:candidate=generate(family,m,g,best,args['cores'],rng,
index)
72                 gains[family]+=assess(candidate,family)
73             except (ValueError,RuntimeError) as e:
74                 rows.append(dict(name=family,status='generation_invalid',
error=str(e)))
75                 finally:spent[family]+=time.monotonic()-started
76                 atomic(out/'search.json',dict(evaluations=rows,family_counts=dict(
counts),family_gain=dict(gains),family_seconds=dict(spent)))
77             except Exception:(out/'error.txt').write_text(traceback.format_exc(),
encoding='utf-8')
78
79
80 def run(graph,seed_plan,output,anchor=None,cores=5,seconds=590,arm='
combined',seed=0,max_proposals=96):
81     if cores not in range(2,6) or not 0<seconds<=590 or arm not in ['
seed_only','local','combined']:raise ValueError('Invalid budget/arm/
cores')
82     started=time.monotonic();out=Path(output);out.mkdir(parents=True,
exist_ok=False)
83     args=dict(graph=str(Path(graph).resolve()),seed_plan=str(Path(
seed_plan).resolve()),output=str(out.resolve()),
84             anchor=str(Path(anchor).resolve()) if anchor else None,cores=cores
,seconds=seconds,arm=arm,seed=seed,max_proposals=max_proposals)
85     p=mp.get_context('spawn').Process(target=worker,args=(args,started+
seconds));p.start()
86     p.join(max(0,seconds-(time.monotonic()-started)));stopped=p.is_alive()
87     if stopped:p.terminate();p.join(2)

```

```

88     if p.is_alive():p.kill();p.join(2)
89     summary=dict(arguments=args,seconds=time.monotonic()-started,
deadline_stop=stopped,worker_exitcode=p.exitcode,
90     worker_error=(out/'error.txt').exists(),status='ok' if (out/'
verified.json').exists() else 'no_verified_solution',
91     source_sha256=sha(Path(__file__).read_bytes()))
92     if (out/'verified.json').exists():summary.update(json.loads((out/'
verified.json').read_text(encoding='utf-8')))
93     atomic(out/'summary.json',summary);return summary
94
95
96 if __name__=='__main__':
97     p=argparse.ArgumentParser();p.add_argument('graph');p.add_argument('--
seed-plan',required=True);p.add_argument('--anchor')
98     p.add_argument('--output',required=True);p.add_argument('--cores',type
=int,default=5);p.add_argument('--seconds',type=float,default=590)
99     p.add_argument('--arm',choices=['seed_only','local','combined'],
default='combined');p.add_argument('--seed',type=int,default=0)
100    p.add_argument('--max-proposals',type=int,default=96)
101    print(json.dumps(run(**vars(p.parse_args()))),ensure_ascii=False))

```

B 问题一逐用例官方结果

加速比为固定单核时间除以本配置时间。

表 14 问题一全部算例的官方评价结果

算例	核数	Makespan	额外搬运量	加速比
case_001	1	233110	0	1.000000
case_001	2	116868	1152	1.994644
case_001	3	78545	2304	2.967853
case_001	4	58984	3456	3.952089
case_001	5	47502	4608	4.907372
case_002	1	261945	0	1.000000
case_002	2	138077	477696	1.897094
case_002	3	95931	532992	2.730556
case_002	4	74618	536064	3.510480
case_002	5	60651	536064	4.318890
case_003	1	602212	0	1.000000
case_003	2	462800	8565562	1.301236
case_003	3	378576	9989392	1.590729
case_003	4	329107	11891502	1.829836
case_003	5	294109	11891502	2.047581
case_004	1	392357	0	1.000000
case_004	2	199074	110592	1.970910
case_004	3	134043	221184	2.927098
case_004	4	104188	331776	3.765856
case_004	5	88341	442368	4.441392
case_005	1	95618	0	1.000000
case_005	2	91571	2718104	1.044195

算例	核数	Makespan	额外搬运量	加速比
case_005	3	88392	2736536	1.081749
case_005	4	87624	2736440	1.091231
case_005	5	87840	2736440	1.088547
case_006	1	82837	16400	1.000000
case_006	2	42379	56200	1.954671
case_006	3	32925	337092	2.515930
case_006	4	31129	533902	2.661088
case_006	5	31729	91264	2.610766
case_007	1	1012562	0	1.000000
case_007	2	507494	27648	1.995220
case_007	3	338094	55296	2.994913
case_007	4	254398	82944	3.980228
case_007	5	203834	110592	4.967581
case_008	1	487605	0	1.000000
case_008	2	244266	0	1.996205
case_008	3	163359	0	2.984868
case_008	4	123060	0	3.962335
case_008	5	100603	0	4.846824
case_009	1	153261	0	1.000000
case_009	2	92259	182950	1.661204
case_009	3	92004	193702	1.665808
case_009	4	92192	225586	1.662411
case_009	5	91903	214604	1.667639
case_010	1	86969	0	1.000000
case_010	2	45626	48384	1.906128
case_010	3	34518	395696	2.519526
case_010	4	30262	575880	2.873868
case_010	5	28842	958646	3.015360
case_011	1	184697	0	1.000000
case_011	2	92646	180224	1.993578
case_011	3	69625	1081344	2.652740
case_011	4	46880	540672	3.939782
case_011	5	47047	1630208	3.925798
case_012	1	59088	0	1.000000
case_012	2	29489	71816	2.003730
case_012	3	20230	43320	2.920811
case_012	4	15724	86400	3.757822
case_012	5	13406	53568	4.407579
case_013	1	263949	0	1.000000
case_013	2	132198	8192	1.996619
case_013	3	88351	16384	2.987504
case_013	4	66982	24576	3.940596
case_013	5	54363	32768	4.855306
case_014	1	17698626	71010720	1.000000
case_014	2	8909568	69998520	1.986474
case_014	3	6002519	84088488	2.948533
case_014	4	4543360	76510104	3.895493

算例	核数	Makespan	额外搬运量	加速比
case_014	5	3694137	88763688	4.791004
case_015	1	177362	180256	1.000000
case_015	2	90480	199944	1.960234
case_015	3	61018	298252	2.906716
case_015	4	48866	365824	3.629558
case_015	5	44068	331776	4.024735
case_016	1	7715523	0	1.000000
case_016	2	5440637	134500220	1.418129
case_016	3	4478897	134500220	1.722639
case_016	4	4266542	134500220	1.808379
case_016	5	4158373	134500220	1.855419
case_017	1	162810	22528	1.000000
case_017	2	81117	110100	2.007101
case_017	3	53799	196920	3.026264
case_017	4	41101	285984	3.961217
case_017	5	33081	359460	4.921556
case_018	1	1004367	1970176	1.000000
case_018	2	501235	2622524	2.003785
case_018	3	336388	4077600	2.985740
case_018	4	253676	4938752	3.959251
case_018	5	203806	5603348	4.928054
case_019	1	81144	0	1.000000
case_019	2	40136	42540	2.021726
case_019	3	27848	83520	2.913818
case_019	4	24465	285312	3.316738
case_019	5	23450	568740	3.460299
case_020	1	532746	0	1.000000
case_020	2	266580	0	1.998447
case_020	3	177950	0	2.993796
case_020	4	133704	0	3.984518
case_020	5	107650	0	4.948871
case_021	1	6567937	4501504	1.000000
case_021	2	3168548	50104320	2.072854
case_021	3	2124156	50071552	3.092022
case_021	4	1550324	4608000	4.236493
case_021	5	1249916	11845632	5.254703
case_022	1	56780	0	1.000000
case_022	2	28303	6912	2.006148
case_022	3	18978	13824	2.991885
case_022	4	14336	20736	3.960658
case_022	5	11479	24612	4.946424
case_023	1	122614	0	1.000000
case_023	2	61065	73728	2.007926
case_023	3	40682	207264	3.013962
case_023	4	30516	281280	4.018023
case_023	5	24940	280992	4.916359
case_024	1	2555343	0	1.000000

算例	核数	Makespan	额外搬运量	加速比
case_024	2	1803647	44243528	1.416764
case_024	3	1483888	44243528	1.722059
case_024	4	1413568	44243528	1.807726
case_024	5	1378992	44243528	1.853051
case_025	1	4863026	0	1.000000
case_025	2	2431840	27648	1.999731
case_025	3	1622164	55296	2.997863
case_025	4	1217444	82944	3.994456
case_025	5	975282	110592	4.986277
case_026	1	99203	0	1.000000
case_026	2	51839	218580	1.913675
case_026	3	42174	381752	2.352231
case_026	4	42089	217872	2.356982
case_026	5	43946	414504	2.257384
case_027	1	2363952	1400832	1.000000
case_027	2	1116984	1843200	2.116371
case_027	3	804739	19496960	2.937539
case_027	4	559196	3317760	4.227412
case_027	5	558753	5529600	4.230764
case_028	1	43836040	277412256	1.000000
case_028	2	20808183	310682880	2.106673
case_028	3	13957556	310848768	3.140667
case_028	4	10676362	310848768	4.105897
case_028	5	9004662	286344264	4.868149
case_029	1	37555	0	1.000000
case_029	2	19740	0	1.902482
case_029	3	12743	0	2.947108
case_029	4	11028	0	3.405423
case_029	5	9383	0	4.002451
case_030	1	2808034	5519360	1.000000
case_030	2	1399339	5285020	2.006686
case_030	3	932367	8365156	3.011726
case_030	4	700342	10404864	4.009518
case_030	5	560306	11168828	5.011608
case_031	1	1414369	3491328	1.000000
case_031	2	712262	5250048	1.985743
case_031	3	473822	7544832	2.985022
case_031	4	356417	7926272	3.968298
case_031	5	284848	7931904	4.965346
case_032	1	41443	0	1.000000
case_032	2	20900	96344	1.982919
case_032	3	14009	96406	2.958313
case_032	4	10933	63656	3.790634
case_032	5	10280	80036	4.031420
case_033	1	1774196	4592640	1.000000
case_033	2	888544	6312520	1.996745
case_033	3	590697	7602264	3.003564

算例	核数	Makespan	额外搬运量	加速比
case_033	4	443447	7214688	4.000920
case_033	5	353592	8227976	5.017636
case_034	1	842996	1079832	1.000000
case_034	2	419167	1915392	2.011122
case_034	3	278067	2762240	3.031629
case_034	4	205314	3207748	4.105887
case_034	5	167930	3885104	5.019925
case_035	1	169571	0	1.000000
case_035	2	95406	0	1.777362
case_035	3	95782	42278	1.770385
case_035	4	94869	47410	1.787423
case_035	5	95983	83532	1.766677
case_036	1	931510	0	1.000000
case_036	2	466068	1152	1.998657
case_036	3	311345	2304	2.991890
case_036	4	233584	3456	3.987902
case_036	5	187182	4608	4.976493
case_037	1	210001	0	1.000000
case_037	2	105374	204800	1.992911
case_037	3	74038	1064960	2.836395
case_037	4	53320	614400	3.938503
case_037	5	43800	1925120	4.794543
case_038	1	1364555	5627904	1.000000
case_038	2	674500	6790656	2.023062
case_038	3	450730	7879736	3.027433
case_038	4	338163	8930344	4.035199
case_038	5	274658	9635364	4.968197
case_039	1	1435262	7929856	1.000000
case_039	2	708557	11550720	2.025613
case_039	3	482318	15020032	2.975759
case_039	4	395323	17604608	3.630606
case_039	5	431161	20480000	3.328831
case_040	1	253454	0	1.000000
case_040	2	120926	283200	2.095943
case_040	3	109777	642836	2.308808
case_040	4	109607	1022796	2.312389
case_040	5	108131	727212	2.343953
case_041	1	6003957	30898176	1.000000
case_041	2	2990398	33669120	2.007745
case_041	3	1991911	34851840	3.014169
case_041	4	1490603	39164928	4.027871
case_041	5	1201884	40673280	4.995455
case_042	1	1686740	6629376	1.000000
case_042	2	845091	6614304	1.995927
case_042	3	559018	8452672	3.017327
case_042	4	419432	8676192	4.021486
case_042	5	336330	10425184	5.015134

算例	核数	Makespan	额外搬运量	加速比
case_043	1	677656	49160	1.000000
case_043	2	353710	589446	1.915852
case_043	3	267112	2381570	2.536973
case_043	4	266590	2584974	2.541941
case_043	5	264401	2875646	2.562986
case_044	1	154407	4438112	1.000000
case_044	2	119062	5274528	1.296862
case_044	3	123085	5678624	1.254475
case_044	4	120120	5792192	1.285440
case_044	5	131771	6309600	1.171783
case_045	1	125614	0	1.000000
case_045	2	63059	0	1.992007
case_045	3	42601	0	2.948616
case_045	4	31878	0	3.940461
case_045	5	26164	0	4.801024
case_046	1	276455	3176288	1.000000
case_046	2	152419	5255872	1.813783
case_046	3	128115	4762304	2.157866
case_046	4	116069	5013664	2.381816
case_046	5	111389	4672800	2.481888
case_047	1	326508	0	1.000000
case_047	2	276535	4380996	1.180711
case_047	3	241969	5051902	1.349379
case_047	4	225654	5078720	1.446941
case_047	5	222583	5064318	1.466904
case_048	1	222220	0	1.000000
case_048	2	197496	1716280	1.125187
case_048	3	197613	1716280	1.124521
case_048	4	197613	1716280	1.124521
case_048	5	197613	1716280	1.124521
case_049	1	196276	0	1.000000
case_049	2	156968	2635492	1.250420
case_049	3	132866	4012736	1.477248
case_049	4	123251	4012736	1.592490
case_049	5	117489	4013760	1.670590
case_050	1	149690	0	1.000000
case_050	2	108323	51814	1.381886
case_050	3	104366	3416202	1.434279
case_050	4	98003	3448972	1.527402
case_050	5	99382	3424394	1.506208
case_051	1	607628	0	1.000000
case_051	2	429872	10225204	1.413509
case_051	3	352843	10225204	1.722092
case_051	4	336378	10225204	1.806384
case_051	5	328989	10225204	1.846955
case_052	1	180301	0	1.000000
case_052	2	90903	102912	1.983444

算例	核数	Makespan	额外搬运量	加速比
case_052	3	60160	181248	2.997025
case_052	4	45777	254976	3.938681
case_052	5	37846	333312	4.764070
case_053	1	1472686	5405536	1.000000
case_053	2	774757	5959552	1.900836
case_053	3	509253	6480220	2.891855
case_053	4	491606	10892964	2.995663
case_053	5	474708	20272004	3.102299
case_054	1	815051	0	1.000000
case_054	2	510200	6569360	1.597513
case_054	3	402111	6732340	2.026930
case_054	4	362394	11175330	2.249074
case_054	5	333071	11208098	2.447079
case_055	1	118835	69632	1.000000
case_055	2	56902	176664	2.088415
case_055	3	38966	168968	3.049710
case_055	4	32118	173568	3.699950
case_055	5	24831	280068	4.785752
case_056	1	277285	0	1.000000
case_056	2	231882	4095676	1.195802
case_056	3	199699	6006214	1.388515
case_056	4	189615	6006214	1.462358
case_056	5	186638	6006214	1.485684
case_057	1	59726	94216	1.000000
case_057	2	30047	182060	1.987753
case_057	3	20691	18228	2.886569
case_057	4	16471	32328	3.626131
case_057	5	14319	189060	4.171101
case_058	1	4539717	28696576	1.000000
case_058	2	2268326	33972224	2.001351
case_058	3	1532367	31514624	2.962552
case_058	4	1152464	42893312	3.939140
case_058	5	1002440	33316864	4.528667
case_059	1	829473	1691648	1.000000
case_059	2	369318	360448	2.245959
case_059	3	254823	4196352	3.255095
case_059	4	185216	1081344	4.478409
case_059	5	150782	4325376	5.501141
case_060	1	888789	2068520	1.000000
case_060	2	441525	2746508	2.012998
case_060	3	295983	3159296	3.002838
case_060	4	223107	3960200	3.983689
case_060	5	180081	3733888	4.935496
case_061	1	77191	0	1.000000
case_061	2	39298	34560	1.964248
case_061	3	28342	43008	2.723555
case_061	4	24024	70656	3.213079

算例	核数	Makespan	额外搬运量	加速比
case_061	5	21941	323072	3.518117
case_062	1	2854809	0	1.000000
case_062	2	1463796	3431424	1.950278
case_062	3	982412	3469824	2.905918
case_062	4	744188	3511296	3.836140
case_062	5	605685	3469824	4.713356
case_063	1	1070397	0	1.000000
case_063	2	552512	1903104	1.937328
case_063	3	371366	1903104	2.882324
case_063	4	283231	1915392	3.779237
case_063	5	229053	1920000	4.673141
case_064	1	19116	0	1.000000
case_064	2	19089	17994	1.001414
case_064	3	19089	17994	1.001414
case_064	4	19076	15692	1.002097
case_064	5	19083	7366	1.001729
case_065	1	53068	0	1.000000
case_065	2	26410	13984	2.009390
case_065	3	17964	27936	2.954130
case_065	4	13800	41904	3.845507
case_065	5	10949	66132	4.846835
case_066	1	359070	0	1.000000
case_066	2	242005	4846614	1.483730
case_066	3	176353	4877334	2.036087
case_066	4	162001	5725374	2.216468
case_066	5	140194	5725374	2.561237
case_067	1	63834834	264449408	1.000000
case_067	2	32203165	262601984	1.982253
case_067	3	21585693	247821184	2.957275
case_067	4	16358776	262601984	3.902177
case_067	5	13455316	262606080	4.744209
case_068	1	362095	0	1.000000
case_068	2	243395	4401532	1.487685
case_068	3	224168	4401532	1.615284
case_068	4	225518	4426108	1.605615
case_068	5	226325	4426684	1.599890
case_069	1	25144	0	1.000000
case_069	2	21862	514378	1.150124
case_069	3	21862	514378	1.150124
case_069	4	21862	514378	1.150124
case_069	5	24830	71044	1.012646
case_070	1	117300	0	1.000000
case_070	2	58498	129536	2.005197
case_070	3	39333	259072	2.982229
case_070	4	30080	333312	3.899601
case_070	5	25374	449024	4.622842
case_071	1	18919	0	1.000000

算例	核数	Makespan	额外搬运量	加速比
case_071	2	18901	93316	1.000952
case_071	3	18757	57220	1.008637
case_071	4	18449	155940	1.025476
case_071	5	18757	57220	1.008637
case_072	1	23897828	108613680	1.000000
case_072	2	11841342	110234400	2.018169
case_072	3	7929522	115280672	3.013779
case_072	4	5990048	120472240	3.989589
case_072	5	4830220	127451824	4.947565
case_073	1	10081626	141570432	1.000000
case_073	2	5168597	124388224	1.950554
case_073	3	3836078	131784832	2.628108
case_073	4	3249411	131636352	3.102601
case_073	5	3017992	124387200	3.340508
case_074	1	1682239	6070280	1.000000
case_074	2	831774	6869040	2.022471
case_074	3	551928	7438392	3.047932
case_074	4	414717	8323084	4.056354
case_074	5	335011	8036364	5.021444
case_075	1	1441809	0	1.000000
case_075	2	1008737	12444746	1.429321
case_075	3	778382	12461130	1.852315
case_075	4	698816	12461130	2.063217
case_075	5	709725	12444746	2.031504
case_076	1	6185113	28280832	1.000000
case_076	2	3065214	31095808	2.017841
case_076	3	2050028	34332672	3.017087
case_076	4	1540201	37442560	4.015783
case_076	5	1236760	38841548	5.001062
case_077	1	495320	0	1.000000
case_077	2	248734	57344	1.991364
case_077	3	168976	114688	2.931304
case_077	4	127832	114688	3.874773
case_077	5	115141	225046	4.301856
case_078	1	2296143	12644568	1.000000
case_078	2	1179848	11466240	1.946135
case_078	3	840459	12294144	2.732011
case_078	4	621588	14093136	3.693995
case_078	5	635493	14093136	3.613168
case_079	1	31052251	32448512	1.000000
case_079	2	14956594	242368512	2.076158
case_079	3	10106197	40280064	3.072595
case_079	4	7609995	31555584	4.080456
case_079	5	6022007	50976768	5.156462
case_080	1	292138	548864	1.000000
case_080	2	145757	2707456	2.004281
case_080	3	100325	3047424	2.911916

算例	核数	Makespan	额外搬运量	加速比
case_080	4	111314	4190208	2.624450
case_080	5	113455	4571136	2.574924
case_081	1	151795	0	1.000000
case_081	2	76860	0	1.974954
case_081	3	51943	0	2.922338
case_081	4	39588	0	3.834369
case_081	5	30943	0	4.905633
case_082	1	721041	0	1.000000
case_082	2	497483	5740350	1.449378
case_082	3	378014	7216752	1.907445
case_082	4	328283	5723710	2.196401
case_082	5	312290	5703230	2.308883
case_083	1	1057993	13934688	1.000000
case_083	2	543456	25772480	1.946787
case_083	3	436908	19450976	2.421546
case_083	4	417384	19407232	2.534819
case_083	5	391729	19753696	2.700829
case_084	1	2507412	0	1.000000
case_084	2	1254320	27648	1.999021
case_084	3	836116	18432	2.998881
case_084	4	629028	0	3.986169
case_084	5	503472	0	4.980241
case_085	1	3187899	0	1.000000
case_085	2	2334501	34301210	1.365559
case_085	3	1626396	34301210	1.960100
case_085	4	1324330	34301210	2.407179
case_085	5	1169044	34301210	2.726928
case_086	1	118403	0	1.000000
case_086	2	102016	2674522	1.160632
case_086	3	97297	3334272	1.216923
case_086	4	98592	3359618	1.200939
case_086	5	99507	3359618	1.189896
case_087	1	3974071	7902720	1.000000
case_087	2	1987606	10142208	1.999426
case_087	3	1374901	11036160	2.890442
case_087	4	1046846	12607488	3.796233
case_087	5	833349	12487680	4.768796
case_088	1	243623	0	1.000000
case_088	2	163881	2935364	1.486585
case_088	3	153688	2932484	1.585179
case_088	4	153194	2957060	1.590291
case_088	5	151301	2932484	1.610188
case_089	1	693092	0	1.000000
case_089	2	346832	238720	1.998351
case_089	3	243614	477440	2.845042
case_089	4	178160	716160	3.890278
case_089	5	147432	954880	4.701096

算例	核数	Makespan	额外搬运量	加速比
case_090	1	1178363	21233592	1.000000
case_090	2	625263	18405960	1.884588
case_090	3	550183	25354368	2.141766
case_090	4	527825	19402440	2.232488
case_090	5	509204	19405512	2.314128
case_091	1	9186769	53947008	1.000000
case_091	2	4571759	57904096	2.009460
case_091	3	3070918	60928832	2.991538
case_091	4	2315897	69360288	3.966830
case_091	5	1880163	73234432	4.886156
case_092	1	4815554	66000096	1.000000
case_092	2	2412929	113053056	1.995730
case_092	3	2109464	112651936	2.282833
case_092	4	2037129	113053056	2.363893
case_092	5	1661234	66980256	2.898781
case_093	1	74205	0	1.000000
case_093	2	37827	8192	1.961694
case_093	3	25103	16384	2.956021
case_093	4	19779	24576	3.751706
case_093	5	15876	32768	4.674036
case_094	1	94892	0	1.000000
case_094	2	47498	106272	1.997810
case_094	3	32728	212544	2.899413
case_094	4	24495	318552	3.873933
case_094	5	21804	424800	4.352045
case_095	1	2084853	0	1.000000
case_095	2	1042787	36864	1.999309
case_095	3	695775	0	2.996447
case_095	4	524422	0	3.975525
case_095	5	420852	0	4.953886
case_096	1	84194	0	1.000000
case_096	2	43782	116620	1.923028
case_096	3	34182	661880	2.463109
case_096	4	27087	719200	3.108281
case_096	5	27053	661824	3.112187
case_097	1	11491509	10584064	1.000000
case_097	2	5247114	85295104	2.190063
case_097	3	3509566	85327872	3.274339
case_097	4	2622226	14221312	4.382349
case_097	5	2101905	22016000	5.467188
case_098	1	177430	0	1.000000
case_098	2	93465	98340	1.898358
case_098	3	79351	1479456	2.236015
case_098	4	79805	1479456	2.223294
case_098	5	73104	3282976	2.427090
case_099	1	1622999	5484544	1.000000
case_099	2	808380	6221840	2.007718

算例	核数	Makespan	额外搬运量	加速比
case_099	3	540127	8245256	3.004847
case_099	4	405336	9838592	4.004083
case_099	5	334609	10674180	4.850434
case_100	1	151223	0	1.000000
case_100	2	80622	16896	1.875704
case_100	3	58758	698388	2.573658
case_100	4	58793	534528	2.572126
case_100	5	59649	534528	2.535214

C 问题二逐用例官方结果

各核数均为 100 图，额外搬运单位为 bytes。

算例	核数	周期	额外搬运	加速比
1	1	233110	0	1.000000
2	1	261945	0	1.000000
3	1	602212	0	1.000000
4	1	392357	0	1.000000
5	1	95618	0	1.000000
6	1	82837	16400	1.000000
7	1	1012562	0	1.000000
8	1	487605	0	1.000000
9	1	153261	0	1.000000
10	1	86969	0	1.000000
11	1	184697	0	1.000000
12	1	59088	0	1.000000
13	1	263949	0	1.000000
14	1	17698626	71010720	1.000000
15	1	177362	180256	1.000000
16	1	7715523	0	1.000000
17	1	162810	22528	1.000000
18	1	1004367	1970176	1.000000
19	1	81144	0	1.000000
20	1	532746	0	1.000000
21	1	6567937	4501504	1.000000
22	1	56780	0	1.000000
23	1	122614	0	1.000000
24	1	2555343	0	1.000000
25	1	4863026	0	1.000000
26	1	99203	0	1.000000
27	1	2363952	1400832	1.000000
28	1	43836040	277412256	1.000000
29	1	37555	0	1.000000
30	1	2808034	5519360	1.000000
31	1	1414369	3491328	1.000000
32	1	41443	0	1.000000
33	1	1774196	4592640	1.000000
34	1	842996	1079832	1.000000
35	1	169571	0	1.000000

算例	核数	周期	额外搬运	加速比
36	1	931510	0	1.000000
37	1	210001	0	1.000000
38	1	1364555	5627904	1.000000
39	1	1435262	7929856	1.000000
40	1	253454	0	1.000000
41	1	6003957	30898176	1.000000
42	1	1686740	6629376	1.000000
43	1	677656	49160	1.000000
44	1	154407	4438112	1.000000
45	1	125614	0	1.000000
46	1	276455	3176288	1.000000
47	1	326508	0	1.000000
48	1	222220	0	1.000000
49	1	196276	0	1.000000
50	1	149690	0	1.000000
51	1	607628	0	1.000000
52	1	180301	0	1.000000
53	1	1472686	5405536	1.000000
54	1	815051	0	1.000000
55	1	118835	69632	1.000000
56	1	277285	0	1.000000
57	1	59726	94216	1.000000
58	1	4539717	28696576	1.000000
59	1	829473	1691648	1.000000
60	1	888789	2068520	1.000000
61	1	77191	0	1.000000
62	1	2854809	0	1.000000
63	1	1070397	0	1.000000
64	1	19116	0	1.000000
65	1	53068	0	1.000000
66	1	359070	0	1.000000
67	1	63834834	264449408	1.000000
68	1	362095	0	1.000000
69	1	25144	0	1.000000
70	1	117300	0	1.000000
71	1	18919	0	1.000000
72	1	23897828	108613680	1.000000
73	1	10081626	141570432	1.000000
74	1	1682239	6070280	1.000000
75	1	1441809	0	1.000000
76	1	6185113	28280832	1.000000
77	1	495320	0	1.000000
78	1	2296143	12644568	1.000000
79	1	31052251	32448512	1.000000
80	1	292138	548864	1.000000
81	1	151795	0	1.000000
82	1	721041	0	1.000000
83	1	1057993	13934688	1.000000
84	1	2507412	0	1.000000
85	1	3187899	0	1.000000

算例	核数	周期	额外搬运	加速比
86	1	118403	0	1.000000
87	1	3974071	7902720	1.000000
88	1	243623	0	1.000000
89	1	693092	0	1.000000
90	1	1178363	21233592	1.000000
91	1	9186769	53947008	1.000000
92	1	4815554	66000096	1.000000
93	1	74205	0	1.000000
94	1	94892	0	1.000000
95	1	2084853	0	1.000000
96	1	84194	0	1.000000
97	1	11491509	10584064	1.000000
98	1	177430	0	1.000000
99	1	1622999	5484544	1.000000
100	1	151223	0	1.000000
1	2	116868	1152	1.994644
2	2	134224	216576	1.951551
3	2	333443	4013282	1.806042
4	2	199074	110592	1.970910
5	2	60847	882310	1.571450
6	2	38351	17280	2.159970
7	2	506754	35328	1.998133
8	2	225487	999552	2.162453
9	2	89967	547342	1.703525
10	2	39899	335304	2.179729
11	2	92646	180224	1.993578
12	2	24958	94960	2.367497
13	2	132198	8192	1.996619
14	2	8859761	74348472	1.997641
15	2	78150	109824	2.269507
16	2	4027025	404200	1.915936
17	2	71079	87552	2.290550
18	2	498153	2194048	2.016182
19	2	38506	354360	2.107308
20	2	266580	0	1.998447
21	2	3128962	7260160	2.099079
22	2	27720	112464	2.048341
23	2	57331	74016	2.138703
24	2	1335245	396856	1.913763
25	2	2431840	27648	1.999731
26	2	46422	74016	2.136982
27	2	1118273	3450880	2.113931
28	2	20486349	75738432	2.139768
29	2	18871	10248	1.990091
30	2	1390496	1292288	2.019448
31	2	712262	5250048	1.985743
32	2	17445	6216	2.375638
33	2	888692	6476832	1.996413
34	2	392369	1008128	2.148478
35	2	93692	801138	1.809877

算例	核数	周期	额外搬运	加速比
36	2	466068	1152	1.998657
37	2	105374	204800	1.992911
38	2	674500	6790656	2.023062
39	2	700340	9998336	2.049379
40	2	134642	1227916	1.882429
41	2	2982616	32735232	2.012984
42	2	840111	7262496	2.007758
43	2	335777	506880	2.018173
44	2	48197	181600	3.203664
45	2	62983	0	1.994411
46	2	112410	930400	2.459345
47	2	197413	1264006	1.653934
48	2	150049	467846	1.480983
49	2	126174	1027062	1.555598
50	2	87435	1022694	1.712015
51	2	319230	394084	1.903418
52	2	86025	102912	2.095914
53	2	723016	4631434	2.036865
54	2	465524	2642570	1.750825
55	2	50549	16384	2.350887
56	2	172836	3326338	1.604324
57	2	29563	26400	2.020296
58	2	2243125	30949376	2.023836
59	2	369318	360448	2.245959
60	2	423532	2264192	2.098517
61	2	39268	34560	1.965748
62	2	1431735	2347008	1.993951
63	2	536070	1049088	1.996749
64	2	13854	46596	1.379818
65	2	21029	13968	2.523563
66	2	203504	1653418	1.764437
67	2	32040130	215994240	1.992340
68	2	196346	1624576	1.844168
69	2	14953	46276	1.681535
70	2	54493	129536	2.152570
71	2	11248	171686	1.681988
72	2	11832440	110063664	2.019687
73	2	5254133	144189824	1.918799
74	2	809182	7266304	2.078938
75	2	754663	4745798	1.910534
76	2	3065214	31095808	2.017841
77	2	247013	57344	2.005239
78	2	1050081	2782080	2.186634
79	2	15118882	194232320	2.053872
80	2	145505	1667072	2.007752
81	2	75991	10248	1.997539
82	2	407535	3389668	1.769274
83	2	430744	2197088	2.456199
84	2	1132696	31812096	2.213667
85	2	1843091	17607566	1.729648

算例	核数	周期	额外搬运	加速比
86	2	73871	1046854	1.602835
87	2	1986290	10142208	2.000751
88	2	132639	1032234	1.836737
89	2	329220	1475712	2.105255
90	2	469626	3240624	2.509152
91	2	4532609	52032556	2.026817
92	2	2013738	10072672	2.391351
93	2	37559	28672	1.975692
94	2	42420	243400	2.236964
95	2	1033019	73728	2.018214
96	2	41628	346520	2.022533
97	2	5152906	2490368	2.230103
98	2	89551	0	1.981329
99	2	803543	5488640	2.019804
100	2	71342	466686	2.119691
1	3	78545	2304	2.967853
2	3	90515	423936	2.893940
3	3	274348	9497892	2.195066
4	3	134043	221184	2.927098
5	3	54097	1475274	1.767529
6	3	27680	52628	2.992666
7	3	338094	55296	2.994913
8	3	154543	516096	3.155141
9	3	72361	737384	2.118006
10	3	29582	48384	2.939930
11	3	63214	1658880	2.921774
12	3	17099	189892	3.455641
13	3	88351	16384	2.987504
14	3	5951263	56526504	2.973928
15	3	56871	208128	3.118672
16	3	2877849	341100	2.681003
17	3	46284	175104	3.517630
18	3	337027	3733504	2.980079
19	3	27532	24576	2.947261
20	3	177888	0	2.994839
21	3	2090922	34854912	3.141168
22	3	18817	13824	3.017484
23	3	38402	206976	3.192907
24	3	954537	332124	2.677050
25	3	1622164	55296	2.997863
26	3	41510	95376	2.389858
27	3	744579	6094848	3.174884
28	3	13687852	44350128	3.202551
29	3	12743	0	2.947108
30	3	932354	7923712	3.011768
31	3	473052	7087104	2.989881
32	3	11828	12360	3.503805
33	3	588739	6949440	3.013553
34	3	262853	2288640	3.207101
35	3	75717	1276568	2.239537

算例	核数	周期	额外搬运	加速比
36	3	311345	2304	2.991890
37	3	71873	790528	2.921834
38	3	449487	7680000	3.035805
39	3	478013	11911168	3.002559
40	3	105492	1985768	2.402590
41	3	1984509	33162240	3.025412
42	3	559150	8323648	3.016615
43	3	250900	4852554	2.700901
44	3	38592	280512	4.001011
45	3	42214	73728	2.975648
46	3	83842	773312	3.297333
47	3	156690	1857324	2.083783
48	3	113666	1177612	1.955026
49	3	83711	2408560	2.344686
50	3	60410	1834792	2.477901
51	3	228581	328736	2.658261
52	3	57758	176640	3.121663
53	3	495435	6090084	2.972511
54	3	359793	4112418	2.265333
55	3	34440	74240	3.450494
56	3	122252	3283132	2.268143
57	3	20565	14112	2.904255
58	3	1488602	32673792	3.049651
59	3	254073	1171456	3.264703
60	3	280590	2404480	3.167572
61	3	27964	43008	2.760370
62	3	958443	2293248	2.978590
63	3	358763	1262592	2.983577
64	3	13074	41446	1.462139
65	3	14696	27792	3.611051
66	3	146357	2500738	2.453385
67	3	21240727	161681152	3.005304
68	3	180284	2355384	2.008470
69	3	12637	362890	1.989713
70	3	39333	259072	2.982229
71	3	9285	241448	2.037588
72	3	7939626	121293600	3.009944
73	3	3805931	134138624	2.648925
74	3	542996	5619712	3.098069
75	3	557783	12507404	2.584892
76	3	2046322	17912832	3.022551
77	3	168976	114688	2.931304
78	3	718637	7071360	3.195136
79	3	10013937	35385344	3.100903
80	3	98725	3047424	2.959109
81	3	51943	0	2.922338
82	3	288553	5338828	2.498817
83	3	290539	6672576	3.641484
84	3	831104	0	3.016965
85	3	1276351	31680716	2.497666

算例	核数	周期	额外搬运	加速比
86	3	63229	2197292	1.872606
87	3	1374901	11036160	2.890442
88	3	122079	1572244	1.995618
89	3	225282	3951872	3.076553
90	3	339896	5560704	3.466834
91	3	3042680	60113508	3.019302
92	3	1337266	20259008	3.601044
93	3	25103	16384	2.956021
94	3	30976	212544	3.063404
95	3	677299	703104	3.078187
96	3	28731	123264	2.930424
97	3	3444086	59834368	3.336592
98	3	64378	418684	2.756066
99	3	538274	4403200	3.015191
100	3	55427	423036	2.728327
1	4	58984	3456	3.952089
2	4	68370	387072	3.831286
3	4	217774	7333194	2.765307
4	4	103756	331776	3.781536
5	4	52228	1498226	1.830780
6	4	23190	514498	3.572100
7	4	254398	82944	3.980228
8	4	100137	1033728	4.869379
9	4	61974	845826	2.472989
10	4	24940	495644	3.487129
11	4	46880	540672	3.939782
12	4	13138	38592	4.497488
13	4	66982	24576	3.940596
14	4	4496941	54943128	3.935703
15	4	48866	365824	3.629558
16	4	2244367	605676	3.437728
17	4	37467	259200	4.345424
18	4	253579	4606976	3.960766
19	4	21217	305508	3.824480
20	4	133635	180224	3.986575
21	4	1550324	4608000	4.236493
22	4	14224	20736	3.991845
23	4	29500	280992	4.156407
24	4	746863	595068	3.421435
25	4	1217444	82944	3.994456
26	4	32505	549760	3.051930
27	4	559196	1843200	4.227412
28	4	10335263	73292160	4.241405
29	4	10103	63508	3.717213
30	4	697351	6002688	4.026715
31	4	359862	7811072	3.930309
32	4	9309	74980	4.451928
33	4	441967	7114848	4.014318
34	4	198751	3059200	4.241468
35	4	63767	1505978	2.659228

算例	核数	周期	额外搬运	加速比
36	4	233584	3456	3.987902
37	4	53320	614400	3.938503
38	4	337826	8947200	4.039224
39	4	368961	14536704	3.890010
40	4	89539	1836692	2.830655
41	4	1489666	38066176	4.030405
42	4	422994	8641376	3.987622
43	4	207895	5894554	3.259607
44	4	37419	375328	4.126433
45	4	31878	0	3.940461
46	4	71284	2131744	3.878220
47	4	142700	2257042	2.288073
48	4	114384	1270930	1.942754
49	4	75814	2607184	2.588915
50	4	57912	2053712	2.584784
51	4	182222	591064	3.334548
52	4	44007	259584	4.097098
53	4	394500	8679750	3.733044
54	4	274779	7279264	2.966206
55	4	29368	74240	4.046411
56	4	115358	3688062	2.403691
57	4	16345	28224	3.654084
58	4	1126941	37941248	4.028354
59	4	185216	1081344	4.478409
60	4	209972	2615552	4.232893
61	4	22111	51456	3.491068
62	4	722454	2603520	3.951544
63	4	270138	1420800	3.962408
64	4	10069	131980	1.898500
65	4	13004	41760	4.080898
66	4	116226	3806464	3.089412
67	4	16409509	226600576	3.890112
68	4	156922	2662918	2.307484
69	4	11292	221642	2.226709
70	4	29290	333312	4.004780
71	4	9235	298702	2.048619
72	4	5949999	125192992	4.016442
73	4	3295138	137012864	3.059546
74	4	408908	6623232	4.113979
75	4	462730	11096688	3.115875
76	4	1527039	19563520	4.050396
77	4	127386	114688	3.888339
78	4	540614	6975360	4.247287
79	4	7498380	37787648	4.141195
80	4	90584	1904640	3.225051
81	4	39588	0	3.834369
82	4	235778	6420240	3.058135
83	4	220202	6953760	4.804648
84	4	616840	580608	4.064931
85	4	995687	27902578	3.201708

算例	核数	周期	额外搬运	加速比
86	4	58864	2230254	2.011467
87	4	1046846	12607488	3.796233
88	4	112497	2044228	2.165596
89	4	165365	1827200	4.191286
90	4	291652	3457848	4.040305
91	4	2314443	67112896	3.969322
92	4	1015491	17357600	4.742094
93	4	19511	180224	3.803239
94	4	23186	318240	4.092642
95	4	506597	0	4.115407
96	4	22385	368320	3.761179
97	4	2622394	14221312	4.382068
98	4	53404	316638	3.322410
99	4	404441	7999488	4.012944
100	4	47125	778698	3.208976
1	5	47502	4608	4.907372
2	5	55153	514560	4.749424
3	5	199233	7908308	3.022652
4	5	86076	3185664	4.558262
5	5	46194	1671860	2.069923
6	5	22215	295130	3.728877
7	5	203834	110592	4.967581
8	5	88631	1327104	5.501518
9	5	56088	898504	2.732510
10	5	22006	562432	3.952059
11	5	45658	1984512	4.045228
12	5	11971	24768	4.935928
13	5	54363	32768	4.855306
14	5	3663108	62443416	4.831587
15	5	40523	317952	4.376823
16	5	2088965	444880	3.693467
17	5	30276	331776	5.377527
18	5	201919	3342336	4.974108
19	5	17710	238688	4.581818
20	5	107118	0	4.973450
21	5	1241101	5120000	5.292025
22	5	11479	20736	4.946424
23	5	24735	270624	4.957105
24	5	695849	432232	3.672267
25	5	975282	110592	4.986277
26	5	27735	316396	3.576816
27	5	456093	17250304	5.183048
28	5	8286707	73953792	5.289923
29	5	8493	49164	4.421877
30	5	559882	7484416	5.015403
31	5	284848	7931904	4.965346
32	5	9937	12288	4.170575
33	5	355089	7690368	4.996483
34	5	161651	2717184	5.214914
35	5	58308	1058710	2.908194

算例	核数	周期	额外搬运	加速比
36	5	187182	4608	4.976493
37	5	43013	819200	4.882268
38	5	275008	9543168	4.961874
39	5	308840	8556544	4.647267
40	5	78337	2144676	3.235432
41	5	1195784	39892992	5.020938
42	5	335450	9936736	5.028290
43	5	199041	6609206	3.404605
44	5	38080	406912	4.054806
45	5	25798	0	4.869137
46	5	69008	750976	4.006130
47	5	135565	2575320	2.408498
48	5	101585	1273560	2.187528
49	5	76809	2912076	2.555378
50	5	56538	1337182	2.647600
51	5	170016	427456	3.573946
52	5	36733	333312	4.908420
53	5	357068	9823040	4.124385
54	5	257061	8197794	3.170652
55	5	24809	115712	4.789996
56	5	117134	4124830	2.367246
57	5	14028	14112	4.257628
58	5	919506	35348480	4.937126
59	5	150736	2326528	5.502819
60	5	170335	2681216	5.217888
61	5	18159	110592	4.250840
62	5	579973	2746368	4.922314
63	5	218333	1563648	4.902589
64	5	11391	159314	1.678167
65	5	10945	55872	4.848607
66	5	106067	3979388	3.385313
67	5	13438701	229484544	4.750075
68	5	130876	3208994	2.766703
69	5	12621	441008	1.992235
70	5	25365	449024	4.624483
71	5	9822	106180	1.926186
72	5	4826156	77245216	4.951731
73	5	2848898	94477312	3.538781
74	5	328499	7151616	5.120987
75	5	378054	12427608	3.813765
76	5	1227673	22311936	5.038079
77	5	111374	172032	4.447358
78	5	465017	6643200	4.937761
79	5	6023008	41365504	5.155605
80	5	85701	2539520	3.408805
81	5	30943	0	4.905633
82	5	203392	4495412	3.545080
83	5	181378	3690880	5.833083
84	5	495292	0	5.062492
85	5	890510	38398484	3.579858

算例	核数	周期	额外搬运	加速比
86	5	52711	2218740	2.246267
87	5	832912	12648960	4.771298
88	5	112814	2047240	2.159510
89	5	133148	2186320	5.205426
90	5	267434	7981056	4.406182
91	5	1872567	71035764	4.905976
92	5	826308	19945856	5.827795
93	5	15876	32768	4.674036
94	5	21745	424800	4.363854
95	5	412506	629376	5.054116
96	5	22134	448896	3.803831
97	5	2102348	21311488	5.466036
98	5	47355	1166870	3.746806
99	5	329580	6850580	4.924446
100	5	45410	1468652	3.330170

D 问题三最终方案的同方案成对结果

每行的无 L2 与 L2 周期来自同一最终方案的独立官方评价，两次额外逻辑搬运相同，表中合列一次；命中率为官方字节命中率。

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
1	1	233110	233110	0	0.000000
2	1	261945	261945	0	0.000000
3	1	602212	602212	0	0.000000
4	1	392357	392357	0	0.000000
5	1	95618	95618	0	0.000000
6	1	82837	82837	16400	0.029275
7	1	1012562	1012562	0	0.000000
8	1	487605	487605	0	0.000000
9	1	153261	153261	0	0.000000
10	1	86969	86969	0	0.000000
11	1	184697	184697	0	0.000000
12	1	59088	59088	0	0.000000
13	1	263949	263949	0	0.000000
14	1	17698626	17689602	71010720	0.122910
15	1	177362	177362	180256	0.169269
16	1	7715523	7715523	0	0.000000
17	1	162810	162810	22528	0.040398
18	1	1004367	1003110	1970176	0.343750
19	1	81144	81144	0	0.000000
20	1	532746	532746	0	0.000000
21	1	6567937	6558837	4501504	0.500597
22	1	56780	56780	0	0.000000
23	1	122614	122614	0	0.000000
24	1	2555343	2555343	0	0.000000
25	1	4863026	4863026	0	0.000000
26	1	99203	99203	0	0.000000
27	1	2363952	2359515	1400832	0.268939
28	1	43836040	43836040	277412256	0.000000

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
29	1	37555	37555	0	0.000000
30	1	2808034	2807891	5519360	0.259868
31	1	1414369	1414159	3491328	0.260594
32	1	41443	41443	0	0.000000
33	1	1774196	1774080	4592640	0.096277
34	1	842996	842733	1079832	0.309047
35	1	169571	169571	0	0.000000
36	1	931510	931510	0	0.000000
37	1	210001	210001	0	0.000000
38	1	1364555	1363312	5627904	0.173617
39	1	1435262	1435262	7929856	0.000000
40	1	253454	253454	0	0.000000
41	1	6003957	6001575	30898176	0.101637
42	1	1686740	1675633	6629376	0.318692
43	1	677656	677656	49160	0.033207
44	1	154407	107814	4438112	0.819989
45	1	125614	125614	0	0.000000
46	1	276455	244994	3176288	0.753030
47	1	326508	326508	0	0.000000
48	1	222220	222220	0	0.000000
49	1	196276	196276	0	0.000000
50	1	149690	149690	0	0.000000
51	1	607628	607628	0	0.000000
52	1	180301	180301	0	0.000000
53	1	1472686	1428109	5405536	0.624871
54	1	815051	815051	0	0.000000
55	1	118835	118835	69632	0.100143
56	1	277285	277285	0	0.000000
57	1	59726	59726	94216	0.218682
58	1	4539717	4539717	28696576	0.000000
59	1	829473	815938	1691648	0.610044
60	1	888789	887801	2068520	0.423143
61	1	77191	77191	0	0.000000
62	1	2854809	2854809	0	0.000000
63	1	1070397	1070397	0	0.000000
64	1	19116	19116	0	0.000000
65	1	53068	53068	0	0.000000
66	1	359070	359070	0	0.000000
67	1	63834834	63834834	264449408	0.000000
68	1	362095	362095	0	0.000000
69	1	25144	25144	0	0.000000
70	1	117300	117300	0	0.000000
71	1	18919	18919	0	0.000000
72	1	23897828	23890756	108613680	0.026210
73	1	10081626	10081626	141570432	0.000000
74	1	1682239	1678120	6070280	0.166805
75	1	1441809	1441809	0	0.000000
76	1	6185113	6183098	28280832	0.133577
77	1	495320	495320	0	0.000000
78	1	2296143	2296143	12644568	0.000000

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
79	1	31052251	31052251	32448512	0.000000
80	1	292138	291826	548864	0.168614
81	1	151795	151795	0	0.000000
82	1	721041	721041	0	0.000000
83	1	1057993	946145	13934688	0.795868
84	1	2507412	2507412	0	0.000000
85	1	3187899	3187899	0	0.000000
86	1	118403	118403	0	0.000000
87	1	3974071	3969779	7902720	0.183368
88	1	243623	243623	0	0.000000
89	1	693092	693092	0	0.000000
90	1	1178363	1178363	21233592	0.000000
91	1	9186769	9168820	53947008	0.105386
92	1	4815554	4333631	66000096	0.793550
93	1	74205	74205	0	0.000000
94	1	94892	94892	0	0.000000
95	1	2084853	2084853	0	0.000000
96	1	84194	84194	0	0.000000
97	1	11491509	11491509	10584064	0.000000
98	1	177430	177430	0	0.000000
99	1	1622999	1618902	5484544	0.252355
100	1	151223	151223	0	0.000000
1	2	116868	116868	1152	0.000000
2	2	132388	132388	230400	0.040495
3	2	333443	333370	4013282	0.092458
4	2	199074	199074	110592	0.000000
5	2	60854	60660	882310	0.128335
6	2	38717	38717	16256	0.029053
7	2	506606	506606	104448	0.016103
8	2	224747	224747	1036416	0.000000
9	2	89590	89589	542126	0.058753
10	2	40668	40452	286920	0.122968
11	2	92646	92646	180224	0.000000
12	2	24280	24280	70380	0.020855
13	2	132198	132198	8192	0.000000
14	2	8904271	8895264	74110392	0.226391
15	2	82522	82522	183568	0.079817
16	2	4027025	4026193	404200	0.496532
17	2	66218	66218	87552	0.022201
18	2	498345	497046	3437568	0.460276
19	2	37458	37458	387128	0.115873
20	2	266580	266580	0	0.000000
21	2	3137808	3137119	7327744	0.399746
22	2	27826	27826	145236	0.074916
23	2	57018	56835	74016	0.145661
24	2	1335245	1334413	396856	0.498846
25	2	2431840	2431840	27648	0.000000
26	2	44686	44686	109756	0.115555
27	2	1117252	1117252	1634304	0.106007
28	2	20486349	20474273	75738432	0.038278

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
29	2	18880	18880	16388	0.000000
30	2	1390496	1390292	1292288	0.345096
31	2	712262	711649	5250048	0.430542
32	2	17378	17378	6216	0.021782
33	2	889278	887618	6648864	0.210126
34	2	390278	389539	943616	0.288909
35	2	93684	93683	801138	0.154664
36	2	466068	466068	1152	0.000000
37	2	105374	105374	204800	0.000000
38	2	674500	674382	6790656	0.278831
39	2	701354	697932	9785344	0.413763
40	2	134642	134149	1227916	0.127347
41	2	2986252	2982392	31100928	0.167959
42	2	840111	839453	7262496	0.412962
43	2	342932	342932	477184	0.250073
44	2	55792	46025	895072	0.379346
45	2	62983	62983	0	0.000000
46	2	112310	112310	991840	0.333987
47	2	197413	197408	1264006	0.106421
48	2	149459	149459	467846	0.145220
49	2	125566	125566	1047286	0.034291
50	2	87003	86990	1022694	0.045964
51	2	319230	318398	394084	0.499724
52	2	84855	84815	102912	0.082440
53	2	722906	722908	4636042	0.239649
54	2	465795	465795	2620302	0.055908
55	2	49447	49343	16384	0.025517
56	2	172793	172799	3326338	0.045065
57	2	29563	29563	18208	0.051320
58	2	2230952	2228121	55287808	0.077340
59	2	369318	369318	360448	0.000000
60	2	428990	428579	2675840	0.445239
61	2	39268	39173	34560	0.059731
62	2	1431735	1431735	2297856	0.001873
63	2	536070	536070	1049088	0.005688
64	2	13858	13858	47620	0.025581
65	2	21281	21281	13968	0.029839
66	2	202704	202704	1653418	0.091560
67	2	32105013	31763756	215908736	0.237436
68	2	196346	196325	1624576	0.101324
69	2	14880	14851	348422	0.077256
70	2	53322	53223	129536	0.235780
71	2	11260	11204	162950	0.078917
72	2	11807716	11798958	114166560	0.105033
73	2	5576908	5041517	85522816	0.193491
74	2	810616	810220	3997696	0.363013
75	2	754663	754663	4745798	0.104451
76	2	3065214	3063843	31095808	0.278782
77	2	247101	247101	57344	0.026082
78	2	1050081	1049840	2782080	0.332127

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
79	2	15116703	15116703	193798144	0.001092
80	2	145475	145475	1794048	0.379845
81	2	76860	76860	0	0.000000
82	2	407392	407358	3389412	0.073348
83	2	433526	425955	2180704	0.198197
84	2	1122732	1118852	24488448	0.054900
85	2	1842459	1841833	17607566	0.075254
86	2	72593	71886	1069062	0.147349
87	2	1982000	1980853	9455616	0.259558
88	2	132030	131481	1033770	0.092347
89	2	329216	329216	1475712	0.079996
90	2	463664	452647	3196848	0.362019
91	2	4532609	4531465	52032556	0.352324
92	2	2052957	2009155	20225120	0.576965
93	2	37559	37559	28672	0.013841
94	2	42420	42420	243400	0.059393
95	2	1033019	1033019	110592	0.000000
96	2	42010	42003	346520	0.277275
97	2	5152906	5152906	2490368	0.009021
98	2	89620	89620	0	0.000000
99	2	803794	802533	6561792	0.409806
100	2	70331	69982	462338	0.277587
1	3	78545	78545	2304	0.000000
2	3	90565	90565	423936	0.000000
3	3	274348	265160	9497892	0.217401
4	3	134043	134043	221184	0.000000
5	3	54077	52419	1479690	0.243845
6	3	26792	26792	58776	0.087833
7	3	338094	338094	55296	0.000000
8	3	157239	157239	0	0.000000
9	3	71678	71649	740904	0.087123
10	3	29582	29582	48384	0.047949
11	3	63333	63333	1658880	0.208115
12	3	17926	17926	189892	0.067289
13	3	88351	88351	16384	0.000000
14	3	5918421	5906299	50501016	0.390001
15	3	54310	54286	208128	0.130506
16	3	2877849	2877209	341100	0.451189
17	3	45302	45302	175104	0.108135
18	3	335653	334510	3143680	0.462892
19	3	27188	27188	24576	0.000000
20	3	177888	177888	0	0.000000
21	3	2090922	2090922	34854912	0.016546
22	3	18852	18852	6912	0.031354
23	3	38878	38878	207264	0.323150
24	3	954537	953897	332124	0.453425
25	3	1622164	1622164	55296	0.000000
26	3	41510	41510	95664	0.163654
27	3	744579	744579	6094848	0.252547
28	3	13694984	13683867	39800832	0.132294

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
29	3	12743	12743	0	0.000000
30	3	932659	932670	8620032	0.492515
31	3	473822	473052	7544832	0.499810
32	3	11773	11773	12360	0.042399
33	3	589213	587856	6964800	0.326359
34	3	261986	261876	2288640	0.488396
35	3	75100	75026	1254808	0.196197
36	3	311345	311345	2304	0.000000
37	3	72397	72397	1687552	0.419204
38	3	449590	449048	8107008	0.344232
39	3	479217	475590	12500992	0.272683
40	3	105492	105156	1985768	0.217865
41	3	1986870	1985351	34842624	0.190303
42	3	563361	559213	8588864	0.392938
43	3	250648	250445	4852554	0.323330
44	3	42170	37575	537536	0.259560
45	3	42214	42214	0	0.000000
46	3	83842	83675	773312	0.235543
47	3	156780	156551	1863980	0.216684
48	3	113666	113339	1177612	0.132880
49	3	83711	83597	2408560	0.098889
50	3	60161	59766	1828552	0.124593
51	3	228581	227941	328736	0.454277
52	3	57038	57038	181248	0.075925
53	3	495343	494343	6090084	0.255514
54	3	359071	359075	4093218	0.120931
55	3	34195	33849	74240	0.106067
56	3	121923	121783	3274876	0.093998
57	3	20565	20565	14112	0.000821
58	3	1491108	1486029	32583680	0.199331
59	3	254073	253977	1171456	0.507273
60	3	295948	295328	3080448	0.447323
61	3	26332	26332	43008	0.073262
62	3	955896	955896	2268672	0.000000
63	3	358583	358583	1262592	0.013478
64	3	13031	13031	41446	0.136616
65	3	15330	15330	27936	0.085260
66	3	144914	143677	2491906	0.218192
67	3	21240727	20710881	161681152	0.667591
68	3	180284	179131	2355384	0.184746
69	3	12716	12339	361994	0.207627
70	3	39333	39333	259072	0.272994
71	3	9349	8875	248488	0.271428
72	3	7933930	7926463	116640544	0.126407
73	3	3784981	3445658	132268800	0.344227
74	3	544370	541942	8257536	0.290673
75	3	557783	554461	12507404	0.233541
76	3	2046296	2045044	17794048	0.550543
77	3	168923	168923	114688	0.025419
78	3	718637	707111	7071360	0.438614

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
79	3	9964215	9962808	45273088	0.271568
80	3	101086	100325	2412544	0.411765
81	3	51202	51202	20496	0.011579
82	3	288690	287402	5338828	0.172505
83	3	290539	290539	6672576	0.249963
84	3	831104	831104	0	0.000000
85	3	1276027	1264447	31684812	0.200416
86	3	63144	61666	2190284	0.182610
87	3	1374975	1372337	11036160	0.261725
88	3	121527	118465	1620500	0.216949
89	3	225282	225282	3951872	0.125018
90	3	335455	308906	5730048	0.489213
91	3	3042680	3029129	60113508	0.444121
92	3	1336815	1337343	20263104	0.135594
93	3	25103	25103	16384	0.000000
94	3	30976	30976	212544	0.243414
95	3	677299	677299	703104	0.000000
96	3	27764	27764	66240	0.039580
97	3	3444086	3444086	59834368	0.087900
98	3	64378	64372	418684	0.172512
99	3	536907	535436	7561216	0.447950
100	3	57811	57811	72192	0.017127
1	4	58984	58984	3456	0.000000
2	4	68574	68574	377856	0.000000
3	4	217719	210143	7341394	0.300544
4	4	103972	103972	331776	0.000000
5	4	52014	48865	1502642	0.298709
6	4	22211	22211	309062	0.183606
7	4	254398	254398	82944	0.000000
8	4	100137	100137	1033728	0.000000
9	4	62725	60949	838982	0.186730
10	4	25602	24531	1005300	0.340036
11	4	46880	46880	540672	0.000000
12	4	13549	13549	39168	0.074853
13	4	66982	66946	24576	0.006000
14	4	4446784	4441943	95588760	0.157408
15	4	46552	46137	231168	0.117026
16	4	2244367	2240364	605676	0.596428
17	4	38607	37813	255744	0.261224
18	4	254107	253273	4320256	0.551012
19	4	21120	21120	352868	0.175382
20	4	133704	133704	0	0.000000
21	4	1550324	1550324	4608000	0.000000
22	4	14245	14245	13824	0.060801
23	4	28698	28698	547992	0.199213
24	4	748435	744432	595068	0.598810
25	4	1217444	1217444	82944	0.000000
26	4	31181	30464	539828	0.211558
27	4	559196	559196	1843200	0.000000
28	4	10367033	10166935	79760256	0.548329

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
29	4	9857	9857	65600	0.202459
30	4	697351	697321	6002688	0.701366
31	4	360157	358663	7823840	0.445867
32	4	10285	10285	95460	0.037322
33	4	441987	441813	7131744	0.359016
34	4	202711	196762	2951680	0.390875
35	4	63882	62988	1502522	0.290606
36	4	233584	233584	3456	0.000000
37	4	53320	53320	614400	0.000000
38	4	337826	337208	8947200	0.338420
39	4	369254	355737	14848000	0.573640
40	4	89275	87298	1878644	0.269201
41	4	1490603	1487676	39164928	0.343802
42	4	429998	424220	8639328	0.526487
43	4	207784	204016	5891482	0.367064
44	4	38436	34177	589088	0.270154
45	4	31871	31871	0	0.000000
46	4	76947	63321	2347808	0.315230
47	4	142700	142139	2257042	0.281333
48	4	114384	114072	1270930	0.196958
49	4	75814	75141	2607184	0.118947
50	4	57895	56820	2040400	0.152724
51	4	183794	179791	591064	0.599715
52	4	44506	43391	259584	0.216838
53	4	394500	388744	8679750	0.222102
54	4	274306	272727	7284480	0.145037
55	4	29269	29269	74240	0.094363
56	4	115534	114420	3675902	0.139754
57	4	16345	16345	28224	0.077366
58	4	1130116	1118404	38928384	0.284507
59	4	185216	185216	1081344	0.000000
60	4	217728	217158	3320192	0.450924
61	4	22111	22085	53760	0.079224
62	4	722454	722454	2582016	0.001228
63	4	270138	270138	1420800	0.008567
64	4	10007	9994	131980	0.220020
65	4	11464	11338	41616	0.108421
66	4	116226	112359	3806464	0.223124
67	4	16614171	16091598	224860288	0.378489
68	4	156693	152583	2646854	0.087615
69	4	14177	13520	361896	0.293207
70	4	29290	29272	333312	0.244046
71	4	10259	9541	223564	0.232073
72	4	5949999	5944202	125192992	0.144673
73	4	3368258	2804718	138700416	0.362646
74	4	408908	408908	6623232	0.417092
75	4	462730	457083	11096688	0.246572
76	4	1527039	1526713	19563520	0.552634
77	4	126549	126308	237568	0.052932
78	4	540759	531949	6950784	0.698760

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
79	4	7459488	7456209	51107840	0.344727
80	4	83835	80643	1994752	0.289130
81	4	38500	38500	127080	0.074708
82	4	235608	232001	6416144	0.244128
83	4	220202	217945	6953760	0.332197
84	4	617896	617896	9216	0.000000
85	4	995687	989911	27902578	0.186296
86	4	58950	55397	2204494	0.298368
87	4	1046832	1042938	12655552	0.244062
88	4	108932	107657	1823044	0.111847
89	4	165365	165365	1827200	0.102735
90	4	296947	258453	7548576	0.384746
91	4	2314443	2289607	67112896	0.485453
92	4	1015537	1003903	18235168	0.214281
93	4	19511	19472	180224	0.024242
94	4	25518	22561	318528	0.333266
95	4	506597	506597	0	0.000000
96	4	22689	22385	368316	0.214551
97	4	2622394	2622413	14221312	0.090315
98	4	53273	53262	316638	0.124193
99	4	408342	403320	8929280	0.506331
100	4	50101	46534	1418568	0.423464
1	5	47502	47502	4608	0.000000
2	5	55153	55153	514560	0.003109
3	5	199233	187997	7908308	0.339473
4	5	85698	83248	3231744	0.230198
5	5	46279	42856	1706228	0.282472
6	5	21324	20637	296160	0.166080
7	5	203834	203834	110592	0.000000
8	5	92069	92069	516864	0.000000
9	5	56088	53324	898504	0.234920
10	5	22516	20828	563780	0.291198
11	5	41774	41082	839680	0.455793
12	5	13248	11971	47232	0.088894
13	5	54363	54117	32768	0.005976
14	5	3606343	3563208	59885976	0.487973
15	5	39126	38214	317952	0.073664
16	5	2088965	2086755	444880	0.516272
17	5	30911	30911	336384	0.355139
18	5	201988	200642	3372672	0.590779
19	5	17271	17271	267360	0.245941
20	5	107256	107256	0	0.000000
21	5	1241101	1241101	5120000	0.249315
22	5	11479	11479	20736	0.088511
23	5	24361	24361	280992	0.255467
24	5	695849	693639	432232	0.518754
25	5	975282	975282	110592	0.000000
26	5	26487	26486	316400	0.237717
27	5	457150	455888	17268736	0.255402
28	5	8334300	8163258	73953792	0.599495

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
29	5	8458	8458	45076	0.014488
30	5	559420	559431	8979968	0.549993
31	5	284862	283939	7900160	0.484815
32	5	9937	9936	12288	0.042162
33	5	356282	355844	5639232	0.281730
34	5	165651	160224	3487744	0.557829
35	5	58308	57848	1058710	0.272976
36	5	187182	187182	4608	0.000000
37	5	43013	43013	860160	0.327941
38	5	275163	272531	9538560	0.407899
39	5	312806	299122	12238848	0.344046
40	5	77348	76698	1927588	0.317660
41	5	1195788	1192741	39894016	0.274585
42	5	335450	335061	9936736	0.509124
43	5	199245	182034	6592822	0.390252
44	5	38879	35982	452992	0.188355
45	5	25722	25722	0	0.000000
46	5	69320	67778	750976	0.196320
47	5	135565	134070	2575320	0.331166
48	5	101604	101557	1273560	0.170384
49	5	76843	74586	2894220	0.153636
50	5	56538	55970	1337182	0.139994
51	5	170016	167806	427456	0.519702
52	5	38113	36766	304128	0.179908
53	5	357373	331284	9857696	0.451058
54	5	256867	252882	8198466	0.166230
55	5	24831	24831	173568	0.165273
56	5	117211	115035	4129054	0.166860
57	5	14028	14028	14112	0.040240
58	5	931102	905225	40996864	0.279374
59	5	150782	150736	2072576	0.544156
60	5	168515	166686	3510656	0.513562
61	5	18351	17886	110592	0.143578
62	5	579973	579973	2746368	0.000000
63	5	218333	218333	1563648	0.003840
64	5	11362	11313	159314	0.230994
65	5	10829	10530	55728	0.117891
66	5	105645	99989	3969404	0.227325
67	5	13558894	12923295	228327936	0.494676
68	5	128131	116565	3414178	0.191845
69	5	12831	11650	443024	0.285109
70	5	25367	25262	449024	0.269293
71	5	10080	9550	309518	0.307527
72	5	4779320	4771746	121799344	0.113980
73	5	2939947	2480426	93595904	0.411840
74	5	328499	328279	7151616	0.471513
75	5	378054	364732	12427608	0.261490
76	5	1230397	1226477	31962112	0.357131
77	5	111222	111222	172032	0.074367
78	5	470359	435870	10766304	0.698241

算例	核数	无 L2 周期	L2 周期	额外搬运	命中率
79	5	5999242	5993296	54609920	0.297258
80	5	97924	71493	2920448	0.442275
81	5	30943	30943	0	0.000000
82	5	206876	200449	7712376	0.326444
83	5	200940	177046	7756160	0.441417
84	5	495240	495240	0	0.000000
85	5	890475	846640	38398484	0.276150
86	5	52595	48379	2221332	0.305539
87	5	832912	829449	12648960	0.278387
88	5	110791	108035	1831812	0.082518
89	5	133148	133139	2186320	0.142902
90	5	274501	225215	8826240	0.572609
91	5	1872567	1837577	71035764	0.533593
92	5	819013	801052	20375936	0.270768
93	5	15876	15628	61440	0.036304
94	5	20178	19391	424800	0.391765
95	5	412506	412506	629376	0.000000
96	5	22134	21963	448896	0.309014
97	5	2102348	2102250	21311488	0.245235
98	5	47066	45067	1175058	0.307458
99	5	330667	324458	9244672	0.459041
100	5	45651	41579	1468656	0.411372